Aula 02 - Prof. Vinícius Borges e Raphael Lacerda

SEFAZ-AL (Auditor Fiscal da Administração Tributária Estadual - AFTE) Desenvolvimento de Sistemas - 2026 (Pós-Edital)

Autor:
Vinicius Borges

27 de Agosto de 2026

02648216103 - HENRIQUE AMORIM BARRETO

Índice

1) JavaScript - Teoria 3

2) JavaScript - Questões Comentadas 86

3) JavaScript - Lista de Questões 141

JAVASCRIPT

Panorama Geral

JavaScript (JS) é a linguagem de programação mais popular do mundo. Ela é responsável por desenvolver quase que a totalidade dos aplicativos Web (ou web apps). O JS é uma linguagem que lida diretamente com o HTML e o CSS, criando interações dinâmicas e interfaces de usuário, transformando as páginas estáticas em verdadeiros aplicativos rodando no navegador.

Originalmente uma linguagem funcional focada no *front end*, hoje em dia já conseguimos aplicar JavaScript tanto no back end, usando ferramentas como o Node.JS, quanto em aplicações mobile, com o React Native.

> Front-end é a parte de um sistema de software que lida com a interação e apresentação ao usuário, incluindo interface gráfica, layout, design e interatividade, geralmente implementada com HTML, CSS e JavaScript em aplicações web.

O seu uso é intrinsicamente ligado ao conhecimento de HTML e CSS, já que suas funções, em boa parte, manipulam elementos do DOM. Dessa forma, deixo aqui uma recomendação: só estude essa aula após a aula de HTML. Para a aula, usaremos o padrão ECMAScript 11, a versão mais recente, para orientar os padrões de JavaScript que veremos na aula.

É RECOMENDADO O CONHECIMENTO BÁSICO DE HTML E CSS PARA ESSA AULA!

Para a aula de hoje, se possível, acompanhe os códigos através do uso de alguma plataforma de programação baseada no desktop, como o Visual Studio Code, ou em alguma IDE online - recomendo o uso do REPLIT.

Além disso, usaremos nossa "IDE" em texto, para destacar os códigos. Veja:

```JavaScript
Aqui iremos inserir os blocos de código
como se estivéssemos trabalhando em uma IDE
num arquivo "script.js"
```

```
aqui representamos o terminal, onde iremos
Expor os resultados dos comandos executados
```


02648216103 - HENRIQUE AMORIM BARRETO

02648216103 - HENRIQUE AMORIM BARRETO

Local de Uso

O JavaScript por si só não é nada - ele precisa estar conectado a um arquivo HTML para que suas funcionalidades sejam vistas e utilizadas. Podemos abordar essa ligação entre o JS e o HTML de duas formas diferentes:

- Diretamente no código HTML, através de uma *tag* específica
- Em um arquivo separado.

Quando utilizado dentro do código HTML, o JavaScript fica situado dentro da tag <script>. Veja:

HTML

```
<html>
   <body>
      <h1> Header </h1>

      <script>
         //Script JS aqui
      </script>

   </body>
</html>
```

Já quando trabalhamos com o código JS em um arquivo separado, deveremos linká-lo dentro do HTML, também na tag `script`, da seguinte forma:

HTML

```
<html>
   <body>
      <h1> Header </h1>

      <script src="scriptJs.js"> </script>

   </body>
</html>
```

Essa *tag* não possui um local específico para ser incluído no HTML, porém, por boa prática, recomenda-se o uso da tag `<script>` entre as tags `<head>` `<head/>`, ou logo depois do fechamento do corpo do HTML, ou seja, da tag `</body>`.

Comentários

Comentários são blocos de código não interpretados pelo leitor, que servem para esclarecer funções, explicar variáveis, enfim, para explicar informações que talvez não sejam tão óbvias no código. Além disso, comentários podem ser úteis para destacar partes do nosso código que não desejamos que sejam executadas. No JS, podemos comentar os códigos de duas formas distintas:

- Comentando apenas uma linha de código, com o comando `//`
- Delimitando um início, com `/*`, e um fim, com `*/` para o comentário

HTML

```
//esse é um comentário de uma linha

console.log('Hello World')

/* esse é um comentário
que pode abranger
múltiplas linhas */
```

(FGV/SEMSA MANAUS/2022) A respeito da inserção de comentários no código JavaScript, analise as afirmativas a seguir.

É ignorado pelo JavaScript

I. qualquer texto entre // e o fim da linha.
II. qualquer texto entre /* e */, abrangendo uma ou mais linhas.
III. qualquer texto entre {* e *}, abrangendo uma ou mais linhas.

Está correto o que se afirma em

a) I e II, somente.
b) I e III, somente.
c) I, II e III.
d) II e III, somente.
e) I, somente.

Comentários:

Os itens que apresentam comentários corretamente lidos pelo JS são os itens 1 e 2. O item 3 não apresenta uma sintaxe correta para o contexto do JS. (Gabarito: Letra A)

Sintaxe

Tipos de Dados

Os tipos de dados definem que tipo de informação será carregada pelo dado - números, caracteres, *arrays* etc. No JS, temos 8 tipos de dados diferentes. São eles:

- String: é uma sequência de caracteres, como texto, delimitada por aspas simples ('') ou duplas (""). Por exemplo: `“John Doe”`, `‘Olá, mundo!’`.
- Number: representa números, tanto inteiros, quanto decimais. Exemplo: `4`, `9.7`, `-13.39`.
- BigInt: representa números inteiros de grande monta. Esse padrão foi introduzido mais recentemente devido a uma limitação dos dados do tipo number, que só aceitam até 64 bits. Os BigInt são caracterizados pela presença de um `n` após a cadeia de números. Exemplo: `12345678901234567890123456789012345678901234567890n`.
- Boolean: representam valores lógicos, podendo ser verdadeiros (`true`) ou falsos (`false`).
- Undefined: indica uma variável não inicializada, que não assume algum valor.
- Null: indica a ausência intencional de valor
- Symbol: cria valores únicos e imutáveis, usados como chaves de propriedades em objetos
- Object: é uma construção destinada a abrigar dados complexos e coleções de dados. Os objetos são representados por um par de chaves `{}`.

Para verificar o tipo de determinado dado, usamos o comando `typeof`. Veja:

JavaScript

```
console.log(typeof 42)

console.log(typeof "John Doe")

console.log(typeof null)

var naoIniciada
console.log(typeof naoIniciada)
```

```
number
string
object
undefined
```

(FGV/BBTS/2023) Os tipos de dados suportados pela linguagem ECMAScript (versão 2021) são

a) Null, Boolean, Binary, Symbol, Real e Integers.
b) Null, Boolean, Varchar, Symbol, Real, Integers e Object.
c) Undefined, Null, Boolean, String, Symbol, Number, BigInt e Object.
d) Undefined, Null, Boolean, Varchar, Symbol, BigInt, Real, Money e Polimorphic.
e) Complex, Null, Boolean, Binary, Varchar, LongInt, Real, Money, Object e Polimorphic

Comentários:

Vamos analisar as alternativas - irei tachar todos os tipos de dado que não fazem parte do JavaScript.

a) Errado. Null, Boolean, ~~Binary~~, Symbol, ~~Real~~ e ~~Integers~~.
b) Errado. Null, Boolean, ~~Varchar~~, Symbol, ~~Real~~, ~~Integers~~ e Object.
c) Certo. Undefined, Null, Boolean, String, Symbol, Number, BigInt e Object.
d) Errado. Undefined, Null, Boolean, ~~Varchar~~, Symbol, BigInt, ~~Real~~, ~~Money~~ e ~~Polimorphic~~.
e) Errado. ~~Complex~~, Null, Boolean, ~~Binary~~, ~~Varchar~~, ~~LongInt~~, ~~Real~~, ~~Money~~, Object e ~~Polimorphic~~

A correta é, portanto, a letra C. (Gabarito: Letra C)

Variáveis

O JavaScript é uma linguagem orientada, basicamente, para criar funções que irão manipular nosso HTML. Sua sintaxe define dois tipos de valores:

- Literais – valores fixos
- Variáveis – valores que podem variar no tempo

Valores literais são escritos de forma, adivinhe: literal! Não há simbologia, atribuição nem nada.

```JavaScript
10.50
“Olá mundo”
```

As literais são muito pouco usadas. Já as variáveis, em contrapartida, são o coração do JavaScript. Quase tudo que trabalharemos envolve as variáveis. O JavaScript usa o operador = para atribuir valores a variáveis.

Importante: para visualizar os resultados das nossas funções, usaremos uma função para printar o resultado do código no terminal, chamada de `console.log( )`.

```JavaScript
var x = 10
var y = “Olá, mundo!”
console.log(x, y)
```

```
10 Olá, mundo!
```

Podemos declarar variáveis de 4 formas diferentes:

- Automaticamente (sem definir o tipo)
- Usando `var`
- Usando `let`
- Usando `const`

As variáveis são atribuídas automaticamente quando simplesmente nomeamos a variável e atribuímos um valor, sem definir o tipo anteriormente.

```JavaScript
x = 10
```

Mas essa é uma prática que eu recomendo evitar. Em JavaScript, é ideal que declaremos o tipo de variável que estamos usando. Nesse sentido, vamos conhecer os três tipos de variáveis utilizadas no JavaScript e quando podemos as utilizar. Muito cuidado, pois esse é um tópico estruturante para todo o entendimento da linguagem.



var

- Variáveis que podem ser alteradas ao longo do código.
- Possui escopo global, podendo ser acessada até mesmo fora do bloco declarado
- São içadas (hoisted) para o topo do escopo em que são definidas, dessa forma podemos acessá-las antes mesmo de defini-las
- Recomendado seu uso apenas quando necessitar compatibilidade com navegadores antigos, já que o hoisting não é uma prática desejável

let

- Variáveis que podem ser alteradas dentro do código
- Seu escopo é de bloco, ou local, isso quer dizer que são visíveis apenas dentro do bloco que forem criadas (por exemplo, dentro de uma função, do if, por aí vai)
- Não suporta hoisting

const

- São variáveis constantes – não podem ser alteradas depois da atribuição inicial
- Assim como let, tem escopo de bloco
- Não suporta hoisting

> Hoisting é um comportamento em linguagens de programação onde declarações de variáveis e funções são movidas para o topo de seus escopos durante a fase de compilação ou interpretação do código, permitindo que sejam acessadas antes de serem declaradas no código fonte. Isso ocorre devido ao processo de pré-processamento do código.

QUESTÃO DE PROVA



(FGV/TJ RN/2023) No contexto da linguagem JavaScript, analise as afirmativas sobre variáveis declaradas como const.

I. Devem ter seus valores atribuídos na declaração.
II. Podem ter seus valores alterados depois de declaradas.
III. Têm sempre escopo global.

Está correto o que se afirma em:

a) somente II e III;
b) somente I;
c) somente II;
d) somente III;
e) I, II e III.

Comentários:

Vamos analisar as alternativas.

I. Certo. Seus valores são constantes, de forma que devem ser atribuídos na sua declaração inicial - portanto, não podemos ter variáveis const não inicializadas.
II. Errado. Novamente, seus valores são constantes - não podem ser alterados após a definição.
III. Errado. A variável const, assim como a let, possui escopo local. A única variável com escopo global é a var.

Sendo assim, correto somente o item I. (Gabarito: Letra B)

Então, basicamente:

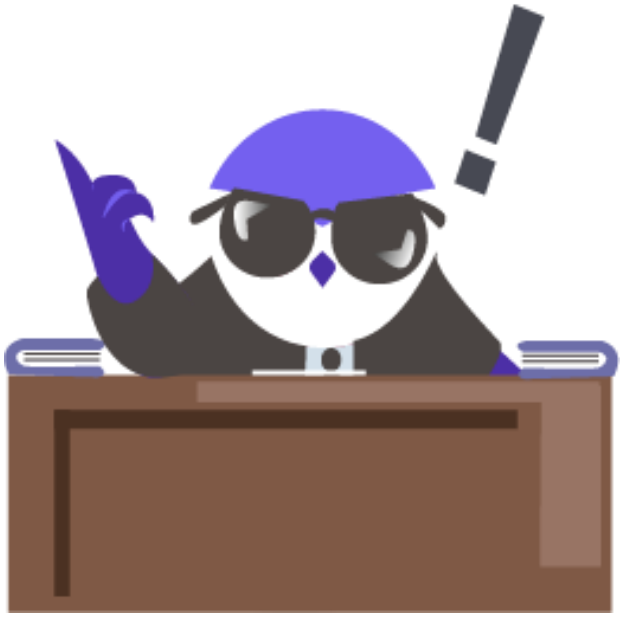
- Variável e varia ao longo do código – let ou var
- Variável e não pode variar, é constante – const

Pode parecer confuso ter uma "variável que não varia". Mas é isso mesmo, já que as variáveis funcionam como contêiners de valores.

Por último, sobre a nomeação de variáveis, o JavaScript apresenta um conjunto de regras:

- Variáveis podem começar apenas com letras, $ e _
- Podemos usar letras, números, $ e _ para definir o nome das variáveis
- Os nomes são sensíveis a minúsculas e maiúsculas
- Nomes reservados, como de funções específicas, não podem ser usados

Por último, quando não definimos um valor para uma variável, apenas a declaramos, temos um valor ausente. Valores ausentes, em JavaScript, são tratados como `undefined`.



Vamos declarar umas variáveis e ver o que acontece no terminal.

JavaScript

```
let numero = 10
let string = "John Doe"
const pi = 3.14
let carro
let oMelhor = true

console.log(numero, typeof numero)
console.log(string, typeof string)
console.log(pi, typeof pi)
console.log(carro, typeof carro)
console.log(oMelhor, typeof oMelhor)
```

```
10 number
John Doe string
3.14 number
undefined undefined
true boolean
```

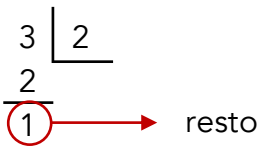
Operadores

O JS, assim como outras linguagens de programação, conta com operadores matemáticos, lógicos e de comparação, destinados a inserir capacidades matemáticas na linguagem. Vou trazer uma tabela que resumirá todos os operadores e, em seguida, traçarei alguns comentários sobre os operadores mais "diferentes".

OPERADOR	DESCRIÇÃO	EXEMPLO	RESULT.
OPERADORES MATEMÁTICOS			
+	Soma	4 + 4	8
-	Subtração	4 – 4	0
*	Multiplicação	4 * 4	16
/	Divisão	4 / 4	1
**	Potência	4 ** 4	256
%	Módulo (resto de divisão)	5 % 2	1
++	Incremento	5++	6
--	Decremento	5--	4
OPERADORES LÓGICOS E COMPARATIVOS			

&&	E lógico (AND)	true && false	false
||	OU lógico (OR)	true || false	true
!	Negação lógica (NOT)	!true	false
==	Igualdade	5 == 5	true
!=	Desigualdade	5 != 3	true
>	Maior que	5 > 3	true
<	Menor que	3 < 5	true
>=	Maior ou igual a	5 >= 5	true
<=	Menor ou igual a	3 <= 5	true
===	Igualdade estrita (valor e tipo)	5 === '5'	false

O módulo (%) é responsável por trazer o resto de uma divisão. Ele é extremamente importante no contexto de definir itens de divisão inteira ou números pares. Por exemplo, a divisão entre 3 e 2 tem resto 1 - que indica que o número não é divisível por 2, e, portanto, é ímpar. Veja:



Outro elemento de destaque, ou melhor, o elemento de maior destaque nessa lista é a comparação de igualdade. Temos duas formas de comparação de igualdade:

- `==` → faz comparações de igualdade de valor, ignorando o tipo de dado
- `===` → faz comparações restritas, comparando o valor e o tipo de dado

Então, quando usamos o `==`, estamos comparando apenas o valor. O tipo de dado é irrelevante - o 5 do tipo numérico tem o mesmo valor do '5', uma *string*. Porém, ao usarmos o comparador restrito `===`, estamos comparando também o tipo do dado. Não basta o valor ser igual, o tipo também tem que ser. Vamos rodar um código para que você veja o resultado.

JavaScript

```
let x = 4 == '4'
let y = 4 === '4'

console.log(x)
console.log(y)
```

```
true
false
```

QUESTÃO DE PROVA



(FGV/SEF MG/2023) Considere as seguintes expressões usando a linguagem javascript:

1 == '1' e 1 === '1'

Os resultados são, respectivamente,

a) 1 e 1
b) true e true
c) '1' e '1'
d) false e false
e) true e false

Comentários:

Estamos comparando 1 número, com a string "1". Em 1 == '1', ignoramos o tipo, portanto retornamos true. Já em 1 === '1' há uma comparação de tipo também, por isso retornamos false.(Gabarito: Letra E)

Ainda temos um outro tipo de operador extremamente utilizados: os **operadores de atribuição**. Eles representam formas compactadas, resumidas, de escrever operações matemáticas, associando o resultado dessa operação a uma variável.

Veja:

OPERADOR	Exemplo	Equivalência
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x – y
*=	x *= y	x = x * y

/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

JavaScript

```
let x = 5
x += 5

let y = 5
y++

console.log(x)
console.log(y)
```

```
10
6
```

Operações Bitwise

Antes de começar essa seção, preciso te explicar como funcionam os *bits*.

REVISÃO BINÁRIOS

Números binários são um sistema numérico que utiliza apenas dois dígitos: 0 e 1. Enquanto o sistema decimal, que é o sistema numérico mais comum, utiliza 10 dígitos (de 0 a 9), o sistema binário é mais simples, pois utiliza apenas dois dígitos, refletindo a base 2.

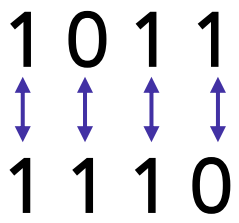
Num sistema binário, o 1 indica que o número está presente, e o 0 indica que não está. O número mais à direita sempre equivale a 2^0 e, a partir dali, subimos uma casa exponencial. Veja como fica, em base decimal, os 8 primeiros bits:

$2^7\ 2^6\ 2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$
128 64 32 16 8 4 2 1

Então, se quisermos escrever o número 3, por exemplo, podemos escrever 11 – que equivale a $2^1 + 2^0 = 2 + 1 = 3$. Outro exemplo – o número 99. Para chegar nele, precisamos "ativar" o 64, 32, 2 e 1 (64 + 32 + 2 + 1 = 0). Então, precisamos do 2^0, 2^1, 2^5 e do 2^6. O número ficaria assim:

1100011

Quando comparamos logicamente dois números, fazemos uso de operadores **bitwise**, que comparam nossas variáveis *bit a bit*. Ou seja, se temos um número 1011, e outro 1110 (em bits, obviamente), iremos comparar 1 com 1, 0 com 1, 1 com 1 e 1 com 0.

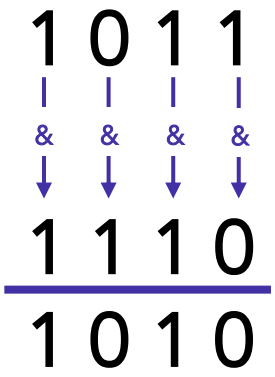


Vamos ver os principais operadores – que derivam de operadores lógicos (E, OU e afins). Encare o 1 como verdadeiro, e 0 como falso, que ficará bem mais tranquilo de entender.

Bitwise AND (&)

As operações ***bitwise and*** (E lógico) usamos o ***e*** lógico do JavaScript: **&**. As comparações apontarão valores 1 apenas quando ambos os binários comparados forem 1. Vamos seguir com o nosso exemplo 1011 e 1110 – que equivalem a 11 e 14, respectivamente. A operação 11 & 14 ocorre da seguinte forma:

- `1 & 1 → 1`
- `0 & 1 → 0`
- `1 & 1 → 1`
- `1 & 0 → 0`



Como resposta, temos 1010 em binários, ou 10, em base numérica. O código para implementar essa operação segue abaixo:

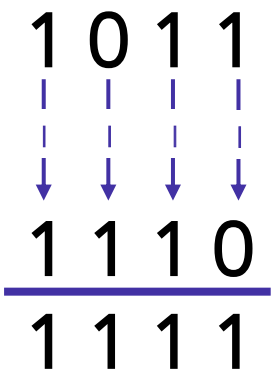
```javascript
let bitwiseAnd = 11 & 14 // 1011 & 1110
console.log(bitwiseAnd)
```

```
10 // 1010
```

Bitwiser OR (|)

Para operações ***bitwise or*** (OU lógico) usamos o *ou* lógico do JavaScript: `|`. Aqui, teremos o valor 1 apontado quanto pelo menos um, e qualquer um, dos dois elementos for 1. A operação 11 | 14 ocorre da seguinte forma:

- 1 | 1 → 1
- 0 | 1 → 1
- 1 | 1 → 1
- 1 | 0 → 1



Como resposta, temos 1111, ou 15. Veja em código:

```javascript
let bitwiseOr = 11 | 14 // 1011 & 1110
console.log(bitwiseOr)
```

```
15 // 1111
```

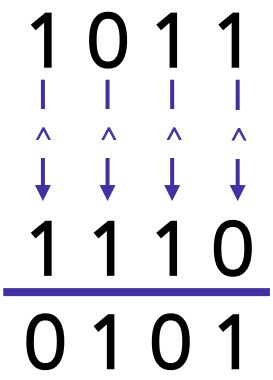
XOr (^)

Para operações ***bitwise xor*** (OU EXCLUSIVO lógico) usamos o *xor* lógico do JavaScript: `^`. O *XOR* corresponde, em lógica, ao *ou exclusivo*, ou ao *ou A ou B*, mas não ambos. Dessa forma, retornaremos 1 quando somente um dos dois números for 1, e 0 quando ambos forem 1 ou 0. Então:

- `1 ^ 1 = 0`
- `1 ^ 0 = 1`
- `0 ^ 0 = 0`

A operação 14 ^ 11 se desenrola da seguinte forma:

- `1 ^ 1 → 0`
- `0 ^ 1 → 1`
- `1 ^ 1 → 0`
- `1 ^ 0 → 1`



Como resposta, temos 0101, ou 5. Veja em código:

```javascript
let bitwiseXOr = 11 ^ 14 // 1011 | 1110
console.log(bitwiseXOr)
```

```
5 // 0101
```

Bitwise Not

O operador ***bitwise not*** (negação lógica) usa o símbolo `~` para indicar uma negação do número analisado. Aqui, não estamos mais comparando dois números, e sim negando um número. Mas aqui temos um detalhe: estamos trabalhando com números binários. A negação 5 não é, simplesmente, -5.

O que ocorre com o operador é a inversão total de bits. Usualmente, trabalhamos com espaços de 32 bits. O número 5 seria representado da seguinte forma:

```
00000000000000000000000000000101
```

Porém, é comum que façamos a omissão dos 0s para simplificar o entendimento – já que, em conversões, eles não contribuem em nada. Mas, para negarmos, iremos fazer uma complementar e transformar todos os bits 0 em 1, e 1 em 0:

```
11111111111111111111111111111010
```

Acontece que a negação transforma um número positivo em negativo, nesse caso os binários funcionam um pouco diferente. Aqui, o bit mais significativo (o bit mais à esquerda) é usado como bit de sinal. O bit mais significativo igual a `1` indica um número negativo, e, se for igual a `0`, indica um número positivo.

Quando os números forem indicados como negativos, temos uma mudança – os 0s representam complementos de 2. E, ao contrário de números positivos, não começamos com 2^0, e sim com 2^1.

$$11111111111111111111111111110\mathbf{1}0\mathbf{1}0 = -((2^1 + 2^2) = -6$$

Ok, parece complexo né? Mas tenho uma forma muito mais simples de resolver isso, seguindo uma fórmula infalível:

$$\sim X = -(X + 1)$$

A negação de qualquer número vai ser igual ao negativo do seu número subsequente. Então:

- ~5 = -(5+1) = -6
- ~0 = -(0+1) = -1
- ~12 = -(12+1) = -13
- ...
- ~14.324 = -(14.324+1) = -14.325

Deslocamento de Bits

A última operação que veremos, que na verdade são 3, representam operações de deslocamento – chamadas de ***bitwise left shift*** (deslocamento de bits para a esquerda), e ***bitwise right shift*** (deslocamento de bits para a direita). Sua simbologia é:

- Bitwise Right Shift = `>>`;
- Bitwise Left Shift = `<<`;

Aqui, deslocamos todos os bits para a direita, ou esquerda, preenchendo o bit restante com o bit de sinal – 0 se positivo, 1 se negativo. Como a sintaxe do operador é `X >> n`, podemos, em *X*, definir o número que queremos operar, e em *n* definir a quantidade de casas que iremos deslocar. Vamos deslocar alguns números:

5 >> 1

- Estamos deslocando 5 (0101) 1 bit para a direita. Iremos excluir o bit mais à direita (que destaquei em vermelho), e preencher o restante com 0s - por se tratar de um número positivo.
- 0101 >> 1 → 0010
- Convertendo para base decimal, temos que 5 >> 1 = 2

5 << 1

- Estamos deslocando 5 (0101) 1 bit para a esquerda. Aqui é mais simples – basta adicionar mais um 0 no final da cadeia de bits, já que estamos trabalhando com números positivos.
- 0101 << 1 → 01010
- Convertendo para base decimal, temos que 5 >> 1 = 10

-5 >> 1:

- Em *bitwise right shift* com números negativos, por estarmos trabalhando com números negativos (bit mais significativo igual a 1), o preenchimento é feito com 1s, e não com 0s.
- Lembrem que a negação de um número é igual a ele, somado a um, negativo (~X = -(X+1). Então, para acharmos -5 em notação binária, temos que fazer ~4.
- 4 em binário: 000...0100 (vou omitir os 0s, mas saibam que existem 32 números aqui)
- Negando todos os bits, para achar -5: 111...1011
- Agora temos um número para trabalhar: 11111111111111111111111111111011
- Vamos excluir o elemento mais à direita, e adicionar o 1 no começo.
- 11111111111111111111111111111011 >> 1 → 11111111111111111111111111111101
- Agora, voltando para a base decima, podemos simplesmente aplicar a fórmula. Na base de 2, consideramos os 0s como bits ativos, então teríamos 2 no binário acima, já que só o penúltimo bit está ativado. Retornando, basta fazer VALOR = -(X+1) = -(2+1) = -3

Bom, lembrem que eu falei que são 3 operadores, mas só vimos 2 né? O outro operador é uma variação, chamado de zero-fill. O *zero-fill*, como o nome aponta, preencherá *n* casas, à direita ou à esquerda, apenas com 0s.

Nesse caso, temos o zero-fill right shift. Ele possui uma diferença significativa: ele muda o bit mais significativo. não preencheremos mais o número com os bits significativos, e sim sempre com 0. Então, aqui, números negativos se tornarão positivos. Sua notação é dada ao adicionarmos mais um símbolo ao shift, >>> .

5 >>> 1:

- Em números positivos, também temos o mesmo efeito. 5, em binários, é 0101. Iremos deslocar o número à direita, excluindo o último 1, e preenchendo o restante com 0s.
- 0101 >>> 1 → 0010
- Veja que o resultado foi o mesmo, 2, sem diferença.

-5 >>> 1:

- Agora o buraco é mais embaixo. Já temos a notação em 32 bits de -5: 111...1011
- Iremos preencher o campo à esquerda com um 0 – só que agora esse campo que mudaremos é o bit mais significativo. Isso irá transformar nosso número negativo em um número positivo.
- 11111111111111111111111111111011 >> 1 → 01111111111111111111111111111101
- Agora, retornando à base decimal, temos um número gigante rs – 2.147.483.645



(FGV/CÂMARA DOS DEPUTADOS/2023) Analise as operações bitwise do JavaScript, exibidas a seguir.

4 & 1
7 | 2
~ -5
9 >> 2
9 >>> 1

Os valores de cada uma dessas expressões, na ordem, são:

a) 0; 7; 5; 1; 2.
b) 1; 9; 5; 1; 2.
c) 0; 7; 4; 2; 4.
d) 0; 9; 4; 0; 4.
e) 1; 1; -5; 4; 2.

Comentários:

Vamos lá, fazer uma por uma.

1. 4 & 1

Primeiro, convertemos 4 e 1 para bits, ficando com 0100 & 0001. Agora, comparamos:

- 0 & 0 = 0
- 1 & 0 = 0
- 0 & 0 = 0
- 0 & 1 = 0

Resultado 0000 = 0

2. 7 | 2

Convertendo-os para binários, ficamos com 0111 e 0010.

- 0 | 0 = 0
- 1 | 0 = 1
- 1 | 1 = 1
- 1 | 0 = 1

Resultado 0111 = 7

3. ~ -5

Vamos aplicar a fórmula, já que é a forma mais prática de resolvermos a questão. Lembrem:

- ~X = - (X+1)
- ~ (-5) = - (-5+1)
- ~ (-5) = - (-4)
- ~ (-5) = 4

Resultado = 4

4. 9 >> 2

Essa envolve um deslocamento bitwise de 2 casas à direita. 9, em binário, é 1001. Deslocando:

- 1001 >> 2 = 0010
- Portanto, temos 0010 = 2

5. 9 >>> 1

Como falei para vocês, operações zero-fill com números positivos não mudam nada. Assim, vamos deslocar 1001 uma casa para a direita, preenchendo com 0s.

- 1001 >> 2 = 0100

Portanto, temos 0100 = 4

Sendo assim, ficamos com 0, 7, 4, 2 e 4 – correta a letra C. (Gabarito: Letra C)

Funções

Vamos agora aprender a criar funções. Nosso foco aqui será aprender a criar funções personalizadas. Não iremos abordar, pelo menos nessa seção, as funções nominadas, nativas do JavaScript, como *count* e *length* – que veremos logo em seguida, no que chamamos de **funções nominadas**.

Para criar uma função, usamos a seguinte estrutura:

```
function minhaFunção(param1, param2) {
   return x
}
```

Onde:

- **`Function`** define que estamos criando uma função
- **`minhaFuncao`** é o nome da função
- **`(param1, param2)`** define os parâmetros da função, sendo que podemos ter quantos parâmetros quisermos – inclusive nenhum
- **`return`** indica qual será o retorno da função

Para chamar a função, basta colocar seu nome com os parâmetros devidos – no caso do exemplo, *minhaFuncao(param1, param2)*. Se chamarmos uma função sem o parênteses, o retorno será apenas sua definição, isso é, o bloco de código que está dentro dela. Lembrando que, se houver parâmetros expressos, só é possível chamar a função corretamente se apontarmos todos eles.

Vamos criar uma função para multiplicar dois itens.

JavaScript

```
function minhaFuncao(param1, param2) {
  return param1 * param2
}
//vamos chamar a função dentro do console.log, para ver o resultado
console.log(minhaFuncao(3, 4))
```

```
12 // 3 * 4 = 12
```

Podemos, com a variável `let`, criar variáveis locais dentro de uma função, que terá apenas um escopo local. Ou seja, só funcionará dentro da função e não poderá ser chamada fora dela.

(FGV/TJ RJ/2023) No contexto de um script JavaScript numa página Web, considere o trecho a seguir.

```
<script>
function teste(x) {
return (x / 2);
}
alert (teste);
</script>
```

É correto afirmar sobre esse script que:

a) há um erro de sintaxe que impede sua execução;
b) quando executado, exibe a definição da função teste;
c) quando executado, exibe o valor 0;
d) quando executado, exibe o valor undefined;
e) quando executado, o comando alert não é acionado.

Comentários.

Galera, guarde bem isso: sempre que fomos invocar uma função com o intuito de fazer com que ela funcione, ou seja, que ela produza seu resultado, é imprescindível que tenhamos o elemento de "ativação" `()`. Se não tivermos parâmetros na função, ele aparece vazio, caso contrário, ele recebe os parâmetros.

Quando rodamos uma função só com seu nome, sem ativá-la, temos como retorno a sua definição – no caso, retornaria *return (x/2)*. Nesse sentido, a afirmativa que descreve corretamente o que acontece é a letra B. (Gabarito: Letra B)

Existe uma forma específica de escrevermos funções, chamada de arrow function. Elas permitem que escrevamos funções em menos espaço, pelo uso da "arrow" `=>`. Digamos que, em uma função tradicional, você quer que a variável olá retorne, no console, o texto "Olá, mundo!"

JavaScript

```
ola = function () {
    return "Olá, mundo!"
}
console.log(ola())
```

```
Olá, mundo!
```

Podemos "transformar" todo esse código da função em um código de apenas uma linha. Para isso, usaremos a seguinte estrutura:

`variavelQualquer = (param) => "Resultado da função"`

JavaScript

```
ola = () => "Olá, mundo"
console.log(ola())
```

```
Olá, mundo!
```

Podemos aplicar resultados mais complexos a uma arrow function. Para isso, basta abrirmos um par de colchetes `{}` no lugar do resultado da função, expandindo o que podemos retornar, inclusive colocando outras arrow dentro deles. Vou trazer um exemplo que aborda algumas definições que ainda não vimos, apenas para que você possa entender a possibilidade.

JavaScript

```
let verificarSituacao = (estudante) => {
    let media = (estudante.notas.reduce((acc, nota) => acc + nota, 0)) /
estudante.notas.length;
    return media >= 7 ? 'Aprovado' : 'Reprovado';
};

const aluno = {
    nome: 'João',
    notas: [8, 7, 6, 9]
};
```

Este código define uma função chamada verificarSituacao que recebe um objeto estudante como argumento. A função calcula a média das notas do estudante, somando todas as notas do array (com o *reduce*) e dividindo pelo número de notas, calculado a partir da função *length*. Em seguida, a função retorna 'Aprovado' se a média for maior ou igual a 7, caso contrário, retorna 'Reprovado'.

(FGV/CÂMARA DOS DEPUTADOS/2023) Analise o pretenso código JavaScript a seguir.

const xpto = (a, b) => a + b;

Sobre esse trecho, é correto afirmar que esse código:

a) é válido, porque define uma função que retorna a soma de dois números fornecidos como parâmetros;
b) não compila devido à ausência de um par dos delimitadores "{" e "}";
c) não compila porque o símbolo "=>" não é permitido na definição de uma função JavaScript;
d) não compila porque o símbolo "=>" não é permitido numa declaração const;
e) pode ser compilado, mas gera erro em tempo de execução quando xpto for invocada.

Comentários:

Questão tranquila. A constante xpto apresenta uma arrow function que retorna a soma de dois elementos, passados como parâmetro. Caso você tenha dificuldade em entender, podemos sempre expandir a função:

```
function xpto(a, b) {
      return a + b
}
```

Dessa forma, a única questão que aponta uma afirmativa correta é a letra A. (Gabarito: Letra A)

Funções Nominadas

Funções nominadas são um conjunto de funções "pré-fabricadas", prontas, com objetivos específicos, fornecidas pelo JavaScript. Sua estrutura é a mesma das funções comuns mas, como já temos o objetivo dele definido, isso é, o bloco de código que ela irá executar, não precisamos defini-lo. Importante ressaltar que os nomes dessas funções nominadas são reservados, de forma que não podemos definir funções "personalizadas" com seus nomes.

Aqui não tem segredo: é saber o que a função faz, seu nome e seus eventuais parâmetros. Algumas funções são projetadas diretamente para lidar com *arrays*, então as veremos mais à frente.

Função	Descrição
`length()`	Mede o comprimento de algum objeto. Em *strings,* medirá a quantidade de caracteres; em arrays, a quantidade de elementos.
`charAt(indice)`	Retorna o caractere na posição específica indicada no índice
`concat(args)`	Método que combina duas ou mais strings e retorna uma única string
`toUpperCase()`	Altera todos os caracteres para maiúsculas
`toLowerCase()`	Altera todos os caracteres para minúsculas
`toString()`	Converte um objeto em uma *string*
`parseFloat(string)`	Analisa uma *string* e retorna um elemento de ponto flutuante (float)
`parseInt(string, radix)`	Analisa uma string e retorna um número inteiro, na base especificada em *radix*
`toFixed(digitos)`	Formata um número com notação de ponto fixo, especificando uma quantidade de casas decimais
`alert()`	Exibe um *pop-up* de alerta, que contêm determinada mensagem
`console.log()`	Traz um *print* no terminal de determinada função ou arquivo

Funções Construtoras

No JavaScript, as funções construtoras são utilizadas para criar objetos que compartilham as mesmas propriedades e métodos. Elas são uma forma de definir um modelo ou "classe" a partir do qual podemos instanciar múltiplos objetos com as mesmas características. Isso talvez possa ser confuso para você se você não possui conhecimentos do Paradigma Orientado a Objeto, mas logo ao longo da aula explicarei melhor o que é esse paradigma e como ele se relaciona com as funções construtoras.

Entenda apenas que as funções construtoras são usadas para adicionar um determinado objeto a partir de um modelo "pré-definido", que define suas características, informações, métodos e interfaces. Para caracterizar uma função construtora, temos a presença do elemento new. Por exemplo:

```javascript
var honda = new Carro (Honda, Civic, 2018, 32000)
```

Perceba que, quando usamos uma função construtora, ela sempre receberá os parâmetros na sua definição - de forma a preencher os campos do objeto com esses determinados valores. Quando falarmos de objetos, retornaremos nessas funções.

Funções e Métodos de Data

Quando lidamos com datas em JavaScript, usamos primariamente uma função construtora chamada de Date(). Quando usada como construtora, a função tem os seguintes parâmetros:

```
new Date (ano, mês, dia, hora, minuto, segundo, milissegundo)
```

Essa função tem algumas nuances um pouco complexas. A primeira delas é que os meses começam a contar a partir de 0 - então, janeiro é 0, fevereiro 1, e assim em diante. Segundo ponto é o excesso de valor: cada valor desses tem um limite lógico, por exemplo, temos apenas 12 meses. Se o valor em mês for 13, iremos ter passados os 12 meses do ano (0-11), e passaremos os 2 meses do ano seguinte (12-13). Dessa forma, o excesso de valor é transportado ao elemento subjacente. Vamos fazer uma questão para entender melhor.



(FGV/SEMSA MANAUS/2022) Com respeito à representação de datas no JavaScript, considere as declarações a seguir.

const d1 = new Date(2022, 0, 24, 10, 33, 30, 0);

```
const d2 = new Date(2022, 15, 24, 10, 33, 30, 0);
const d3 = new Date(2022, -1, 24, 10, 33, 30, 0);
const d4 = new Date(2023, 1, 29, 10, 33, 30, 0);
```

Os valores das datas d1, d2, d3 e d4 são, respectivamente,

a) undefined
   undefined
   24/12/2021 10:33:30
   01/03/2023 10:33:30
b) undefined
   undefined
   24/12/2021 10:33:30
   undefined
c) undefined
   24/04/2023 10:33:30
   24/12/2021 10:33:30
   01/03/2023 10:33:30

d) 24/01/2022 10:33:30
   24/04/2023 10:33:30
   24/12/2021 10:33:30
   undefined
e) 24/01/2022 10:33:30
   24/04/2023 10:33:30
   24/12/2021 10:33:30
   01/03/2023 10:33:30

Comentários:

Lembrem da estrutura:

```
new Date (ano, mês, dia, hora, minuto, segundo, milissegundo)
```

Vamos analisar cada sintaxe.

const d1 = new Date(2022, 0, 24, 10, 33, 30, 0);

- Ano = 2022
- Mês = 0 (janeiro)
- Dia = 24
- Hora = 10

- Minuto = 33
- Segundos = 30
- Milissegundos = 0

Então, temos 24/01/2022 10:33:30

```
const d2 = new Date(2022, 15, 24, 10, 33, 30, 0);
```

- Ano = 2022* *(cuidado que valores posteriores irão modificar o ano)*
- Mês = 15 – equivale a percorrermos os 12 meses e mais 4, considerando que começamos no 0, portanto estamos no mês 4 do ano seguinte, 2023
- Dia = 24
- Hora = 10
- Minuto = 33
- Segundos = 30
- Milissegundos = 0

Então, temos 24/04/2023 10:33:30

```
const d3 = new Date(2022, -1, 24, 10, 33, 30, 0);
```

- Ano = 2022* *(cuidado que valores posteriores irão modificar o ano)*
- Mês = -1 – quer dizer que voltamos 1 mês, para dez/21
- Dia = 24
- Hora = 10
- Minuto = 33
- Segundos = 30
- Milissegundos = 0

Então, temos 24/12/2021 10:33:30

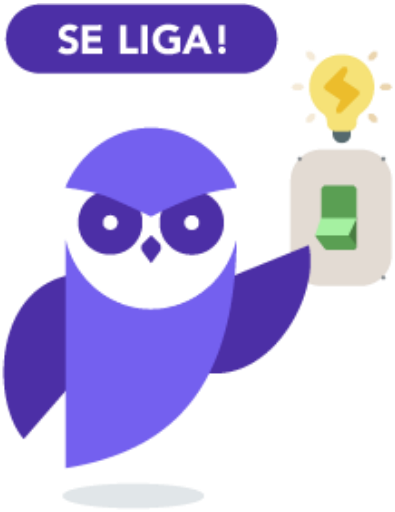
```
const d4 = new Date(2023, 1, 29, 10, 33, 30, 0);
```

- Ano = 2023
- Mês = 1 (fevereiro)* *(cuidado que valores posteriores irão modificar o mês)*
- Dia = 29 – Acontece que 2023 não é bissexto, então não temos 29 de fev. Passamos para o mês seguinte, 01 de mar, portanto.
- Hora = 10
- Minuto = 33
- Segundos = 30
- Milissegundos = 0

Então, temos 01/03/2023 10:33:30

Sendo assim, a alternativa que aponta corretamente todos os horários é a letra E. (Gabarito: Letra E)

Podemos lidar com os diferentes dados oferecidos pela Date() a partir de alguns métodos específicos - para obter, por exemplo, a hora ou o dia da semana. Para isso, usamos os métodos `get*`. Veja uma tabela abaixo com todos:



Método	Descrição
new Date	Cria um o objeto Date
getDate()	Retorna o dia do mês (1-31)
getDay()	Retorna o dia da semana (0-6)
getFullYear()	Retorna o ano
getHours()	Retorna a hora (0-23)
getMilliseconds()	Retorna os milissegundos (0-999)
getMinutes	Retorna os minutos (0-59)
getMonth()	Retorna o mês (0-11)
getSeconds	Retorna os segundos()
getTime()	Retorna o tempo de milissegundos contados desde 1-1-1970 e a data atual

Então, digamos que você tenha criado um objeto com a função construtora, e quer saber a hora atual da execução do comando. Podemos usar a seguinte sintaxe:

JavaScript

```
var dia = new Date()

console.log(dia.getDate(), dia.getMonth(), dia.getHours())
console.log(dia.getTime())
```

```
27 2 17
1711569980835
```

Estruturas de Dados

Arrays

Arrays são estruturas de dados que podem armazenar diversos valores, de diversos tipos, em uma mesma variável. Isso é um ponto importante: não existe, em JavaScript, uma estrutura de dados que não permita a presença de diversos tipos de dados ao mesmo tempo, como ocorrem com as listas em R, por exemplo.

Para criar uma array, usamos colchetes `[]`. Veja:

```
const arrayNumeros = [1, 3, 4, 9, 2, 1]
```

POR PADRÃO, RECOMENDA-SE O USO DE CONST PARA DEFINIRMOS ARRAYS

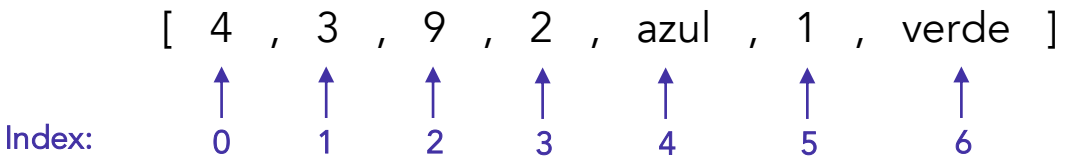
Por padrão, as arrays no JavaScript são unidimensionais. Porém, podemos criar diversas arrays dentro de uma mesma array, criando uma matriz. Vou mostrar como criamos uma matriz 3x3 em JavaScript:

JavaScript

```
const matriz = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
]
console.log(matriz)
```

```
[ [ 1, 2, 3 ], [ 4, 5, 6 ], [ 7, 8, 9 ] ]
```

Cada elemento numa array possui um índice (index), que indica sua posição relativa em relação ao início do elemento. Em JavaScript, o índice inicial, do primeiro elemento na array, será sempre 0.






```javascript
const arrayNumeros = [1, 3, 4, 9, 2, 1]

console.log(arrayNumeros[0]) //posição inicial
console.log(arrayNumeros[3]) //quarto elemento (9)
console.log(arrayNumeros[6]) //não temos a posição 6 na array
```

```
1
9
undefined
```

Caso tenhamos arrays dentro de arrays, basta navegar até a posição da array inicial, e selecionar a posição dentro dela através de uma dupla de chaves, como `[ ][ ]`.

```javascript
const arrayNumeros = [1, 3, [4, 5, 6], 9, 2, 1]

console.log(arrayNumeros[2][1])
```

```
5
```

Métodos de Arrays

Quero muita atenção sua nesse tópico, já que é, provavelmente, o assunto mais cobrado dentro do conteúdo de JavaScript.

Os métodos são as funções nominadas do JavaScript, projetadas para lidar diretamente com estruturas de objetos. Cada método possui um objetivo específico com a array. Vamos ver os métodos, categorizando por objetivos.

Adicionar e remover elementos

- **`push()`** → insere um elemento no final de uma *array*

```JavaScript
const numeros = [1, 2, 3]
numeros.push(4, 5) //irá inserir 4 e 5 ao fim da array
console.log(numeros)
```

```
[ 1 , 2 , 3 , 4 , 5 ]
```

- **`pop()`** → remove o último elemento de uma *array*

```JavaScript
const numeros = [1, 2, 3]
numeros.pop( ) //irá remover o 3
console.log(numeros)
```

```
[ 1, 2 ]
```

- **`splice(x, y, z)`** → remove ou adiciona elementos em posições específicas de uma array. Recebe até três parâmetros:
  - x → indica o índice em que queremos começar a manipulação. A inserção ocupará esse índice (por exemplo, x = 2 indica que o elemento que inserirmos terá a posição 2, empurrando o resto para a frente)
  - y → indica quantos elementos desejamos remover. 0 indica sem remoções.
  - z → indica o elemento que queremos inserir. Z pode ser composto de 1 ou *n* números, separados por virgulas

Abaixo, vamos inserir os números 3, 4 e 5 numa array que possui 3 números - [1, 2, 6]. Nós queremos que a posição inicial seja no índice 2 (logo após o 2), que não deletemos nada e que o número inserido seja 3, 4 e 5. Então:

- x = 2
- y = 0
- z = 3, 4 ,5

JavaScript

```
const numeros = [1, 2, 6]
numeros.splice(2, 0, 3, 4, 5)
console.log(numeros)
```

```
[ 1, 2 , 3 , 4 , 5 ]
```

(Inédita/Prof. Felipe Mathias) Carlos, analista de dados, estava analisando a estrutura de uma array sequencial da empresa PREVSis, e se deparou com alguns números faltantes na sequência. Veja:

```
const sequencia = [0, 1, 2, 3, 7, 8, 9]
```

Assinale a alternativa correta que preencherá a array, na ordem crescente, e na posição respectiva, com o conjunto de números [4, 5, 6]

a) sequencia.splice(3, 1, 4, 5, 6)
b) sequencia.splice(4, 0, 4, 5, 6)
c) sequencia.splice(5, 1, 4, 5, 6)
d) sequencia.splice(6, 0, 4, 5, 6)
e) sequencia.splice(7, 0, 4, 5, 6)

Comentários:

Vamos lá, queremos que o conjunto seja inserido logo após o 3, ou seja, que o 4 que será inserido ocupe a posição 4. Não iremos remover nenhum número, e os números inseridos são [4, 5, 6].

Então temos:

x = 4
y = 0
z = 4, 5, 6

Dessa forma, a sintaxe correta é sequencia.splice(4, 0, 4, 5, 6). (Gabarito: Letra B)

- **`shift()`** → remove o primeiro elemento de uma *array*

```JavaScript
const numeros = [1, 2, 3]
numeros.shift()
console.log(numeros)
```

```
[ 2 , 3 ]
```

- **`unshift()`** → adiciona um ou mais elementos no início de uma *array*

```JavaScript
const numeros = [1, 2, 3]
numeros.unshift(-1, 0)
console.log(numeros)
```

```
[ -1, 0, 1, 2, 3 ]
```

Encontrar elementos

Podemos saber que elemento está em posição específica dentro da nossa array, ou citar determinado elemento específico. O modo mais tradicional é colocando o índice do elemento dentro de colchetes – por exemplo, *meuArray[2]* retorna o terceiro elemento de uma array.

Porém, existe uma função que nos dá mais liberdade de trabalharmos – a função **`at(index)`**. Ela foi introduzida para resolver um pequeno problema. Usualmente, em linguagens de programação, podemos acessar o último elemento de uma array usando a posição [-1], porém, como os colchetes são usados tanto para acessar objetos quanto arrays no JS, isso dava um problemão. Agora, podemos usar `at(-1)` para acessar o último elemento.

```JavaScript
const numeros = [13, 21, 33, 45, 46, 56, 77, 78, 99, 103];
let ultimoNumero = numeros.at(-1)
console.log(ultimoNumero);
```

```
103
```

Pesquisar

Podemos querer obter informações específicas de uma array, como o índice de determinado elemento, seu comprimento, entre outros – são essas funções que veremos agora.

ÍNDICE

A função **`indexOf`** é usada para encontrar o índice de determinado elemento dentro da array. Digamos, numa lista de alunos, queremos encontrar a posição específica de Pedro, poderemos construir uma pesquisa `alunos.indexOf("Pedro")`. Havendo mais de uma ocorrência de um valor, o retorno apontará para a primeira ocorrência do elemento.

Veja:

JavaScript

```
const alunos = ["Rafael", "Carlos", "Joana", "Pedro", "Maria"]
let posicaoPedro = alunos.indexOf("Pedro")
console.log(posicaoPedro)
```

```
3 # lembrando que começamos da posição 0
```

Um uso alternativo do `indexOf` é feito a partir da sua aplicação diretamente sobre uma string. Nesse caso, cada caractere, incluindo os espaços vazios, é considerado um "objeto" autônomo, com uma posição específica - assim como na array. Assim, se usássemos `indexOf("i")` numa array com meu nome - ou seja, `"Felipe"` -, teríamos como retorno o valor 3, referente à posição do *i* nessa string.

(FCC/TRT 20/2016) Em JavaScript, a função do método indexOf( ) é

a) procurar um valor dentro de uma string e retornar a quantidade de ocorrências desse valor, se encontrado.
b) retornar o índice da última ocorrência do texto especificado em uma string.
c) trair um texto de uma string e retornar a string com o texto extraído em uma nova string.
d) trocar um valor especificado por espaço em branco na string.
e) retornar o índice da primeira ocorrência de um texto específico em uma string.

Comentários:

O indexOf() irá encontrar a posição de determinado elemento, retornando o índice de sua primeira ocorrência. Numa string, o método irá encontrar a posição da ocorrência de determinado caractere ou texto. Portanto, podemos analisar a string "A melhor linguagem é o JavaScript" e pesquisar um caractere, como indexOf("r"), por exemplo, onde teremos como retorno o índice do caractere, ou uma palavra completa, como em indexOf("JavaScript"), que retornará o índice do começo da palavra JavaScript - ou seja, o índice da letra J. (Gabarito: Letra E)

INCLUDES

A função **`includes`** é usada para verificar se um elemento está presenta na lista ou não. Seu retorno será `true`, se o elemento estiver presente, e `false` caso contrário. Vamos ver se Victor faz parte da *array* que criamos antes?

JavaScript

```
const alunos = ["Rafael", "Carlos", "Joana", "Pedro", "Maria"]
let contemVictor = alunos.includes("Victor")
console.log(contemVictor)
```

```
false
```

ENCONTRAR

Podemos usar uma função chamada de **`find( )`** para encontrar o primeiro elemento de uma array que atenda a uma condição específica. Nesse exemplo, temos 10 números aleatórios, e quero encontrar o primeiro número que seja maior que 39.

JavaScript

```
const numeros = [13, 21, 33, 45, 46, 56, 77, 78, 99, 103];
let encontrar = numeros.find(funcaoEncontrar);
function funcaoEncontrar(valor, index, array) {
    return valor >= 39;
}
console.log(encontrar);
```

```
45
```

Para usarmos a função `find()`, devemos passar como parâmetro uma outra função "personalizada" - no exemplo acima, a `funcaoEncontrar()`. Essa função passada como parâmetro contempla três parâmetros fixos: valor, índice ou array. Portanto, podemos definir um valor que queremos encontrar, um índice específico para consulta ou uma array dentro da função. Temos uma adaptação da função também, que encontra o índice, ao invés do valor – nome é `findIndex( )`. Os parâmetros e funcionamento são os mesmos.

Subgrupos

Funções de subgrupos criam novas variáveis a partir de uma manipulação da array. Temos algumas funções muito importantes aqui.

SLICE

A função **`slice(a, b)`** cria uma fatia da nossa array, retornando os elementos que atenderem aos parâmetros.

- a → indica o índice inicial, que será incluso
- b → indica o índice final, que não será incluso

Então perceba, damos o começo e o fim, com intervalo fechado no começo, e aberto ao fim.

JavaScript

```
const sequencia = [1, 2, 3, 4, 5, 6]
let x = sequencia.slice(2, 4)
console.log(x)
```

```
[ 3, 4 ] #veja que o número 5 (índice 4), não foi incluso
```

FILTER

A função **`filter( )`** cria um novo array com todos os elementos que passarem pela condição que definirmos. Para isso, usualmente, passamos uma função como parâmetro dentro do filter. Pegue um conjunto que vai de 0 a 10, por exemplo. Queremos filtrar apenas os números pares para uma nova array – ou seja, números cujo resto da divisão por 2 (numero % 2) seja igual a 0. Para isso, usaremos essa sintaxe:

JavaScript

```
const numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
/* usaremos o parâmetro numero, que representa cada elemento da array.
Assim, a função verificará elemento por elemento se é par */
let par = numeros.filter(function (numero) {
  return numero % 2 === 0
})
console.log(par)
```

```
[ 0 , 2 , 4 , 6 , 8 , 10 ]
```

FOR EACH

A função **`forEach( )`** (que, obrigatoriamente, tem que ser escrita assim, em camelCase) manipula uma array existente, aplicando uma função de callback a cada elemento da array em análise.

> Função de callback é uma função passada como argumento para outra função. A função de callback é então executada dentro da função externa em algum ponto do seu fluxo de execução. Essa técnica permite que a função externa seja capaz de chamar de volta (callback) a função que foi passada a ela, permitindo que a função de callback seja executada em um momento específico ou em resposta a algum evento.

Diferentemente do filter, que retornará todos os elementos que tiverem como resultado o valor TRUE na verificação de condição, o forEach aplica uma função a cada elemento da array, individualmente.

O forEach(numero, indice, array) recebe 3 argumentos:

- numero → representa o valor atual da array. É cada elemento da array que está recebendo a função forEach. Por exemplo, numa array [1, 2, 3], os parâmetros numero serão 1, 2 e 3.
- indice → representa o índice do elemento atual que está sofrendo a função
- array → representa a array original que está sendo iterada, sendo um parâmetro opcional e raramente usado.



Digamos que temos uma array [1, 2, 3, 4, 5], e queremos multiplicar cada elemento da array por 10. Vou criar uma função `forEach(a, b)` onde:

- a → indica o elemento da array. Será substituído por 1, 2, 3, 4 e 5, um a cada iteração
- b → indica o índice de cada elemento que será analisado. O JavaScript substitui, automaticamente, pelo respectivo índice do número (a) que estamos analisando

Veja o código:

JavaScript

```
const numeros = [1, 2, 3, 4, 5]
numeros.forEach(function(numero, indice) {
    numeros[indice] = numero * 10;
})

console.log(numeros)
```

```
[ 10 , 20 , 30 , 40 , 50 ]
```

(FGV/SEF MG/2023) Analise o código Javascript a seguir.

```
let a = {x: 1};
let b = {x: 2};
let c = {x: 3};

let arr = [a, b, c];

arr.forEach((obj) => {
obj.x += 1;
});
console.log(a.x, b.x, c.x);
```

O resultado exibido ao rodar esse código será

a) 1 2 3
b) 2 3 4
c) 4 5 6
d) [1, 2, 3]
e) [2, 3, 4]

Comentários:

Criamos três objetos (assunto que veremos logo em seguida), com os valores 1, 2 e 3. Podemos ignorar o fato de ser um objeto, pois estamos selecionando cada elemento no console.log ao final, então considere apenas a = 1, b = 2, c = 3, para fins de resolução da questão. Por fim, criamos a array arr, com os valores de a, b e c.

Depois, passamos a função forEach para cada elemento. Essa função vai pegar cada elemento e somá-lo 1 unidade (x += 1 – que equivale a x = x + 1). De forma que, como resultado, cada unidade da array será acrescida de 1.

O console.log registra cada elemento separado, então eles apareceram como números, não como parte de uma array – 2, 3, 4. (Gabarito: Letra B)

MAP

A função **`map()`** funciona exatamente como o forEach, mas com uma "pequena" mudança: ao invés de manipular a própria array, agora temos como retorno uma nova array e, por isso, devemos atribuí-lo a uma variável. Assim como o `forEach`, a função `map` recebe três argumentos – elemento, índice e array.

JavaScript

```
const numeros = [1, 2, 3, 4, 5]
let numeros10 = numeros.map(function (n) {
  return n*10
})
console.log(numeros)
console.log(numeros10)
```

```
[ 1 , 2 , 3 , 4 , 5 ]
[ 10 , 20 , 30 , 40 , 50 ]
```

(FGV/PC AM/2022) Considere o trecho JavaScript exibido a seguir.

```
const a = [1,2,3,4,5];
const b = a.map(xpto);
alert(b);
function xpto (x, y) {
return x * y;
}
```

Assinale o conteúdo exibido na execução do script acima.

a) 0,2,6,12,20
b) 1,2,3,4,5
c) 1,3,5,7,9
d) 1,4,9,16,25
e) 2,4,6,8,10

Comentários:

Temos uma array a, e, a partir dela, com o comando map, aplicando a função xpto, estamos criando uma nova função. Essa função, a xpto, recebe dois parâmetros, x e y, e retorna a multiplicação entre eles.

O ponto central dessa questão é entender o que é o parâmetro X, e o que é o parâmetro Y. Lembrem, a estrutura da função: map(elemento, indice). Dessa forma, X representa o elemento, e Y o seu respectivo índice. Se rodarmos esse código com return x e return y, podemos entender melhor do que se trata:

X → 1, 2, 3, 4, 5 (são os elementos da array)
Y → 0, 1, 2, 3, 4 (são os índices)

Agora, basta multiplicar os elementos que encontraremos a array B:

b = [(1*0, (2*1), (3*2), (4*3), (5*4)] = [0, 2, 6, 12, 20] (Gabarito: Letra A)

Ordenação

A ordenação de arrays sempre é um ponto importante. Para o JavaScript, temos duas formas principais de fazer a ordenação. A primeira, é através do método **`sort()`**. O `sort()` é o método mais básico de ordenação, ordenando nossa array em ordem alfabética, por padrão. É possível inverter a ordem de uma array também, pelo método **`reverse( )`**.

JavaScript

```
const nomes = ["Rafael", "Maria", "Ana", "Mauro", "Roberta"]
let ordenado = nomes.sort()
console.log(ordenado)
console.log(ordenado.reverse())
```

```
[ 'Ana', 'Maria', 'Mauro', 'Rafael', 'Roberta' ]
[ 'Roberta', 'Rafael', 'Mauro', 'Maria', 'Ana' ]
```

Apesar do método ser aplicado usualmente para ordenar *strings*, podemos manipulá-lo para termos a ordenação que queremos. Veja como faríamos para ordenar os números em ordem crescente:

JavaScript

```
const numeros = [3, 1, 34, 2, 23, 49, 13, 11, 29]
numeros.sort((a, b) => a - b)
console.log(numeros)
```

```
[1, 2, 3, 11, 13, 23, 29, 34, 49]
```

Explicando melhor o que acontece acima:

- numeros.sort(...): Este é o método sort aplicado à array numeros. O sort reorganiza os elementos da array com base em uma função de comparação que você fornece.
- (a, b) => a - b: Esta é a função de comparação que você passa para o sort. Ela recebe dois parâmetros a e b, que são dois elementos da array que estão sendo comparados.
    - Se a função retornar um número negativo, o elemento a será ordenado antes do elemento b.
    - Se a função retornar zero, a ordem entre a e b permanece inalterada.
    - Se a função retornar um número positivo, o elemento b será ordenado antes do elemento a.

Neste caso específico, a - b faz com que a array seja ordenada em ordem crescente, porque se a for menor que b, a subtração será negativa, indicando que a deve vir antes de b na ordenação. Se a for maior que b, a subtração será positiva, indicando que b deve vir antes de a na ordenação. Se a for igual a b, a subtração será zero, mantendo a ordem. Essa é uma forma de ordenação comum, chamada de "Bubble Sort".

Reduce

O `reduce` é um método aplicado a arrays que a reduz a um único valor, iterando sobre cada elemento do array e aplicando uma função de callback fornecida pelo programador. A sua sintaxe é a seguinte:

```
array.reduce(callback(accumulator, currentValue, index, array), initialValue)
```

Explicando melhor cada termo:

- accumulator = accumulator, ou acc, é o valor acumulado retornado pelas chamadas anteriores à função de callback
- currentValue = o valor atual que será processado no array
- index = o índice do elemento atual - *é um parâmetro opcional*
- array = o array para o qual o reduce foi chamado - *é um parâmetro opcional*



Vamos aplicar o reduce em uma array [1, 2, 3, 4, 5], com o objetivo de ter como resultado a soma de todos os elementos dessa array.

JavaScript

```
const array = [1, 2, 3, 4, 5]
const soma = array.reduce(
    (accumulator, currentValue) => accumulator + currentValue, 0)
console.log(soma)
```

```
15
```

O *reduce* basicamente está passando por cada elemento e somando o valor atual (do índice 0) ao valor acumulado, passando um valor inicial de 0. Então teremos:

- 1 + 0 = 1
- 2 + 1 = 3
- 3 + 3 = 6
- 4 + 6 = 10
- 5 + 10 = 15

Espalhamento

O último método que veremos para arrays não é bem um método – e sim um operador. Ele se chama "operador de espalhamento", e é representado por três pontos `...`. Ele serve para desestruturar objetos em geral, especialmente arrays.

Basicamente quebramos cada elemento da array em um elemento unitário. Se pegarmos uma array [1, 2, 3, 4 ,5], é como se tivéssemos 5 elementos isolados. Essa abordagem é especialmente útil quando queremos combinar arrays.

Veja o que acontece se unirmos duas arrays sem o espalhamento, e com ele.

JavaScript

```
const ar1 = [1, 2, 3]
const ar2 = [4, 5, 6]
let juncaoSem = [ar1, ar2]
let juncaoCom = [...ar1, ...ar2]
console.log(juncaoSem) //resultado sem desestruturar
console.log(juncaoCom) //resultado desestruturando
```

```
[ [ 1 , 2 , 3 ], [ 4 , 5 , 6 ] ]
[ 1 , 2 , 3 , 4 , 5 , 6 ]
```

Perceba que, ao desestruturarmos, cada elemento é tratado de forma individual, tornando a junção de duas arrays em uma array apenas de elementos. Caso juntemos sem desestruturar, teremos uma array de 2 elementos apenas, sendo cada um deles uma array autônoma. Muitas vezes esse não é o resultado que queremos.

Objetos

Objetos são coleções de valores em pares chave-valor. As chaves correspondem a *strings* ou símbolos, que representam nome de propriedades. Já os valores podem corresponder a qualquer outro tipo de dado, arrays, *strings*, números e até mesmo outros objetos. Para quem já estudou o paradigma orientado a objetos (POO), os conceitos aqui não são novidade.

Sua sintaxe é caracterizada pela presença de chaves `{}`.

NÃO CONFUNDA:

ARRAYS → COLCHETES [ ];
OBJETOS → CHAVES { };

De forma rápida, objetos costumam ser representações de "coisas" da vida real, como pessoas ou carros (que será o exemplo que usaremos aqui). Cada par chave-valor irá conter um atributo e o valor desse atributo.

JavaScript

```
let carros = {
  carro1: {marca: "Toyota", modelo: "Corolla", ano: 2020}
}
console.log(carros)
```

```
{ carro1: {
    marca: 'Toyota',
    modelo: 'Corolla',
    ano: 2020 }
}
```

Podemos criar *n* objetos dentro de um objeto. Apesar de não ter sido criada como uma linguagem orientada a objeto, e sim a scripts, o JS permite a implementação dos paradigmas da computação orientada a objetos.

Podemos acessar as propriedades do nosso objeto de duas formas:

- `nomeObjeto.nomePropriedade`
- `nomeObjeto["nomePropriedade"]`

Digamos que eu quero acessar a marca do carro 1:

JavaScript

```
let carros = {
    carro1: {marca: "Toyota", modelo: "Corolla", ano: 2020}
  }
  console.log(carros.carro1.marca)
  console.log(carros["carro1"]["marca"]) // não esqueça as aspas
```

```
Toyota
Toyota
```

Podemos criar novos objetos, com base em um objeto pré-definido, através das funções construtoras, que vimos mais cedo nessa aula. Usamos a palavra `new`, seguida do nome do objeto e as propriedades que queremos criar. Para isso, precisamos criar uma classe geral, que será a função construtora. Por fim, podemos instanciar as criações de objetos com o `new`.

JavaScript

```
function Carro(marca, modelo, ano) {
    this.marca = marca;
    this.modelo = modelo;
    this.ano = ano;

    this.descricao = function() {
        return 'Marca: ' + this.marca + ', Modelo: ' + this.modelo +
', Ano: ' + this.ano;
    };
}

// Instanciando objetos a partir da função construtora 'Carro'
var carro1 = new Carro('Toyota', 'Corolla', 2020);
var carro2 = new Carro('Honda', 'Civic', 2019);

console.log(carro1.descricao());
console.log(carro2.descricao());
```

```
Marca: Toyota, Modelo: Corolla, Ano: 2020
Marca: Honda, Modelo: Civic, Ano: 2019
```

Um elemento importante na sintaxe acima é o **`this`**. Ele é uma palavra-chave especial que se refere ao objeto atual no qual o código está sendo executado. O valor dele depende do contexto de execução onde é usado. Em uma função regular, o `this` geralmente se refere ao objeto global (no navegador, o objeto window), a menos que a função seja chamada como parte de um objeto, caso em que o this se refere ao próprio objeto. Em métodos de objetos, `this` se refere ao próprio

objeto ao qual o método pertence. Em funções construtoras, `this` se refere ao novo objeto sendo criado pela função construtora.

Math

Os objetos matemáticos **`math`** são objetos "especiais", que funcionam sem um construtor (ou seja, sem necessidade de criar algo para declará-los). Esses métodos fornecem duas funcionalidades:

- Constantes: podemos adicionar constantes ao código, como `Math.E` (número de Euler) e `Math.PI` (PI).
- Métodos: temos um conjunto de métodos para manipular outros objetos. Eles seguem a sintaxe padrão `Math.método(número)`.

Vamos ver um pouco desses métodos agora.

Arredondamento

Temos 4 funções de arredondamento em JavaScript:

- **`Math.round(x)`** → arredonda para o inteiro mais próximo
- **`Math.ceil(x)`** → arredonda para cima
- **`Math.floor(x)`** → arredonda para baixo
- **`Math.trunc(x)`** → retorna apenas a parte inteira

Vou criar uma função que retornará as 4 formas para o parâmetro que colocarmos.

JavaScript

```
function arredondamento(x) {
    return {
        Round: Math.round(x),
        Ceil: Math.ceil(x),
        Floor: Math.floor(x),
        Trunc: Math.trunc(x)
    };
}
console.log(arredondamento(4.39));
console.log(arredondamento(4.52));
```

```
{ Round: 4, Ceil: 5, Floor: 4, Trunc: 4 } // para 4.39
{ Round: 5, Ceil: 5, Floor: 4, Trunc: 4 } // para 4.52
```


02648216103 - HENRIQUE AMORIM BARRETO

Operações matemáticas

Podemos fazer operações matemáticas com os objetos Math.

- **`Math.pow(x,y)`** → potência, retorna x^y
- **`Math.sqrt(x)`** → retorna a raiz quadrada de X
- **`Math.abs(-x)`** → retorna o valor absoluto (positivo) de um número
- **`Math.min( )`** e **`Math.max( )`** → retorna o valor mínimo ou máximo de uma variável, respectivamente
- **`Math.random( )`** → retorna um valor aleatório, entre 0 (inclusive) e 1 (exclusive)
- **`Math.log(x)`** → retorna o logaritmo natural (base E) de x (ln x)
- **`Math.log2(x)`** → retorna o logaritmo de x na base 2 ($\log_2 x$)

Acredito que o uso mais interessante aqui seja o do **`Math.random( )`**, que é usado para criar os geradores de número aleatório (RNG). Vamos criar um objeto usando essa função, para gerar jogos para a mega sena. Esse código é um pouco avançado, então pode ser confuso para você. Mas tente entender, brincar com ele na sua IDE e ver os resultados.

JavaScript

```
function gerarConjuntoAleatorio() {
    // Criar uma array preenchida com números de 1 a 60
    let numeros = []
    for (let i = 1; i <= 60; i++) {
        numeros.push(i)
    }

    let numerosSorteados = []
    // Iterar 6 vezes para escolher 6 números aleatórios e removê-los
    //da array
    for (let i = 0; i < 6; i++) {
        // Gerar um índice aleatório entre 0 e o comprimento atual da array
        let indiceAleatorio = Math.floor(Math.random() * numeros.length)
        // Remover o número correspondente ao índice aleatório e
        //armazená-lo na array de números sorteados
        numerosSorteados.push(numeros.splice(indiceAleatorio, 1)[0])
    }
    return numerosSorteados.sort((a, b) => a - b)
}
console.log("Números sorteados:", gerarConjuntoAleatorio())
```


02648216103 - HENRIQUE AMORIM BARRETO

OBS: Se jogar na *mega* com esses números gerados e ganhar, aceito 10% do prêmio, viu 🤑

Objetos Chaveados

Os objetos chaveados, ou Keyed Objects, são estruturas de dados que armazenam pares de chave e valor, onde cada chave é única dentro do objeto. Temos dois tipos de objetos chaveados que ganham destaque no JavaScript: os mapas (maps) e os conjuntos (sets).

Map

Os mapas são um conjunto de dados em chave-valor, em que a chave pode ser um dado de qualquer tipo. Isso garante mais flexibilidade ao programa. Importante: não confunda o objeto *map* com a função *map*.

Para declararmos um mapa, usamos a seguinte sintaxe:

```
let novoMapa = new Map( )
```

Podemos criar os mapas de forma vazia, como fizemos acima, ou passando arrays. No caso de atribuirmos valores, podemos criar atribuindo arrays (uma array geral, e outra para cada par de valores), da seguinte forma:

JavaScript

```
const frutas = new Map([
    ["maca", 5.30],
    ["banana", 3.81],
    ["morango", 8.20]
  ]);
  console.log(frutas)
```

```
Map(3) { 'maca' => 5.3, 'banana' => 3.81, 'morango' => 8.2 }
```

Podemos, também, preencher o mapa após sua criação usando o comando `set( )`. Com ele, atribuímos o par de chave e valor, sempre um de cada vez. Esse comando pode ser usado também para substituir determinado elemento dentro do mapa.

JavaScript

```
const frutas = new Map()
frutas.set("maca", 5.30)
frutas.set("banana", 3.81)
frutas.set("morango", 8.20)
console.log(frutas)
```

```
Map(3) { 'maca' => 5.3, 'banana' => 3.81, 'morango' => 8.2 }
```

Existe um conjunto de métodos que são aplicados exclusivamente aos mapas:

- `get("chave")` → retorna o valor da respectiva chave
- `delete("chave")` → deleta o par chave-valor a partir da chave
- `has("chave")` → retorna TRUE se o mapa contém a chave
- `entries()` → retorna todas os pares chave-valor no mapa, em texto

É possível que tenhamos um objeto chamado de WeakMap, ou mapa fraco. Ele, assim como o mapa, é uma coleção de pares de chave-valor cujas chaves devem ser objetos ou símbolos não registrados, com valores de qualquer tipo. A sua diferença é que não há a criação de uma referência forte à chave, de forma que seja possível a coleta pelo coletor de lixo (*garbage colector*). Para criarmos um WeakMap, usamos uma sintaxe muito parecida:

```
const mapaFraco = new WeakMap()
```

Em geral, a escolha entre um Map e um WeakMap baseia-se nas necessidades da aplicação. Se precisamos que os pares de chave e valor sejam mantidos no código, sem serem recolhidos pelo coletor de lixo, usamos o Map. Se queremos uma limpeza do programa, garantindo mais eficiência mas sem armazenamento de estado, utilizamos o WeakMap, já que os pares de chave-valor serão coletados e removidos.

Set

Os sets, ou conjuntos, são nosso segundo conjunto de objetos chaveados, consistindo em uma coleção de valores únicos. Cada valor em um set é único dentro do seu próprio objeto. Podemos criar um set através da seguinte sintaxe:

```
const mySet = new Set()
```

A construção do objeto é muito similar ao mapa – a grande diferença é que agora adicionaremos apenas um valor, não um par de valores. Não trabalhamos mais com chave-valor, e sim unicamente com um valor. Os métodos são similares, porém, para adicionarmos um elemento, agora usamos o método add. Vamos criar um set com algumas bandas que eu gosto!

JavaScript

```
const bandas = new Set ()
bandas.add("The Black Keys")
bandas.add("Bad Omens")
bandas.add("Bring Me The Horizon")
bandas.add("Blink 182")
console.log(bandas)
```

```
Set(4) {'The Black Keys','Bad Omens','Bring Me The Horizon','Blink
182'}
```

Assim como nos maps, em sets temos os WeakSets. Os WeakSets são coleções de valores coletáveis pelo coletor de lixo. A principal diferença para os Sets padrões são:

- WeakSets são uma coleções de objetos e símbolos apenas, enquanto os Sets podem ter qualquer tipo de dado
- WeakSets são fracos, isso é, podem ser coletados pelo coletor de lixo
- WeakSets não são enumeráveis



(FGV/TCE SP/2023) O analista José criou a função CalcObjs em JavaScript. A CalcObjs recebe uma coleção de objetos e realiza determinado cálculo em cada objeto da coleção. No caso de existirem objetos repetidos, a CalcObjs considera apenas a primeira ocorrência do objeto na coleção. A fim de rastrear os objetos já processados, José utilizou uma estrutura de dados disponível no JavaScript, do tipo keyed collection, cujos valores são únicos entre si e passíveis de remoção pelo coletor de lixo.

Para rastrear os objetos processados, José utilizou a estrutura de dados do JavaScript:

a) Map;
b) Symbol;

c) WeakSet;
d) SharedArrayBuffer;
e) Uint8ClampedArray.

Comentários:

As características do tipo de objeto são:
• Pertence aos keyed collections – Maps, sets e suas versões weak
• Valores são úncios entre si – Apenas sets e sua versão weak
• Podem ser removidos pelo coletor de lixo – Apenas o WeakSet

Sendo assim, ficamos com a letra C como gabarito. (Gabarito: Letra C)

Condicionais

As estruturas condicionais em programação, como as encontradas em JavaScript, são utilizadas para controlar o fluxo de execução do código com base em condições específicas. Elas permitem que um programa tome decisões dinâmicas, executando diferentes blocos de código dependendo se uma expressão condicional é avaliada como verdadeira ou falsa.

No JS, temos duas estruturas condicionais:

- **`if...else`**
- **`switch`**

If... else

A condicional **`if...else`** é, talvez, a mais tradicional forma de implementarmos condicionais em JavaScript. Delimitamos uma condição no **`if`** e, caso ela seja verdadeira, executaremos o bloco de código contido dentro da condição. Ele funciona como um "se" em português - *se* a condição for obedecida, executaremos o bloco.

A sintaxe geral do código é:

```
if(condição) {
// bloco a ser executado se verificada a condição
}
```

Digamos que você quer rodar um código de bloco apenas para pessoas com mais de 18 anos. Você faria uma condicional assim. Veja:

JavaScript

```
function verificadorIdade(idade) {
    if (idade >= 18) {
        return "Entrada liberada"
}}

console.log(verificadorIdade(18))
```

```
Entrada liberada
```

Agora, e se a idade for menor que 18? O que acontece? Temos como retorno undefined, já que não temos uma resposta definida para esse valor. Nesse caso, implementamos o **`else`**. Ele implementa um bloco de código residual, caso a condição do if não seja implementada.

```javascript
function verificadorIdade(idade) {
    if (idade >= 18) {
        return "Entrada liberada"
    } else {
        return "Entrada proibida"
    }
}
console.log(verificadorIdade(16))
```

```
Entrada proibida
```

Mas Felipe, só podemos trabalhar com duas condições?

Não, meu caro! Podemos fazer uma junção de **`else if`** para implementar condições intermediárias. Podemos ter quantos *else if* quisermos em uma condicional. Só é recomendado que sempre terminemos a condicional com um *else*, para acobertar casos residuais.

Vamos criar um sistema que analisa a nota de alunos e decide se está aprovado, em recuperação ou reprovado de cara.

```javascript
function aprovado(nota) {
    if (nota >= 7) {
        return "Aprovado"
    } else if (nota < 7 && nota >= 5){
        return "Em recuperação"
    } else {
        return "Reprovado"
    }}
console.log(aprovado(6))
```

```
Em recuperação
```

(FGV/PC AM/2022) Analise o código JavaScript a seguir.

```
function enigma(n) {
   if (n == 0 || n == 1) {
      return n;
   } else {
      return enigma (n-1) + enigma (n-2);
```

}}

Assinale o valor retornado para a expressão enigma(4).

a) 21
b) 13
c) 8
d) 5
e) 3

Comentários:

Temos dois blocos de execução na função enigma – o if será executado se n = 0 OU n = 1, e o else será executado nos demais casos. O problema é que o bloco else é recursivo – ou seja, sempre que cairmos nele, vamos rodar a própria função novamente.

Então, como estamos passando *enigma(4)*, ou seja, *n = 4*, acontece o seguinte:

- enigma(4) → caímos no bloco *else*, cujo valor será enigma(3) + enigma(2). Para acharmos o valor, precisamos continuar descendo até chegarmos no bloco if.
- enigma(3) → novamente, no bloco *else*, o valor será *enigma(2) + enigma(1)*
- enigma(2) → novamente, no bloco *else*, o valor será *enigma(1) + enigma(0)*
- enigma(1) → agora vamos no bloco *if*, onde o valor de enigma = n. Portanto, enigma(1) = 1
- enigma(0) → novamente, bloco *if*, portanto enigma(0) = 0

E finalmente chegamos no "fundo do poço" rs agora podemos voltar subindo, somando as variáveis. Vou usar cores para que você visualize melhor.

- enigma(0) = 0
- enigma(1) = 1
- enigma(2) = enigma(1) + enigma(0) = 1 + 0 = 1
- enigma(3) = enigma(2) + enigma(1) = 1 + 1 = 2
- enigma(4) = enigma(3) + enigma(2) = 2 + 1 = 3

Por fim, chegamos no *enigma(4)* = 3. (Gabarito: Letra E)

Condicional ternária

Pode ser só opinião minha, mas você não achou essa estrutura *if...else* muito grande? Bom, vários desenvolvedores concordam comigo – e, por isso, surgiu o que chamamos de condicional ternária. Ela é uma forma de reescrever essas condicionais de forma mais limpa, concisa e ocupando menos espaço de código. Sua estrutura é:

```
condição ? valorSeVerdadeiro : valorSeFalso
```

Retomando o exemplo de idade, vamos fazer o verificador em uma ternária.

JavaScript

```
let status = (idade) => idade >= 18 ? 'Maior de idade' : 'Menor de idade'
console.log(status(18))
```

```
Maior de idade
```

Veja que combinamos as arrow function com condicionais ternárias para reduzir o código a um tamanho bem menor que anteriormente. À primeira vista pode parecer que a ternária só aceitaria um par de if else, mas podemos encadear vários if no meio da nossa condição, da seguinte forma

```
condição ? if : elseIf : elseIf : elseIf ... : else
```

Vamos criar um sistema que atribuirá a nota do padrão numérico ao padrão alfabético (A, B, C, D e F), usando apenas arrow functions e ternárias.

JavaScript

```
let resultado = (nota) => nota >= 90 ? 'A' : //if
                          nota >= 80 ? 'B' : //else if
                          nota >= 70 ? 'C' : //else if
                          nota >= 60 ? 'D' : //else if
                                       'F'   //else
console.log("Nota final:", resultado(70))
```

```
Nota final: C
```

Switch

O `switch` é uma abordagem condicional um pouco diferente do *if...else*, onde expressamos *n* condições em uma lista através da cláusula `case`. Adicionalmente, a cada cláusula, adicionamos o elemento `break`, que irá interromper a execução do código e finalizá-lo, partindo para a próxima expressão.

O *switch* funciona como uma função, e tem a seguinte estruturação:

```
switch(expressão) {
      case ‘caso1’:
      //bloco de código a ser executado
      break
case ‘caso2’:
      //bloco de código
      break
default:
      //bloco padrão, se não encontrar outra correspondência
}
```

O **break** é de suma importância. Caso não seja previsto na estrutura, ao encontrar uma condição, o código irá executar as demais condições, mesmo que não sejam satisfeitas, caindo sempre no caso *default*. Esse problema não se aplica quando o retorno do bloco de código executado consiste em um **return**. Isso pois o *return* interrompe imediatamente a execução do bloco, e encerra a execução. Vamos a um exemplo?

JavaScript

```javascript
function opcao(x) {
    switch (x) {
        case 1:
            return "Opção A selecionada";
        case 2:
            return "Opção B selecionada";
        case 3:
            return "Opção C selecionada";
        default:
            return "Opção inválida";
    }
}
console.log(opcao(3));
```

```
Opção C selecionada
```

Perceba que o x , passado em switch(x) é comparado em cada caso. E a condição de implementação é definida no case – no nosso caso, se x = 1, retorna a opção A, x = 2, retorna a opção B, e assim por diante.

Alguns detalhes do switch:

02648216103 - HENRIQUE AMORIM BARRETO

- Se tivermos mais de um caso de correspondência, apenas o primeiro switch é executado
- Se não tivermos a opção default, o código continua sem apontar resultado
- O switch usa comparações estritas, ou seja, `===` .

Laços

Os laços, ou loops, são estruturas de controle que permitem a execução repetida de um bloco de código enquanto uma condição específica é verdadeira, ou até que uma condição de término seja alcançada. Temos três tipos de laços em JavaScript:

- **`for`**
- **`while`**
- **`do... while`**

For

O loop **`for`** é um laço que é executado em uma quantidade definida de vezes - ou seja, devemos o escolher quando sabemos quantas vezes queremos executar o bloco de códigos. Para isso, usamos a seguinte sintaxe:

```
for (expressao 1; expressao 2; expressao 3) {
     código a ser executado
}
```

A expressão 1 define a variável com que trabalharemos. A expressão 2 define a condição para rodarmos o loop, e a expressão 3 define o valor a ser repassado para a variável toda vez que executarmos o loop.

Digamos que precisamos preencher uma array com números de 1 a 100. Pensa, digitar de 1 em 1 vai ser uma trabalheira, né? Mas com o loop isso é feito em segundos. Nós vamos:

- Criar nossa variável minhaArray (uma array vazia)
- Executar o loop *for,* começando em 1 e indo até 100, subindo de 1 em 1. Então, nossas três expressões serão:
    - Expressão 1: criaremos uma variável local i, que começa com 1
    - Expressão 2: nosso limite superior é quando a variável i chegar a 100
    - Expressão 3: a cada iteração, adicionaremos 1 ao i (i++). Cuidado para não colocar ++i, dessa forma adicionaremos o valor antes de concluir o laço.
- Faremos um *push(i)* para inserir o valor de *i* na array.

JavaScript

```
const minhaArray = []
for (let i = 1; i <= 100; i++) {
    minhaArray.push(i)}
console.log(minhaArray)
```

```
[ 1, 2, 3, 4, 5, 6, 7, ..., 98, 99, 100 ]
```

Então, de forma geral, definimos o valor inicial, o valor final e a forma de incremento/decremento após cada laço. Bom, agora que você sabe o "básico" do *for*, vamos ver suas duas variações: o **`for in`** e o **`for of`**.

For...in

O **`for...in`** executa loops em **`propriedades de objetos`**. Sua sintaxe padrão é:

```
for (chave in object) {
     bloco de código a ser executado
}
```

Então, por exemplo, podemos usar o `for...in` para iterar sobre um objeto e retornar todos os seus componentes em texto.

JavaScript

```
const pessoa = {
    nome: 'Felipe',
    snome: 'Mathias',
    idade: 30
}
let texto = "" //string vazia
for (let x in pessoa) {
    texto += pessoa[x] + " " //equivale a texto = texto + pessoa[x]
}                            //o " " é para dar um espaço
console.log(texto)
```

```
Felipe Mathias 30
```

Outro exemplo – se quisermos contar quantos elementos específicos temos na array. Aqui, por exemplo, vou contar o número de 0s.

02648216103 - HENRIQUE AMORIM BARRETO

JavaScript

```
const array = [1, 3, 4, 5, 1, 3, 2, 0, 1, 0, 1, 3, 4, 5];
let contador = 0

for (let x in array) {
    if (array[x] === 0) {
        contador++
    }}
console.log(contador)
```

```
2
```

For..of

O **`for...of`** faz loops através dos valores de um objeto iterável – isso é, array, strings, objetos, entre outros. Digamos que você tenha uma array e queira *printar* cada elemento dela, mas de forma separada. Se você simplesmente der um *console.log(minhaArray)*, ele irá listar tudo no mesmo resultado do comando – que não é o que você quer.

Para resolver isso, usamos o *for...of*. Ele irá iterar sobre cada elemento de determinado objeto, conforme dada condição, e aplicará o bloco de código sobre cada um dos elementos. Sua sintaxe base é:

```
for (variavel of iteravel) {
código a ser executado em cada iteração
}
```



Exemplo: digamos que você tenha uma array em decimais, mas determinado programa exige que esses números estejam em porcentagem. Vamos criar um *for...of* para jogar cada um dos elementos dessa array antiga numa nova.

JavaScript

```javascript
const arrayOriginal = [0.1, 0.2, 0.3, 0.4, 0.5];
let novaArray = []
//primeiro for...of irá modificar os elementos na array
for (let elemento of arrayOriginal) {
    novaArray.push(elemento * 100)
}
//segundo for...of irá fazer o print, elemnto por elemento
for (let x of novaArray) {
    console.log(x + "%")
}
```

```
10%
20%
30%
40%
50%
```

While

O loop **while** é executado enquanto sua condição implementada for TRUE - ou seja, escolhemos essa opção quando sabemos a condição, mas não sabemos a quantidade de vezes que queremos iterar. A partir do momento que a condição verificada é FALSE, rompemos o laço. Sua estrutura é:

```
while (condição) {
//bloco executado se condição for TRUE
}
```

Um exemplo bem simples para você entender:

JavaScript

```javascript
let i = 0
while (i <= 2) {
    console.log("O número é", i)
    i++
}
```

```
O número é 0
O número é 1
O número é 2
```

Aqui, uma coisa interessante. Cuidado com a posição do incremento no código, se ele vier antes do *console.log*, teremos um resultado diferente:

JavaScript

```
let i = 0
while (i <= 2) {
    i++
    console.log("O número é", i)
}
```

```
O número é 1
O número é 2
O número é 3
```

O `while` também tem uma forma derivada o **`do...while`**, que, numa tradução literal, "faça... enquanto". A sua estrutura aqui é:

```
do {
     //bloco a ser executado enquanto a condição for satisfeita
} while (condição)
```

Veja que agora a execução do bloco é feita antes de ser verificada a condição. Ou seja: sempre iremos, ao menos uma vez, executar o bloco. Analise o código abaixo e veja se você entende o que acontece:

JavaScript

```
let i = 5
do {
    console.log("O número é", i)
    i++
} while (i <= 4)
```

```
O número é 5
```

Perceba, mesmo que a condição não seja obedecida (o que é nosso caso, já que o *i* inicia em 5, o que já é maior ou igual a 4), executaremos o bloco ao menos uma vez.

Tratamento de Exceções

Ao executar um código, mais frequentemente do que você pensa, ocorrem erros. Seja por falta de conexão com uma API, uma entrada de dado errada, um erro de código mesmo, vários são os fatores que podem gerar um erro no código.

Para lidar com isso, o JavaScript oferece um par de operadores que "detectam" o erro, e fazem alguma coisa com ele. Esse par de operadores é o `try...catch`. Vamos ver sobre esse bloco de código agora.

Try...Catch

O par de operadores `try...catch` funciona conjuntamente, de forma que o:

- Try: testa um bloco de código
- Catch: bloco de código para lidar com os erros

Sua sintaxe genérica segue abaixo:

```
try {
bloco de código sendo testado
} catch(err) {
bloco de código para lidar com o erro
}
```

O seu se dá da seguinte forma:

- O bloco de código dentro do bloco `try` é executado. Se não houver erros, a execução continua e nenhum bloco posterior é ativado.
- Se ocorrer algum erro, a execução do bloco é interrompida e o código "pula" para o bloco `´catch´`, passando o objeto de erro como um parâmetro (o `(err)` que temos acima) dentro do *catch*
- Após o tratamento do erro, o código continua sendo executado normalmente

Um detalhe importante é que, para cada `try` só teremos um catch. O JavaScript não permite vários *catch* para um mesmo `try`.

Um operador adicional pode ser adicionado: o `finally`. O `finally` é executado após todos os *try* e *catch*, e será executado `sempre`, independentemente do resultado. Ficamos, então, com essa estrutura:

```
try {
bloco de código sendo testado
} catch(err) {
bloco de código para lidar com o erro
} finally {
bloco executado independente dos resultados acima, sempre
}
```

Como objetos de erro, podemos ter alguns resultados:

ERRO	DESCRIÇÃO
EvalError	Erro na função eval( )
RangeError	Um número fora do alcance da variável
RefrenceError	Referência inválida ocorreu
SyntaxError	Erro de sintaxe
TypeError	Erro de tipo
URIError	Erro ocorreu no encodeURI( )

Quando queremos identificar um erro, podemos optar por expor seu nome, ou seja, o objeto de erro, através do comando **name**, ou uma mensagem, com o comando **message**. Vamos ver como fica isso em código:

JavaScript

```
try {
  //vamos usar a função eval() para avaliar um bloco de código
  eval("alert('Olá)");   //a falta de um ' irá acusar erro de sintaxe
}
catch(err) {
  console.log(err.name)
  console.log(err.message)
}
```

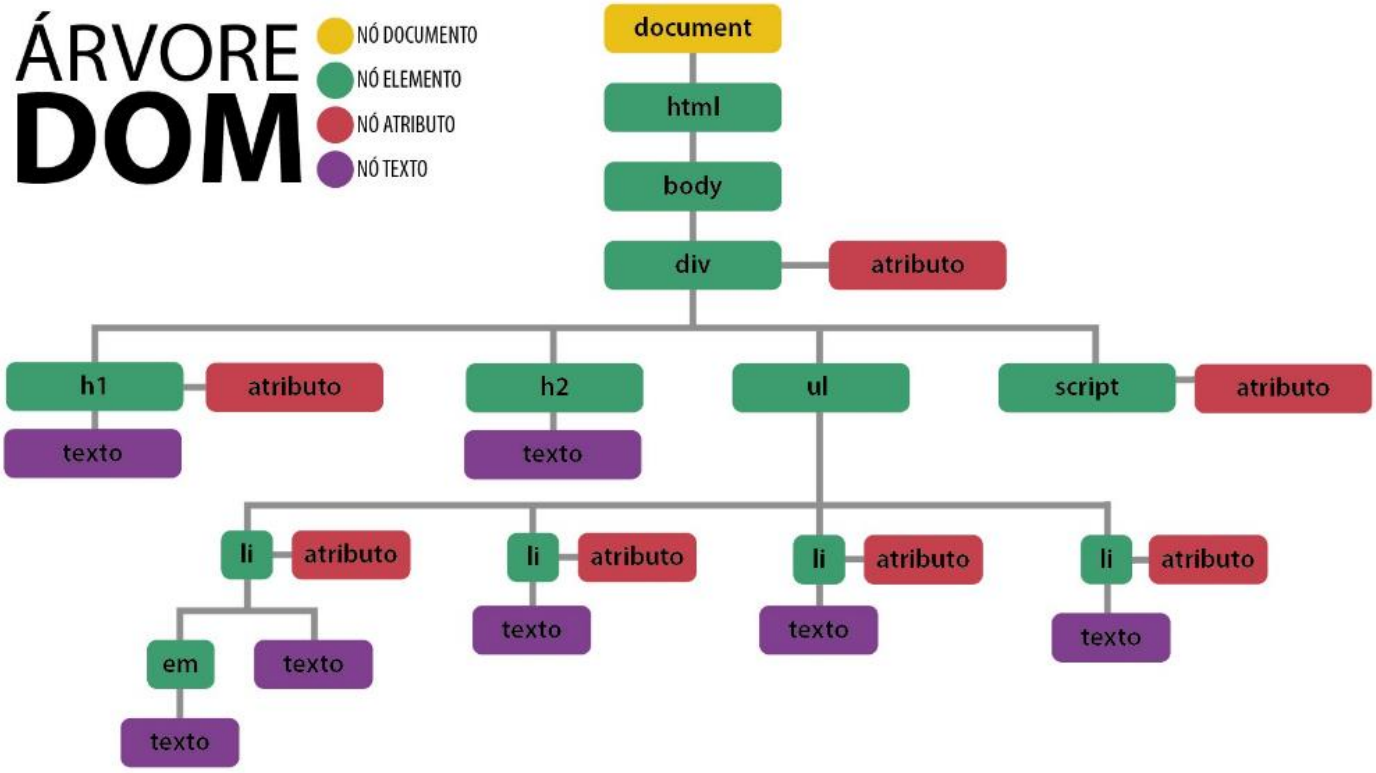
```
SyntaxError
Invalid or unexpected token
```

Manipulação do DOM



Vimos o que é, de forma geral, o JavaScript. Agora espero que você tenha entendido o funcionamento geral da ferramenta, pois entraremos no real propósito da existência do JavaScript: interagir com os documentos HTML para tornar as páginas mais dinâmicas. Isso é feito através da manipulação do DOM.

DOM, ou Document Object Model, são os diferentes objetos de HTML presentes em uma página - *divs*, listas, botões, *headers* etc. O DOM é construído a partir de uma estrutura hierárquica, em um formato de árvore. Veja:



De forma geral, o intuito do JavaScript é aplicar suas funções para manipular os objetos do HTML, tornando as páginas estáticas em páginas dinâmicas. Assim, podemos ter menus que abrem e fecham ao clicarmos, mensagens aparecendo quando fazemos algo errado, inserção de dados, entre outros, que tornam as páginas da internet em verdadeiros aplicativos Web.

Tudo isso é feito a partir de um conjunto de métodos para

- Identificar e selecionar o elemento que queremos alterar
- Alterar esse elemento, tanto no que tange ao HTML, quanto ao CSS

Vamos ver como selecionar determinado elemento, o que podemos fazer com esses elementos, entre outros – obviamente que dentro de uma perspectiva orientada a provas, não à prática do dia a dia.

Selecionando Objetos no DOM

O primeiro passo para manipularmos determinado objeto dentro do nosso DOM é, justamente, selecioná-lo. Para isso, temos alguns métodos específicos. O primeiro deles é o **`document.getElement*`**.

O **`document`** indica que estamos trabalhando com o DOM do HTML. Já com o **`getElement*`**, selecionamos alguma propriedade específica do elemento, como *id*, *class*, entre outros. Basta selecionarmos o tipo de propriedade que queremos, e substituir no asterisco que eu deixei ali.

Vou passar para você as três principais formas com que usamos o `getElement`:

- **`document.getElementById()`** → retorna o elemento com o ID especificado
- **`document.getElementsByClassName()`** → retorna elementos pertencentes a uma classe específica. Cuidado que aqui é *element**s***, não *element*
- **`document.getElementsByTagName()`** → retorna elementos de determinada tag, como *div*, *h1*, *button* etc. Novamente, aqui é *elements*.

Digamos que a gente tenha o código HTML abaixo:

HTML

```
<h1 class="classeAula" id="idAula">
  <p>Olá, mundo!</p>
</h1>
```

Podemos acessar esse *header* (a tag <h1>) de três diferentes formas:

- Pelo seu ID - idAula
- Pela sua classe - classeAula
- Pela sua tag - <h1>

```javascript
let selecionarID = document.getElementById("idAula")
let selecionarClasse = document.getElementsByClass("classeAula")
let selecionarTAG = document.getElementsByTag(“h1”)
```

(FGV/MPE SC/2023) No contexto do JavaScript, analise um trecho do HTML de uma página web.

Texto de teste 1
Texto de teste 2

A linha de código que altera o tamanho da fonte do segundo parágrafo acima é:

a) document.getElementById("234").style.fontSize="48px";
b) document.getElementById("xpto.234").style.fontSize="48px";
c) document.getElementById(p<"234">).style.fontSize="48px";
d) document.getElementsById("div.p").style.fontSize="48px";
e) xpto.getElementById("234").style.fontSize="48px";

Comentários:

Vamos lá, você já sabe o esquema para resolver essas questões. O primeiro passo é definir como selecionar o elemento – e, vendo nas alternativas, está sendo usado o getElementById. Portanto, já que o segundo parágrafo tem como ID “234”, usamos get.ElementById(“234”). Só com isso, já acharíamos a resposta. Mas seguiremos.

Queremos alterar o tamanho da fonte, que é um recurso fornecido pelo CSS – portanto, usaremos a diretiva style. A fonte é alterada através do fontSize. Dessa forma, nossa sintaxe fica:

document.getElementById(“234”).style.fontSize=”48px”

A alternativa correta é, portanto, a letra A. (Gabarito: Letra A)

Porém, temos um outro seletor que é mais "global", e vem disparando no uso para selecionar objetos do DOM: o `querySelector()`. Aqui, não precisamos especificar na sua sintaxe o tipo de propriedade que estamos procurando, isso é feito através de um seletor dentro da função - seguindo o mesmo padrão dos seletores usados no CSS:

CLASSES → PONTO . ;
ID → CERQUILHA # ;
TAGS → NOME DA TAG;

Acontece que o `querySelector` irá selecionar apenas o primeiro elemento correspondente ao parâmetro. Então, se tivermos mais de uma classe ou ID no documento, ele deixará de selecionar os demais - e pode ser que não queiramos isso. Caso seja necessário selecionar todos os elementos de determinada propriedade, usamos o `querySelectorAll( )`.

(Instituto Verbana/UFG/2023) No contexto da linguagem JavaScript, a função do método querySelector( ), é

a) alterar o estilo visual de um elemento HTML.
b) manipular dados em um banco de dados relacional.
c) executar uma requisição de rede de forma assíncrona.
d) capturar um elemento HTML com base em um seletor CSS.

Comentários:

O querySelector destina-se a capturar um elemento HTML do DOM, utilizando seletores CSS. (Gabarito: Letra D)

Partindo do mesmo código HTML de antes, vamos fazer as seleções a partir do querySelector. Vamos usar:

- ID = #idAula
- Classe = .classeAula
- Tag = h1

JavaScript

```
let selecionarID = document.querySelector("#idAula")
let selecionarClasse = document.querySelector(".classeAula")
let selecionarTAG = document.querySelector("h1")
```

Importante destacar: os seletores devem vir entre aspas (simples ou duplas).

Métodos do DOM

Selecionamos o objeto que queremos, agora vamos efetivamente trabalhar com esse elemento. Existem alguns métodos específicos, cada um com um objetivo diferente. Vou setorizar os comandos com base em sua finalidade, conforme classificado pela W3C. Tente dar uma lida e entender o que tudo faz, já que sua sintaxe acaba sendo bem literal.

MÉTODO	DESCRIÇÃO
ENCONTRAR OBJETOS (*document.método*)	
`getElementById()`	Encontra elemento pela ID
`getElementsByClassName()`	Encontra elementos pela classe
`getElementsByTagName()`	Encontra elementos pela tag
`querySelector()`	Método que aceita input de diversas propriedades como parâmetro
`querySelectorAll()`	Adaptação do query selector para selecionar todos os elementos respectivos
MANIPULAR ELEMENTO (*element.método*)	
`innerHTML = ...`	Muda o conteúdo de um elemento
`attribute = ...`	Muda o valor de determinado atributo
`property = ...`	Altera o estio de um elemento
`setAttribute(atributo, valor)`	Muda o valor de determinado atributo, permitindo selecionar o atributo dentro de um conjunto
ADICIONAR OU REMOVER ELEMENTOS (*document.método*)	
`createElement()`	Cria um elemento
`removeChild()`	Remove elementos filho
`appendChild()`	Adiciona um elemento filho
`replaceChild(velho, novo)`	Substitui um elemento
MANIPULAR ESTILOS (*style.método*)	
`backgroundColor = ...`	Altera a cor do fundo do elemento
`color = ...`	Altera a cor do texto do elemento
`fontFamily = ...`	Altera o tipo de fonte do elemento
`fontSize = ...`	Altera o tamanho da fonte do elemento
`fontWeight = ...`	Altera o peso (negrito) da fonte no elemento
`margin = ...`	Altera a margem de um elemento
`padding = ...`	Altera o preenchimento (*padding*) de um elemento

Vamos brincar com nosso código, e aplicar essas funções? Vou incrementar o código um pouco para podermos trabalhar um pouco mais. Veja:

HTML

```
<body>
  <h1 class="classeAula" id="idAula">
    <p id="paragrafo">Olá, mundo!</p>
    <button id="meuBotao">Clique aqui</button>
  </h1>
  <script src="script.js"> </script>
</body>
```

1) Alterando o conteúdo do parágrafo

JavaScript

```
// Selecionamos a tag p
var paragrafo = document.querySelector('p')
// Altera o conteúdo do parágrafo
paragrafo.innerHTML = "Assim que se muda o conteúdo"
```

2) Mudando o estilo (CSS) do botão

JavaScript

```
// Seleciona o botão dentro do <h1> usando querySelector
var botao = document.querySelector('h1.classeAula > button')
  // Define estilos CSS inline para o botão
botao.style.backgroundColor = "blue"
botao.style.color = "white"
botao.style.border = "none"
botao.style.padding = "10px 20px"
botao.style.cursor = "pointer"
```

3) Criando um novo elemento e definindo sua classe

JavaScript

```
/// Criar um novo elemento <p>
var novoParagrafo = document.createElement('p')
// Adiciona a classe 'paragrafo2' ao novo parágrafo
novoParagrafo.className = 'paragrafo2';
```

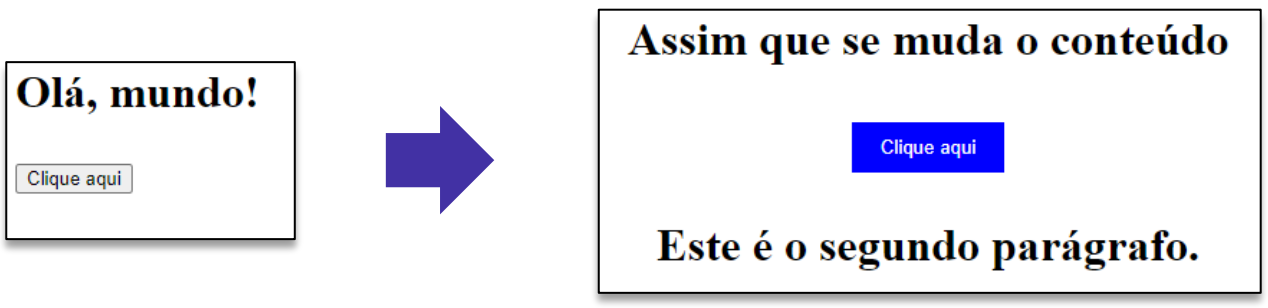
3) Adicionando um texto ao novo parágrafo, e colocando-o dentro de h1

```javascript
// Adiciona o novo parágrafo ao final do <h1>
document.querySelector('h1.classeAula').appendChild(novoParagrafo)
// Define o texto do novo parágrafo
novoParagrafo.textContent = 'Este é o segundo parágrafo.'
```

4) Manipulando o *body* para centralizarmos todos os elementos

```javascript
// Seleciona o body
var body = document.querySelector('body');
// Aplica o estilo para alinhar o conteúdo ao centro
body.style.textAlign = 'center';
```

E, com um simples código, conseguimos transformar nossa página!



(FCC/MPE PB/2023) Para mudar, com JavaScript, o conteúdo de um elemento <h1 id= "title"> para "MPEPB123", utiliza-se o comando

a) document.getElementById("title").text = "MPEPB123";
b) document.innerHTML("h1#title") = "MPEPB123";
c) document.getElementById("#title").innerHTML = "MPEPB123";
d) document.text("#title") = "MPEPB123";
e) document.getElementById("title").innerHTML = "MPEPB123";

Comentários:

O primeiro passo para modificarmos um elemento, é selecionando-o. A questão traz o uso da função getElement, então, uma das formas de retornar esse elemento seria:

document.getElementById(“title”)

Dessa forma, recuperamos o elemento pelo seu ID, usando o getElement. Lembrando que, como já especificamos que é o id na sintaxe, não precisamos por #title.

Para inserir um texto em um elemento, usamos o comando innerHTML. Portanto, nosso comando será:

document.getElementById("title").innerHTML = "MPEPB123";

Portanto, correta a letra E. (Gabarito: Letra E)

Eventos

Eventos são determinadas ações que acontecem no HTML, e que podem ser identificadas, capturadas e manipuladas por scripts do JavaScript. Os eventos podem ser desencadeados por interações específicas do usuário, como clicar em um botão, passar o mouse por cima de uma imagem, envio de formulários, carregamento de página, entre outros.

Esses eventos são previstos, usualmente, dentro da *tag* HTML de determinado objeto. Por exemplo, quando você clica em um botão e abre uma janelinha, ou acontece algum efeito, temos um evento *onClick* adicionado – ficando assim:

```
<button onclick=”código de JavaScript”> </button>
```

Temos uma infinidade de eventos e a maioria deles são bem literais – desde que você saiba inglês. Vou trazer uma listinha para vocês dos 12 principais.

EVENTO	DESCRIÇÃO
`onClick`	Ocorre quando o usuário clica
`onMouseOver`	Ocorre quando o cursor é movido sobre um elemento
`onKeyDown`	Ocorre quando uma tecla é pressionada
`onKeyUp`	Ocorre quando a tecla é liberada, após ser pressionada
`onChange`	Ocorre quando há mudança em algum formulário
`onSubmit`	Ocorre quando algum formulário é enviado

onLoad	Ocorre quando a página é carregada
onUnload	Ocorre quando a página está sendo fechada ou redirecionada
onBlur	Ocorre quando um elemento perde o foco
onDblClick	Ocorre quando ocorre um clique duplo no elemento

Podemos adicionar eventos através do JavaScript passando as funções acima, todas minúsculas (sem camelCase). Vamos para dois exemplos – vou adicionar um evento que acontece no carregamento da página, e outro que acontece ao você clicar no botão.

JavaScript

```
// Evento onload para o botão
window.onload = function() {
  alert("Olá, aluno!")}
// Evento onclick para o botão
document.querySelector("#meuBotao").onclick = () => alert(“clicou")
```

Veja que cada código está passando uma função diferente para o evento. A primeira função lhe trouxe no formato tradicional, a segunda com uma *arrow function*. Podemos manipular os eventos também a partir da diretiva **addEventListener**. É uma forma diferente, de fazer, basicamente, a mesma coisa. Aqui, a sintaxe é a seguinte:

```
elemento.addEventListener(evento, função, useCapture)
```

Aqui, os parâmetros equivalem a:

- Evento → uma string que caracteriza o tipo de evento que queremos, por exemplo “click”, “mouseover”, basicamente uma conversão dos eventos, retirando o *on*;
- Função → a função que rodará a partir do evento
- useCapture → um valor booleano que indica se o evento deve ser capturado durante a fase de captura (true) ou borbulhamento (false). O padrão é *false*.

> A fase de captura e a fase de borbulhamento são duas fases do fluxo de eventos (event flow) que ocorrem quando um evento é disparado em um elemento HTML. A fase de captura é quando um evento é tratado pelos elementos pai antes de chegar ao elemento alvo. A fase de borbulhamento é quando o evento é tratado pelos elementos filhos antes de subir de volta para os elementos pais.

Vamos converter o evento de click para usar o *addEventListener*.

JavaScript

```
let clique = () => alert("clicou")
document.querySelector("#meuBotao").addEventListener("click", clique)
```

QUESTÃO DE PROVA



(FCC/MPE PB/2023) Considere o formulário HTML e a função JavaScript abaixo.

```
<form method="post">
   <label for="n">Usuário:</label>
   <input type="text" name="user" size="30" id="n">
   <label for="s">Senha:</label>
    <input type="password" name="senha" size="30" id="s">
     <input type="button" value="Entrar" ..I.. >
</form>
<script>
function verificar(u, s) { }
</script>
```

Para que ao clicar no botão a função verificar seja chamada, passando o conteúdo dos campos user e senha como parâmetros, a lacuna I deverá ser preenchida corretamente com

a) onclick="verificar(user.value, senha.value)"
b) onEvent("click")="verificar(user, senha)"
c) onclick="verificar(this.user, this.senha)"
d) onclick="verificar(user, senha)"

e) onEvent("click")="call verificar(user.value, senha.value)"

Resolução:

Queremos que, ao clicar no botão, a verificação seja chamada, com os parâmetros user e senha – portanto, queremos um evento onclick(). Bom, logo de cara já descartar as letras b e e, até porque onEvent nem existe.

A sintaxe do onclick é a seguinte:

```
onclick="função(param1, param2)"
```

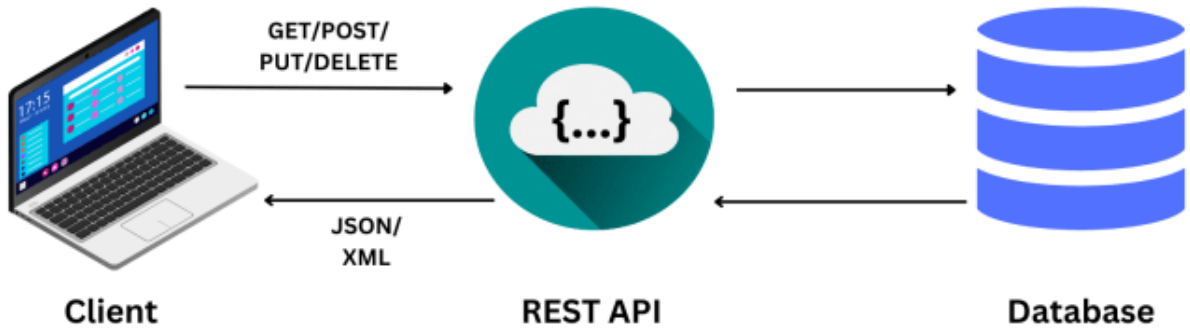
Temos como função verificar, o parâmetro 1 seria o valor de user, ou user.value, e o segundo o valor da senha, senha.value. Nossa sintaxe ficaria assim:

```
onclick="verificar(user.value, senha.value)"
```

A alternativa correta é, portanto, a letra A. (Gabarito: Letra A)

APIs e JavaScript

As APIs (Application Programming Interface) são pontes que conectam dois sistemas diferentes. Elas definem um conjunto de regras e protocolos utilizados para pedir e receber dados. Na *web*, é através da API que interagimos com a parte do back-end da empresa, principalmente com os bancos de dados.



Por exemplo, o usuário digita um formulário de cadastro, com, entre outros, nome, idade e senha. Precisamos que essas informações cheguem ao banco de dados, fiquem registradas e retornem ao usuário, de forma que ele possa usar seu *login* e *senha* para acessar o serviço. Além disso, essas informações precisam ser permanentes, já que queremos que o usuário acesse o site novamente, sem precisar passar por todo esse processo. Para isso, usamos as APIs.

As interações do JavaScript com as APIs são feitas através de métodos HTTPs. Temos 4 métodos principais que você obrigatoriamente deve saber:

MÉTODO	DESCRIÇÃO
GET	Usado para solicitar dados de um recurso específico. As solicitações GET são frequentemente usadas para recuperar informações e não devem alterar o estado do servidor.
POST	Usado para enviar dados para serem processados por um recurso específico. As solicitações POST são usadas para criar novos recursos ou realizar operações que causam alterações no estado do servidor.
PUT	Usado para enviar dados para substituir completamente um recurso existente ou criar um recurso se ele não existir. As solicitações PUT são frequentemente usadas para atualizar completamente um recurso com novos dados.
DELETE	Usado para solicitar a remoção de um recurso específico. As solicitações DELETE são usadas para excluir recursos do servidor.

Temos duas formas de realizar essas requisições: através da **API fetch**, ou da biblioteca JavaScript Naxios. Vamos vê-las.

Fetch

O **`fetch`** é uma API moderna do JavaScript que permite fazer requisições HTTP assíncronas para recuperar recursos de uma URL específica. Ele é nativo do navegador e é suportado em todos os principais navegadores modernos, bem como no Node.js com algumas adaptações. O `fetch` retorna uma `Promise`, que é resolvida com um objeto `Response` representando a resposta da requisição.

Uma **`Promise`** é um objeto em JavaScript que representa o resultado eventual (sucesso ou falha) de uma operação assíncrona. Ela é usada para lidar com operações assíncronas de uma maneira mais limpa e eficiente do que os callbacks tradicionais, permitindo encadear várias operações assíncronas de forma síncrona. Uma Promise pode estar em um dos três estados: pendente, resolvida (com sucesso) ou rejeitada (com falha). Isso permite um melhor controle do fluxo de execução e uma manipulação mais eficiente de operações assíncronas em JavaScript.

A sintaxe genérica para utilizar as promises com o Fetch é a seguinte:

```
fetch(url, options)
  .then(response => {
    // Manipula a resposta da requisição
  })
  .catch(error => {
    // Manipula erros de requisição
  });
```

Onde:

- `url` é a URL para a qual você deseja fazer a requisição.
- `options` é um objeto opcional que pode conter configurações adicionais para a requisição, como método HTTP, cabeçalhos, corpo da requisição, etc.

Dentro da primeira função de callback `.then()`, você manipula a resposta da requisição. O objeto Response possui vários métodos para acessar diferentes aspectos da resposta, como o método `json()` para ler e analisar o corpo da resposta como JSON, `text()` para ler o corpo da resposta como texto, entre outros.

Vamos a um exemplo de uso comum, onde usamos o fetch para fazer uma requisição do tipo GET (ou seja, "baixar" dados), e processá-los como arquivos JSON.

JavaScript

```
fetch('https://api.example.com/data')
  .then(response => {
    if (!response.ok) {
      throw new Error('Erro ao fazer a requisição: ' +
response.status);
    }
    return response.json(); // Retorna uma Promise que será resolvida
com os dados JSON da resposta
  })
  .then(data => {
    // Manipula os dados da resposta
    console.log(data);
  })
  .catch(error => {
    // Manipula erros de requisição
    console.error('Erro na requisição:', error);
  });
```

Este exemplo faz uma requisição GET para 'https://api.example.com/data', verifica se a resposta foi bem-sucedida (status 200-299) e, em seguida, converte os dados da resposta para JSON. Finalmente, os dados JSON são manipulados dentro do segundo `.then()`.

(UFMT/POLITEC MT/2022) A função fetch() do JavaScript serve para

a) percorrer cursores de consultas de banco de dados.
b) acessar e manipular partes do pipeline HTTP.
c) realizar a conversão de respostas para array de objetos.
d) filtrar dados da memória de acordo com os parâmetros.
e) realizar requisições XMLHttpRequest e extensões do HTTP.

Comentários:

Segundo a definição trazida pelo Mozilla, a API Fetch fornece uma interface JavaScript para acessar e manipular partes do pipeline HTTP. A pipeline HTTP é um mecanismo para enviar várias requisições de forma sequencial em uma única conexão TCP, sem precisar esperar pela resposta de cada requisição antes de enviar a próxima. Isso pode melhorar a eficiência e o desempenho das comunicações HTTP, reduzindo a latência e aproveitando melhor a largura de banda disponível.. (Gabarito: Letra B)

Axios

Axios é uma biblioteca JavaScript popular, baseada em Promises, que simplifica o processo de fazer requisições HTTP tanto em navegadores quanto em ambientes Node.js. Ela fornece uma interface limpa e fácil de usar para realizar solicitações AJAX de forma assíncrona, incluindo suporte para interceptores de requisição e resposta, cancelamento de requisições, autenticação, e manipulação de erros. Axios é amplamente adotado na comunidade de desenvolvimento web devido à sua simplicidade, flexibilidade e recursos abrangentes para comunicação com APIs e serviços da web.

> AJAX, ou Asynchronous JavaScript and XML, é uma abordagem de desenvolvimento web que permite atualizar partes específicas de uma página web sem precisar recarregar a página inteira. Isso é alcançado fazendo requisições assíncronas ao servidor web usando JavaScript e manipulando os dados recebidos sem interromper a interação do usuário. Embora o nome inclua "XML", os dados podem ser trocados em vários formatos, como JSON, XML ou simplesmente texto.

Um código genérico para a implementação do Axios segue abaixo:

JavaScript

```
axios({
    method: 'GET', // Método HTTP da requisição
    url: 'https://api.example.com/resource', // URL da requisição
    params: { // Parâmetros da requisição (opcional)
      param1: 'value1',
      param2: 'value2'
    },
    headers: { // Cabeçalhos da requisição (opcional)
      'Content-Type': 'application/json',
      'Authorization': 'Bearer token'
    },
    data: { // Corpo da requisição (opcional, para métodos que
aceitam corpo como POST, PUT, etc.)
      key: 'value'
    }
  }).then(response => {
    // Manipula a resposta da requisição
    console.log(response.data);
  }).catch(error => {
    // Manipula erros de requisição
    console.error('Erro na requisição:', error);});
```

Deixando claro que esse código é apenas um exemplo, não há necessidade de você entendê-lo de forma completa, já que as questões não chegam nesse nível de cobrança.

QUESTÕES COMENTADAS

01. (CEBRASPE/CAU BR/2024) A respeito de Javascript, julgue o próximo item.

Suponha-se que um código precise tratar uma string que apresente o valor 10.5 como um número inteiro. Nessa hipótese, para suportar essa operação, deve-se usar a função parseInt.

Comentários:

Questão interessante. O primeiro ponto de destaque é perceber que estamos tratando de uma string, um texto, e não do número 10.5. Se estivéssemos abordando números, usaríamos math.floor ou math.ceiling para arredondar... mas como faço com textos?

Usamos o *parse*. O *parse* irá interpretar e converter o bloco de texto em um número. Como queremos um número inteiro, usaremos o parseInt. Portanto, está correta a afirmativa da questão.

Gabarito: Correto

---

02. (CEBRASPE/TST/2024)

```
<html>
  <body>
     <p id="demo"></p>
        <script>

        let x = 10;{
         let x = 1;
         x = x - 1;
        }
        {
         let x = 3;
         x = x + 5;
        }
        {
         let x = 20;
         x = x + 2;
        }

document.getElementById("demo").innerHTML = x;
</script>
</body>
</html>
```

Considerando o código precedente, desenvolvido em ECMAScript 6, assinale a opção que apresenta o resultado mostrado ao usuário ao final da execução desse código.

a) 0

02648216103 - HENRIQUE AMORIM BARRETO

b) 8
c) 10
d) 12
e) 22

Comentários:

Para resolvermos essa questão, você precisa lembrar que o *let* é uma variável de escopo local. Então, o *let x = 10*, definido fora dos colchetes, prevalecerá ao longo de todo o código. Se estivéssemos analisando a variável dentro do contexto de cada colchete, ou seja, no contexto local, aí sim essas operações existiram - mas como não é o caso, elas não afetam o output do código, já que todos possuem um contexto fechado.

Dessa forma, podemos garantir que x = 10.

Gabarito: Letra C

---

03. (FGV/CGE PB/2024) Considere o seguinte código em ECMAScript 2021:

```
const num = 1_2_3_4_5+1_0;
console.log(num)
```

Ao ser executado, o código acima exibe o seguinte texto no console:

a) NaN;
b) 12355;
c) undefined;
d) 1_2_3_4_6_0;
e) 1_2_3_4_5+1_0.

Comentários:

Essa questão trouxe algo bem "diferente". Para melhorar a legibilidade, podemos separar os elementos de um número por *underscore*, ou sublinhado. Então, escrever 1_2_3 seria o mesmo que escrever 123. Sabendo disso, a questão passa a ficar tranquila.

1_2_3_4_5 é igual a 12.345; 1_0 é igual a 10; somando os dois, temos o resultado do código - 12.355.

Gabarito: Letra B

---

04. (CEBRASPE/FINEP/2024)

```
var frutas = ["maçã", "laranja", "pera", "banana", "kiwi", "abacaxi", "manga", "uva"];
var posicao = frutas.indexOf("banana");
```


02648216103 - HENRIQUE AMORIM BARRETO

Considerando o código precedente, escrito em JavaScript, assinale a opção em que é corretamente indicado o valor da variável posicao após a execução do código.

a) -1
b) -2
c) 2
d) 3
e) 4

Comentários:

Questão nos cobra, basicamente, a posição (índice) do elemento 'banana'. Lembrem-se que o índice no JavaScript começa em 0. Então temos:

Maçã - 0
Laranja - 1
Pera - 2
Banana - 3

Portanto, o valor da varável posicao é 3.

Gabarito: Letra D

---

05. (FGV/SEF MG/2023) Assinale a opção que indica o tipo de vulnerabilidade que é geralmente associada à função javascript eval()

a) Cross-origin resource sharing (CORS)
b) Cross-site request forgery (CSRF)
c) Cross-site scripting (XSS)
d) Server-side request forgery (SSRF)
e) Cross Site History Manipulation (XSHM)

Comentários:

Excelente questão, que faz um paralelo entre programação e segurança da informação. A função eval() é usada para avaliar ou executar código JavaScript representado como uma string. Ela permite a execução dinâmica de código, o que pode ser útil em certos casos, como na avaliação de expressões matemáticas inseridas pelo usuário ou na criação dinâmica de funções. No entanto, a capacidade de executar código arbitrário também representa um risco significativo de segurança, especialmente quando o conteúdo da string passada para eval() é proveniente de fontes não confiáveis, como entrada do usuário ou dados externos.

Nesse sentido, surge a vulnerabilidade chamada de **Cross-site scripting (XSS)**, que envolve a execução de *scripts* maliciosos, injetados e executados nos navegadores, a partir de dados de entrada. Explico melhor: quando preenchemos algum campo de registro, ou qualquer formulário que seja, essas entradas são processadas pelo JavaScript - e, se não houver sanitização da entrada, pode ser que uma execução de *eval* rode algum código malicioso e prejudique a aplicação.

02648216103 - HENRIQUE AMORIM BARRETO

Gabarito: Letra C

06. (VUNESP/TCM SP/2023) Analise o programa a seguir, elaborado na Linguagem HTML com JavaScript:

```
<!DOCTYPE html>
<html>
<body>
<p id=”conta”></p>
<script>

var a = 1;
var b = 2;
var c = 3;

x = a + b * c;

y = x % 5;

z = Math.pow (y,2);

z--;

document.getElementById(“conta”).innerHTML = z;
</script>
</body>
</html>
```

Quando esse programa for aberto por um navegador como o Chrome ou o Edge, será exibido na tela:

a) ERROR, pois uma das operações aritméticas resultará em um valor inválido.
b) 1.
c) 2.
d) 3.
e) 4.

Comentários:

Temos a definição de 3 variáveis: a = 1; b = 2; c = 3. Além disso, temos 3 outras variáveis, que envolvem operações matemáticas. Cada operação matemática depende da precedente, então precisamos fazer todas para chegar à resposta. Vamos lá.

1) x = a + b * c
   x = 1 + (2 * 3)
   x = 7

2) y = x%5 (o resto da divisão de X por 5)
y = 7%5
y = 2

3) z = Math.pow (y, 2) → equivale a y^2
$z = 2^2$
z = 4

Por fim, temos uma operação de decréscimo (z--) antes de inserirmos o valor, que significa que iremos retirar uma unidade do valor de Z. Sendo assim, o valor inserido será igual a 3 (4 -1).

Gabarito: Letra D

---

07. (VUNESP/TCM SP/2023) Considere o seguinte código HTML 5 com JavaScript:

```
<!DOCTYPE html>
<html>
<body>
<p id=”texto”></p>
<script>

document.getElementById(“texto”).innerHTML = (~(5^2))<<2;

</script>
</body>
</html>
```

O texto na saída apresentado no navegador, ao carregar essa página, é:

a) -12.5
b) -20
c) -32
d) -50
e) -100

Comentários:

Questão que cobra conhecimentos acerta de operações *bitwise*. A equação que será inserida no campo é a seguinte:

(~(5^2))<<2

Devemos fazê-la por partes. O primeiro passo é 5^2, ou seja, estamos fazendo uma comparação **XOR** do número 5 (101 em binários) e 2 (010 em binários). Lembrem-se, as comparações são feitas *bit a bit*.

- 1 xor 0 = 1

- 0 xor 1 = 1
- 0 xor 1 = 1

Vamos ter **111 em binários, ou 7 na base decimal**. Agora, vamos fazer a negação (~) - seguindo a nossa fórmula. Chamarei o valor da negação de X.

- X = -(n + 1)
- X = -(7 + 1)
- X = -8

Por fim, temos a operação ***bitwise* << 2**. Vamos deslocar o -8 em duas casas à esquerda, preenchendo os espaços com 0s. Para isso, precisamos representá-lo em binários. Isso é feito convertendo o número para complemento de 2, da seguinte forma:

- Pegamos o número em módulo e invertemos os bits.
  - No nosso caso, 8 em bits é 1000
  - A inversão ficaria 0111. Isso é chamado de complemento de um.
- Adicionamos 1 ao número
  - 0111 equivale a 7. Se somarmos 1, temos 8 - 1000 em bits.
  - Cuidado que foi pura coincidência da representação ser a mesma em bits do módulo, e do valor negativo.

Agora, podemos fazer o *bitwise*.

- 1111...1000 << 2
- 1111...100**00**

O número 1111...100000 representa -32 no complemento de dois. Para descobrir seu valor, é só fazer o caminho oposto:

- Removemos 1 do número
  - 100000 equivale a 32. Removendo 1, temos 31.
  - 31 em binários é 011111 - valor que equivale ao complemento de 1 de 32.
- Invertemos o número binário.
  - Invertendo 011111 temos 100000 - que equivale a 32.
  - Podemos dizer, portanto, que 1111...1110 0000 é igual a 32 negativo.

Novamente, cuidado que a representação "igual" do valor positivo e negativo em binário foi pura coincidência, por estarmos trabalhando com valores de bis cheios (1, 2, 4, 8 etc.).

Gabarito: Letra C

---

08. (FCC/TRT 18/2023) Um técnico deseja manipular precisamente números inteiros em uma aplicação JavaScript com mais de 15 dígitos. Para armazenar um número nessa proporção em uma variável x, ele deve utilizar a instrução

a) let x = Long("4556567456345345322466456789098877666555");

02648216103 - HENRIQUE AMORIM BARRETO

b) let x = 4556567456345345322466456789098877666555n;
c) let x = Long.parseLong("4556567456345345322466456789098877666555");
d) let x = 4556567456345345322466456789098877666555L;
e) let x = toNumber("4556567456345345322466456789098877666555");

Comentários:

Coruja, o JavaScript possui como limite padrão para números 15 dígitos. Para “contornar” esse problema, foi desenvolvido o tipo numérico chamado de BigInt. A sua caracterização é dada por um n ao final do conjunto de números.

Gabarito: Letra B

---

09. (VUNESP/TCM SP/2023) Considere o seguinte código HTML 5 com JavaScript:

```
<!DOCTYPE html>
<html>
<body>

<canvas id="c" width="200" height="100">
</canvas>

<script>
var c = document.getElementById("c");
var ctx = c.getContext("2d");
ctx.beginPath();
ctx.arc(100,50,40,Math.PI,2*Math.PI);
ctx.stroke();
</script>
</body>
</html>
```

O desenho resultante no canvas, exibido no navegador, é

a) uma circunferência completa.
b) uma semicircunferência na metade superior do canvas.
c) uma semicircunferência na metade inferior do canvas.
d) um arco correspondente a ¼ de circunferência, na metade superior do canvas.
e) um arco correspondente a ¼ de circunferência, na metade inferior do canvas.

Comentários:

O elemento canvas é uma área retangular em uma página HTML onde é possível desenhar gráficos, animações, gráficos vetoriais e outras renderizações gráficas usando JavaScript. Ele fornece uma superfície de desenho bidimensional por meio de APIs de desenho 2D ou 3D, permitindo que os desenvolvedores criem

02648216103 - HENRIQUE AMORIM BARRETO

e manipulem gráficos dinâmicos diretamente no navegador. Com isso em mente, vamos analisar o código fornecido:

- Primeiro, o código HTML cria um elemento canvas com a identificação "c" e define sua largura como 200 pixels e sua altura como 100 pixels.
- Em seguida, o JavaScript obtém o contexto de renderização 2D do canvas usando getContext("2d") e armazena em uma variável ctx.
- Depois, o código desenha um arco usando ctx.arc(). Os parâmetros passados para ctx.arc() são:
  - x: A coordenada x do centro do arco (100 pixels).
  - y: A coordenada y do centro do arco (50 pixels).
  - raio: O raio do arco (40 pixels).
  - startAngle: O ângulo inicial do arco (Math.PI, que é π, ou seja, 180 graus).
  - endAngle: O ângulo final do arco (2*Math.PI, que é 2π, ou seja, 360 graus).
- Por fim, o arco é desenhado no canvas usando ctx.stroke(), que traça o contorno do arco.

Para identificar onde o arco é desenhado no canvas, devemos entender os parâmetros passados para ctx.arc(). O centro do arco é definido como (100, 50), o que coloca o centro do arco na metade da largura e na metade da altura do canvas. Além disso, o ângulo inicial é π (180 graus) e o ângulo final é 2π (360 graus), o que resulta em um arco completo. No *canvas*, começamos em 0 graus ao lado direito, e desenhemos em sentido horário - então, 180-360 quer dizer que começamos ao lado esquerdo, e traçamos um arco até o lado direito.

Com isso, temos o arco desenhado na metade superior do canvas.

Gabarito: Letra B

---

10. (VUNESP/TCM SP/2023) Elaborou-se um programa na Linguagem HTML com JavaScript, apresentado a seguir (os traços nas linhas 4 e 10 representam partes do programa que foram omitidas):

```
1 <!DOCTYPE html>
2 <html>
3 <body>
4 <button _____="mostra()">Mostrar
6 dia e hora</button>
7 <script>
8 function mostra() {
9   document.getElementById("teste").
10 innerHTML = _____;
11 }
12 </script>
13 <p id="teste"></p>
14 </body>
```

Supõe-se que esse programa será aberto por um navegador como o Chrome ou o Edge. Deseja-se que ele apresente na tela um botão que, ao ser pressionado, exiba a data e hora no formato:

Sun Mar 01 2020 23:20:11 GMT-0300 (Horário Padrão de Brasília)

Para tanto, os traços deverão ser substituídos, respectivamente, por:

a) onchange e Date()
b) onchange e DayHour()
c) onclick e Date()
d) onclick e Day()Hour()
e) onclick e DayHour()

Comentários:
Para que tenhamos o desejado, ao clicar no botão e exibir o alerta, devemos gerenciar um **evento**. Esse evento, de clique, é chamado de **onclick.** Para exibir as informações da data no momento atual, basta inserir a função nominada **Date()**. Então, para que o código rode de acordo com o pedido, precisamos preencher os traços por onclick e Date().

Gabarito: Letra C

---

11. (VUNESP/PREF. MARÍLIA/2023) Um dos tipos de operadores da linguagem JavaScript é o operador de deslocamento. Considere uma variável x inicialmente com o valor 10 (em decimal). Após se realizar a operação

x = x << 2

seu novo valor será

a) 2
b) 5
c) 10
d) 20
e) 40

Comentários:

Questão que cobra conhecimentos da operação de *bitwise left-shift*. Iremos deslocar o X em 2 bits, preenchendo com 0 os bits vazios criados à direita. Primeiro passo é converter o 10 em binários - 1010. Segundo passo, é deslocar 2 bits para a esquerda, preenchendo com 0s na direita - 101000. Então, teremos 40, na base decimal.

Gabarito: Letra E

---

12. (VUNESP/PREF. MARÍLIA/2023) O código JavaScript

window.alert(2 + 3 + "5");

quando executado exibe uma caixa de diálogo com o seguinte conteúdo:

a) 55

02648216103 - HENRIQUE AMORIM BARRETO

b) 10
c) 0
d) 5
e) 235

Comentários:

Temos diferentes interações nessa sintaxe. O primeiro trecho, 2 + 3, irá ser uma operação matemática de soma - resultante em 5 -, já que estamos tratando de números. Então, ficaremos com (5 + “5”).

Agora, porém, ao fazermos uma soma de número com string, não iremos mais trabalhar em operações matemáticas - e sim realizar uma **concatenação**. Ela irá juntar os dois números em um bloco de texto só, sendo tratado como *string*. O retorno será 55, a junção dos dois blocos de texto.

Gabarito: Letra A

---

13. (FCC/MPE PB/2023) Para mudar, com JavaScript, o conteúdo de um elemento <h1 id= "title"> para "MPEPB123", utiliza-se o comando

a) document.getElementById("title").text = "MPEPB123";
b) document.innerHTML("h1#title") = "MPEPB123";
c) document.getElementById("#title").innerHTML = "MPEPB123";
d) document.text("#title") = "MPEPB123";
e) document.getElementById("title").innerHTML = "MPEPB123";

Comentários:

Estamos tratando de um elemento com a tag <h1> e id = title. Podemos, portanto, usando o getElement, selecionar esse elemento no DOM de duas diferentes formas - uma pela tag, e outra pelo ID.

- Pela tag: document.getElementByTagName(“h1”)
- Pelo id: document.getElementByID(“title”)

Como queremos alterar o conteúdo interno do bloco, com o texto MPEPB123, usamos o operador **innerHTML**, que irá inserir o conteúdo dentro do elemento do HTML selecionado. Então, podemos ter duas respostas nessa questão:

- Selecionando a tag: document.getElementByTagName(“h1”).innerHTML = “MPEPB123”
- Selecionando o id: document.GetElementById(“title”).innerHTML = “MPEPB123”

A questão optou pela seleção pelo ID - nossa segunda opção acima. A sintaxe é trazida na letra E, que é nosso gabarito.

Gabarito: Letra E

---

14. (VUNESP/UNICAMP/2023) Considere o seguinte trecho de código escrito em Javascript:

02648216103 - HENRIQUE AMORIM BARRETO

```
let str = “Tenha uma boa prova!”;
let x = str.search(“a”);
```

O valor armazenado na variável x será:

a) –1
b) 0
c) 3
d) 4
e) 5

Comentários:

A função *search* funciona de forma muito similar à *indexOf*. Ela procurará o primeiro elemento que é compatível com o definido, e irá retornar o índice dele. Quando estamos tratando de strings, a contagem de índices é feita como se cada letra fosse um elemento de uma *array*, **contando inclusive os espaços vazios**.

Se formos contar a posição da primeira ocorrência do “a”, partindo do 0, veremos que ele está na posição 4.

Gabarito: Letra D

---

15. (VUNESP/UNICAMP/2023) Dois dos operadores utilizados na linguagem Javascript são os operadores de incremento/decremento. Considere o seguinte trecho de código:

```
let x = 10;
x = x++;
x = --x;
```

O valor da variável x após a execução do trecho de código será:

a) 8
b) 9
c) 10
d) 11
e) 12

Comentários:

Questão que aborda conhecimento dos **incrementos** e **decrementos**. Eles podem ocorrer de duas formas, quando estamos tratando de variáveis:

- x++ ou x-- → chamado de **pós-incremento** ou **pós-decremento.** Nesses casos, **não teremos alteração no valor**, já que a adição ou remoção é feita após o elemento.
- ++x ou --x → chamado de **pré-incremento** ou **pré-decremento**. Aqui já teremos uma alteração do valor, pois a adição ou remoção é chamada antes do elemento em si.

02648216103 - HENRIQUE AMORIM BARRETO

Na questão, temos os seguintes casos:

- x = x++; não acarreta alteração, pois temos um pós-incremento;
- x = --x; irá retirar uma unidade de x, já que é um pré-decremento.

Sendo assim, apenas a segunda operação alterará nosso valor, retirando 1 unidade do valor inicial (10), resultando em 9.

Gabarito: Letra B

---

16. (VUNESP/UNICAMP/2023) A linguagem Javascript, ao contrário de outras linguagens, possui apenas uma definição para qualquer tipo de número, utilizando para tal um conjunto de bits para sua representação. O número de bits utilizado no Javascript para representar um número qualquer é:

a) 16
b) 32
c) 64
d) 128
e) 256

Comentários:

Informação não trivial, mas, se cair novamente, você estará blindado. O JavaScript trabalha com o padrão de **64 bits** para números. Para trabalhar com números maiores que isso, usamos o padrão de números Bigint.

Gabarito: Letra C

---

17. (VUNESP/UNICAMP/2023) Considere o seguinte trecho de código escrito na linguagem Javascript.

```
var x = 1;
for(var i = 0; i < 3; i++){
  x = x << 1;}
```

É correto afirmar que o valor da variável x após a execução do trecho de código apresentado será:

a) 1
b) 2
c) 3
d) 8
e) 16

Comentários:

Questão bem complexa - envolvendo loops e operações bitwise de deslocamento. Teremos a execução de 3 laços (i = 0; i = 1; i = 2), já que começamos com i = 0, temos como limite superior i < 3 e incrementos unitários (i++) a cada laço.

PRIMEIRO LAÇO:

- Temos x = 1 ou, em binários, 0001. Vamos deslocar 1 bit:
- 0001 << 1 = 0001**0**. O número binário equivale a 2 - mas não converta, já que trabalharemos as mesmas operações nos próximos laços.

SEGUNDO LAÇO:

- 0010 << 1 = 010**0**

TERCEIRO LAÇO:

- 0100 << 1 = 0100**0**

Então, ao fim de 3 laços, temos 01000 - ou 8 em base decimal.

Gabarito: Letra D

---

18. (VUNESP/EPC/2023) Considere a seguinte linha de código escrita na linguagem Javascript.

let x = 5 * 2 >> 1 + 2;

É correto afirmar que o valor da variável x após essa execução será:

a) 0
b) 1
c) 5
d) 7
e) 8

Comentários:

No JS, operadores matemáticos tem precedência aos operadores de bits. Então, vamos resolver a parte aritmética antes.

- 5 * 2 = 10
- 1 + 2 = 3

Então, temos 10 >> 3 - indica que iremos deslocar o número 10 (1010 em binário) em 3 bits para a direta - ou seja, eliminaremos os três bits menos significativos (da direita).

1**010** >> 3 = 0001

Teremos 0001 como resposta - ou, em base decimal, 1.

Gabarito: Letra B

---

19. (FGV/DPE RS/2023) Atualmente sob responsabilidade da ECMA International, o Java Script (ECMA 6) utiliza o padrão de arrow function para a definição de funções, aproximando um pouco mais a linguagem do paradigma funcional.

Considerando uma função para a multiplicação de dois valores, na forma recursiva, sua implementação no padrão arrow function é:

a) let fnx = (a,b) => (b>1) ? a * fnx(a, b-1) : a;
b) let fnx = (a,b) => (b>1) ? b + fnx(a, b-1) : a;
c) let fnx = (a,b) => (a>1) ? b * fnx(a-1, b) : a;
d) let fnx = (a,b) => (b>1) ? a + fnx(a, b-1) : a;
e) let fnx = (a,b) => (a>1) ? b * fnx(a, b) : a-1;

Comentários:

Temos uma combinação de arrow function com condicionais ternárias

variavelQualquer = (parametro) => condição ? resultado se verdadeiro : resultado se falso

A questão quer que você ache a alternativa que traz uma multiplicação de valores em forma recursiva, isso é, uma função que chama a ela mesma. A cada iteração, vams fazer um decremento de A ou B (em fnx(a-1,b) ou fnx(a, b-1)), até chegarmos ao "fundo do poço" das recursões.

Vamos supor 2 números quaisquer - a = 3, b = 2 - para explorarmos as respostas e encontrar aquela que retorna uma multiplicação entre os valores - no caso, 3*2 = 6.

*a) let fnx = (a,b) => (b>1) ? a * fnx(a, b-1) : a;*

Para o par a, b = (3, 2), começamos com a condição b>1 sendo satisfeita, então executaremos o primeiro bloco:

a*fnx(a, b-1) = 3*fnx(3, 1).

Ainda não chegamos a um número, precisamos pular ao próximo laço - com o par (a, b) = (3, 1). Aqui, 3 > 1 é falso, e executaremos o segundo bloco - que simplesmente retorna o valor de a (3). Então, fnx(3,1) = 3.

Subindo a recursão, teremos 3*fnx(3,1) = 3*3 = 9. Como queremos o resultado 6, essa alternativa está **errada**.

b) let fnx = (a, b) => (b>1) ? b + fnx(a, b-1) : a;

Novamente, temos (a, b) = (3, 2), obedecendo à condição b > 1. O primeiro laço fiará:

b + fnx(3, 1);

02648216103 - HENRIQUE AMORIM BARRETO

Descemos a recursão para o próximo laço - com o par (a, b) = (3, 1). Novamente, quebramos a condição, e o retorno é o valor de a (3). Agora, podemos subir somando:

b + fnx(3, 1)
2 + 3 = 5

Não encontramos o resultado esperado - aqui acontece uma simples soma entre os valores. **Errado**.

*c) let fnx = (a,b) => (a>1) ? b * fnx (a-1, b) : a*

Aqui a condição é para o elemento a. Vamos ter mais um laço, portanto. Veja:
2*fnx(2, 2)
2*fnx(1,2)

Como fnx(1, 2) é igual ao valor de a (3), podemos voltar subindo.

2*fnx(1,2) = 2*3 = 6
2*fnx(2,2) = 2*6 = 12

Ainda não encontramos o valor que queremos. **Errado**.

d) let fnx = (a,b) => (b>1) ? a + fnx(a, b-1) : a

Novamente, de forma mais direta.

a + fnx(3, 1)

Como fnx(3,1) cai na condição *else*, teremos como retorno o valor de a. Podemos subir somando.
a + fnx(3,1) = 3 + 3 = 6

E finalmente encontramos nosso gabarito, que retorna o valor pretendido e faz multiplicações de forma recursiva. **Correto**.

*e) let fnx = (a,b) => (a>1) ? b*fnx(a, b) : a-1*

Aqui não temos nenhum decremento acontecendo no bloco *if*, e, graças a isso, ficaremos executando-o eternamente. Veja:

b*fnx(3, 2)
b*fnx(3,2)
...
b*fnx(3,2)

Portanto, por termos um loop infinito, nunca cairmos na condição do *else*, essa afirmativa está **errada**.

Gabarito: Letra D

20. (CEBRASPE/PREF. FORTALEZA/2023) Julgue o item subsequente, a respeito de HTML5, CSS3 e JavaScript.

O código JavaScript a seguir, ao ser executado, apresenta o valor 3 no console de desenvolvimento do navegador web.

```
<script>
     var cores = [“vermelho”, “verde”, “azul”];
     console.log(cores.length);
</script>
```

Comentários:

Iremos printar no console o comprimento da *array*. Como temos 3 elementos, de fato o resultado no console será 3.

Gabarito: Correto

---

21. (VUNESP/DPE SP/2023) Um programador deseja fazer uma busca em uma String utilizando Expressões Regulares na linguagem Javascript. Dada uma String texto, ele deseja verificar a posição da String em que são encontrados três dígitos, seguidos de um espaço em branco (whitespace), seguidos de pelo menos um caractere a. Para tal, ele pode escrever a linha de código

a) let indice = texto.search(/3\d\sa*/);
b) let indice = texto.search(/000\sa+/);
c) let indice = texto.search(/000_a*/);
d) let indice = texto.search(/\d\d\d_a*/);
e) let indice = texto.search(/\d\d\d\sa+/);

Comentários:

Questão que cobra conhecimentos acerca da formatação do *search*, de forma a temos uma varredura melhor sobre uma string e retornarmos o que queremos. Para isso, são usados **meta caracteres**. Veja uma tabela que traz todos:

Meta Caractere	Descrição
. (ponto)	Qualquer caractere, exceto nova linha e terminador de linha
\w e \W	Palavra e não-palavra
\d e \D	Dígito e não dígito
\s e \S	Espaço em branco e não-branco
\b e \B	No começo/fim da palavra e não-começo/fim da palavra
\0	Caractere NULL
\n	Caractere de nova linha
\f	Caractere do tipo form feed
\r	Caractere de retorno
\t	Caractere que representa *tab*
\v	Caractere que representa *tab* vertical
<caractere>+	Indica que podemos ter mais elementos de um caractere

A questão quer que encontrar 3 dígitos - espaço em branco - pelo menos um caractere *a*. Vamos montar a sintaxe para encontrarmos a resposta, encarando cada exigência de uma vez.

- A sintaxe deve começar e terminar com uma barra comum /
- 3 dígitos - \d \d \d
- 1 espaço em branco - \s
- Pelo menos um caractere a - a+

Juntando tudo: texto.search(/\d\d\d\sa+/

Gabarito: Letra E

---

22. (FGV/ALEMA/2023) Analise o código JavaScript a seguir.

```
function enigma(n) {
  if (n == 0 || n == 1) {
    return n
} else {
    return enigma (n-1) + enigma (n-2)
}}
```

Assinale o valor retornado para a expressão enigma(4).

a) 21
b) 13
c) 8
d) 5
e) 3

Comentários:

A questão nos traz uma função recursiva, utilizando condicionais. Se n = 0 **ou** n = 1, executaremos o bloco *if*. Caso contrário, executaremos o *else*. Como começamos com n = 4, vamos começar no bloco do *else*. A sua sintaxe é enigma (n-1) + enigma (n-2) - irei fazer os laços para você visualizar, lembrando que começamos com n = 4 e, quando n = 1 ou n = 0, retornaremos o próprio valor do n.

- Laço 1 - enigma(4) = enigma(3) + enigma(2)
- Laço 2 - enigma(3) = enigma(2) + enigma(1)
- Laço 3 - enigma(2) = enigma(1) + enigma(0)
- Laço 4 - enigma(1) = 1
- Laço 5 - enigma(0) = 0

Agora que chegamos ao “fundo”, podemos voltar subindo.

- enigma(2) = enigma(1) + enigma(0) = 1 + 0 = 1
- enigma(3) = enigma(2) + enigma(1) = 1 + 1 = 2

- enigma(4) = enigma(3) + enigma(2) = 2 + 1 = 3

Assim, nossa resposta será 3.

Gabarito: Letra E

---

23. (FGV/ALEMA/2023) Os trechos JavaScript a seguir apresentados funcionam corretamente, à exceção de um. Assinale-o.

a)
```
bomDia = function() {
return "Bom dia!";
}
alert(bomDia());
```
b)
```
bomDia = () => {
return "Bom dia!";
}
alert(bomDia());
```
c)
```
x = "!";
bomDia = (val) => "Bom dia" + val;
alert(bomDia(x));
```
d)
```
bomDia => "Bom dia!";
alert(bomDia);
```
e)
```
bomDia = (a) => a + "!";
alert(bomDia("Bom dia"));
```

Comentários:

A questão quer que encontremos a alternativa incorreta. Com exceção da letra D, todas terão como retorno "Bom dia!". Na D, temos um erro na construção da arrow function, faltou definir o espaço de execução da função com parênteses ( ). O correto, com as correções em vermelho destacado, seria:

```
bomDia = ( ) => "Bom dia!";
alert(bomDia ( ) );
```

Gabarito: Letra D

---

24. (FGV/ALEMA/2023) Analise o código JavaScript a seguir.

```
var x = document.getElementById("xpto");
while (x.hasChildNodes()) {
x.removeChild(x.childNodes[0]);
}
```

O efeito da execução desse trecho é

02648216103 - HENRIQUE AMORIM BARRETO

a) a remoção de todos os elementos subordinados ao elemento cujo id é “xpto”.
b) um erro de execução, pois “xtpo” é usado equivocadamente onde deveria haver uma referência numérica.
c) um erro de execução, pois o índice 0 não se aplica no HTML.
d) m erro de sintaxe, pois a declaração da variável “x” deveria explicitar o tipo element.
e) um erro de sintaxe, pois o método removeChild é inválido.

Comentários:

Questão interessante, mistura o loop while com manipulação do DOM.O que está acontecendo aqui é que o while executa uma verificação, procurando se o elemento de ID = xpto possui nodes filhos. Enquanto tiver, ele irá remover o primeiro node da fila. Esse código irá se repetir até a condição while deixar de ser verdade – ou seja, não termos mais nodes filhos, já que teremos eliminado todos. Portanto, a afirmativa que traz um apontamento correto é a letra A.

Gabarito: Letra A

---

25. (CEBRASPE/DATAPREV/2023) Julgue o item abaixo, relacionados com JavaScript, Web Services e análise estatística de código-fonte.

Para garantir uma correta compilação de um código escrito em JavaScript, é necessário que as variáveis sejam definidas e inicializadas antes de seu uso no código.

Comentários:

Coruja, se trabalhássemos só com as variáveis const e let, essa afirmativa estaria correta, pois elas não apresentam hoisting (a capacidade de serem lidas antes de inicializadas). Porém, temos a variável do tipo var, que apresenta a característica de hoisting, tornando a afirmativa incorreta.

Gabarito: Errado

---

26. CEBRASPE/DATAPREV/2023) Considerando a linguagem de programação JavaScript, julgue o item a seguir.

Caso o código a seguir seja executado, o console.log() mostrará que a variável inicio tem valor igual a 1.

```
const inicio = 0;
const fim = 10;
if (inicio >= fim) {
inicio = fim;
} else {
        inicio++;
}
console.log(inicio);
```

Comentários:

Essa questão não foi elaborada para que você resolva o loop – o problema está antes dele. Temos duas variáveis declaradas como const e, dentro do loop, estamos fazendo uma manipulação da variável const inicio. O JavaScript não permite manipulação de variáveis do tipo const fora dela. Caso as variáveis estivessem declaradas com let, teríamos como resposta, de fato, o valor 1.

Gabarito: Errado

---

27. (FGV/MPE SC/2023) No contexto do JavaScript, analise um trecho do HTML de uma página web.

```
<div id="xpto">
        <p>Texto de teste 1</p>
        <p id="234">Texto de teste 2</p>
</div>
```

A linha de código que altera o tamanho da fonte do segundo parágrafo acima é:

a) document.getElementById("234").style.fontSize="48px";
b) document.getElementById("xpto.234").style.fontSize="48px";
c) document.getElementById(p<"234">).style.fontSize="48px";
d) document.getElementsById("div.p").style.fontSize="48px";
e) xpto.getElementById("234").style.fontSize="48px";

Comentários:

Vamos lá, você já sabe o esquema para resolver essas questões. O primeiro passo é definir como selecionar o elemento – e, vendo nas alternativas, está sendo usado o getElementById. Portanto, já que o segundo parágrafo tem como ID “234”, usamos get.ElementById(“234”). Só com isso, já acharíamos a resposta. Mas seguiremos.

Queremos alterar o tamanho da fonte, que é um recurso fornecido pelo CSS – portanto, usaremos a diretiva style. A fonte é alterada através do fontSize. Dessa forma, nossa sintaxe fica:
document.getElementById(“234”).style.fontSize=”48px”

Gabarito: Letra A

---

28. (FGV/TCE SP/2023) Leandro precisa escrever uma mensagem dinâmica em uma div, através do Java Script, e sabe que deve ser utilizado DOM, ou Document Object Model, para acessar os elementos de sua página.

Considerando que o id utilizado na div é "msgOut", e que a variável msg contém a mensagem, Leandro irá utilizar o comando:

02648216103 - HENRIQUE AMORIM BARRETO

a) document["msgOut"].innerHTML=msg;
b) document.getElementById("msgOut").innerHTML=msg;
c) document.setHtmlContent("msgOut",msg);
d) document["msgOut"].html(msg);
e) document.setElementById("msgOut",msg);

Comentários:

Queremos selecionar uma div, com o ID “msgOut”. Através do getElement, podemos usar:

document.getElementById(“msgOut”)

Com isso, já teríamos o gabarito. Porém, queremos adicionar uma variável chamada msg dentro do elemento. Para isso, usaríamos innerHTML=msg.

Gabarito: Letra B

---

29. (CEBRASPE/DPE RO/2023)

```
int a = 3;
int b = 7;
int c;
c = ++a + (--a * b++);
```

A execução do código apresentado, escrito em JavaScript, resulta na variável c com o valor

a) 14.
b) 32.
c) 25.
d) 24.
e) 16.

Comentários:

Lembre do que vimos lá no começo da resolução de exercícios - quando temos um pós-incremento ou pós-decremento, por termos chamado a variável antes de executar a soma ou diminuição, não iremos alterar o valor em nada.

Então, c será igual a:

c = (a + 1) + [(a - 1) * b]
c = (3 + 1) + [(4 - 1) * 7]
c = 4 + [3 * 7]
c = 4 + 21 = 25

Gabarito: Letra C

---

30. (FGV/SEFAZ AM/2022) Arrays são estruturas de dados que armazenam uma coleção de elementos.

A maneira apropriada para criar um array de elementos em JavaScript é

a) const meuarray = new Array ["maçã", "laranja", "2"]
b) const meuarray = {"maçã", "laranja", "2"}
c) const meuarray = ["maçã", "laranja", 2]
d) const meuarray = ("maçã", "laranja", 2)
e) const meuarray := [2]string{"maçã", "laranja"}

Comentários:

Vou trazer o lembrete que fiz na aula:

NÃO CONFUNDA:
ARRAYS → COLCHETES [ ];
OBJETOS → CHAVES { };

Então, para declarar arrays, fazemos:

```
meuArray = [“valor 1”, “valor 2”, 3, 4,...]
```

Alternativamente, podemos construir as arrays a partir de funções construtoras:

```
meuArray = new Array (“valor 1”, “valor 2”, 3, 4, ...)
```

A única alternativa que constrói uma array corretamente é a letra C.

Gabarito: Letra C

---

31. (VUNESP/UFABC/2023) Considere a seguinte linha de código escrita na linguagem Javascript:

```
let x = 10 >> 1
```

É correto afirmar que o valor da variável x, após a execução dessa linha, será:

a) true
b) false
c) 5
d) 10
e) 20

Comentários:

02648216103 - HENRIQUE AMORIM BARRETO

Outra questão de *bitwise*. Apesar de não ser usado quase nunca na programação, as bancas amam. Vamos lá:

1010 >> 1 = 101

Então, temos 101, ou 5 em base decimal.

Gabarito: Letra C

---

32. (CESGRANRIO/TRANSPETRO/2023) Um desenvolvedor web está trabalhando em uma aplicação que envolve a coleta de dados numéricos dos usuários, e, para isso, está usando HTML e JavaScript (ECMAScript 2020) para criar um campo de entrada de texto e um botão. Quando um usuário clicar nesse botão, o valor inserido será processado. Nesse cenário, considere que esse desenvolvedor quer obter o valor atual do campo de texto HTML com o ID idade. Para obter esse valor, ele deve utilizar o seguinte fragmento de código JavaScript:

a) document.getElementById(“idade”).getText()
b) document.getElementById(“idade”).innerHTML
c) document.getElementById(“idade”).value
d) document.getElementByName(“idade”).value
e) document.querySelector(“#idade”).text

Comentários:

Considerando o ID idade, podemos selecionar o campo de duas formas diferentes:

- document.getElementById(“idade”)
- document.querySelector (“#idade”)

Para obtermos o valor do elemento usamos a diretiva ***value*.** Portanto, nosso código ficará:

- document.getElementById(“idade”).value
- document.querySelector(“#idade”).value

A alternativa que corretamente traz essa sintaxe é letra B.

Gabarito: Letra B

---

33. (FGV/TCE SP/2023) Leandro precisa escrever uma mensagem dinâmica em uma div, através do Java Script, e sabe que deve ser utilizado DOM, ou Document Object Model, para acessar os elementos de sua página.

Considerando que o id utilizado na div é "msgOut", e que a variável msg contém a mensagem, Leandro irá utilizar o comando:

a) document["msgOut"].innerHTML=msg;
b) document.getElementById("msgOut").innerHTML=msg;

c) document.setHtmlContent("msgOut",msg);
d) document["msgOut"].html(msg);
e) document.setElementById("msgOut",msg);

Comentários:

Questão tranquila - para selecionarmos o elemento de id = msgOut, usando o JavaScript, usamos:

- document.getElementById(“msgOut”)
- document.querySelector(“#msgOut”)

Para inserirmos a variável msg dentro do elemento, basta usarmos a diretiva innerHTML=msg, ficando com o código document.getElementById(“msgOut”).innerHTML=msg.

Gabarito: Letra B

---

34. (FGV/TCE SP/2023) Jorge é consultor em uma empresa de desenvolvimento e está analisando o código a seguir, na linguagem JavaScript, encontrado em uma das páginas do site.

```
const va = [1,2,5,7,12,14,19,20];
const vb = [], vc = [];

for(let a of va)
        if(a%2==0)
        vb.push(a);
   else
        vc.push(a);

console.log(vb.reduce((a,b)=>a+b));
console.log(vc.reduce((a,b)=>a+b));
```

Jorge abriu a página e viu impressos no console de depuração, respectivamente, os valores:

a) 1 e 20;
b) 1 e 79;
c) 15 e 65;
d) 48 e 32;
e) 80 e 0.

Comentários:

Temos 3 arrays, a `va`, com o conjunto apresentado, e as arrays `vb` e `vc`, vazias. Após a definição das arrays, começamos um loop `for...of`. Ele funciona da seguinte forma:

- Se o elemento de a for divisível por 2, ele irá ser inserido na array b
- Caso contrário, será inserido na array c

02648216103 - HENRIQUE AMORIM BARRETO

Então, de forma geral, faremos uma atribuição dos valores pares para b, e ímpares para c. As arrays ficarão dessa forma:

- b = [2, 12, 14, 20]
- c = [1, 5, 7, 19]

Por fim, estamos fazendo uma redução (reduce) de cada uma dessas arrays - isso é, estamos somando todos os elementos delas - e printando no console. A sintaxe (a+b) => a+b implica que iremos somar os valores de dois em dois, atribuir esse valor da soma ao acumulador e continuar somando até percorrermos toda a *array*. Veja o resultado:

- vb.reduce = 2 + 12 + 14 + 20 = 48
- vc.reduce = 1 + 5 + 7 + 19 = 32

Correta a letra D, portanto.

Gabarito: Letra D

---

35. (FGV/TCE SP/2023) Beatriz tem um site com diversos elementos do tipo div, onde foi utilizada uma classe CSS com o nome "opcional", e deseja que eles fiquem ocultos quando forem clicados. Para adicionar a funcionalidade em todos os elementos via JavaScript, ela desenvolveu a função apresentada a seguir.

```
const associar = (elementos) => {
for(let obj of elementos)
obj.addEventListener('click',(e)=>{
e.target.style.display='none'});
}
```

Para executar a função criada, associando corretamente aos elementos do tipo div, Beatriz deve utilizar para o parâmetro elementos a expressão:

a) document.getElementsByTagName("div.opcional");
b) document.getElementsByClassName("opcional");
c) document.getElementsByName("div:opcional");
d) document.getElementsByClass("opcional");
e) document.getElementsById("div.opcional").

Comentários:

Apesar do comando enorme da questão, ela quer saber apenas como iremos selecionar a div através da classe de nome “opcional”. Podemos selecionar essa classe, especificamente, através do comando document.getElementsByClassName(“opcional”).

Gabarito: Letra B

---

36. (FGV/CÂMARA DOS DEPUTADOS/2023) Assinale o método JavaScript que retorna o número de caracteres em uma string.

a) charAt()
b) count
c) len()
d) length
e) size

Comentários:

Questão super tranquila. Para contarmos a quantidade de caracteres, podemos usar a função `length`. Cada caractere, mesmo que vazio, será contado.

Gabarito: Letra D

---

37. (FGV/CÂMARA DOS DEPUTADOS/2023) Considere o script JavaScript a seguir.

```
function f(f) { return (f*f); }

let value1 = f;
alert(value1);

let value2 = f(100);
alert(value2);

let value3 = f;
alert(value3(100));
```

Assinale os valores exibidos na execução do script acima, na ordem.

a) function f(f) { return (f*f); }
   10000
   10000
b) function f(f) { return (f*f); }
   10000
   undefined
c) undefined
   10000
   undefined
d) 10000
e) undefined
   undefined
   undefined

Comentários:

A função f é definida como f*f - isso é, f multiplicado por f, ou f^2. No **primeiro alerta**, como não estamos chamando a função (não temos o par de parênteses), iremos retornar apenas a definição da função - ou seja, retornará function f(f) {return (f*f)}.

No **segundo alerta**, estamos chamando a função na própria variável *value2*, com o valor de 100. O resultado será 100*100 = 10.000.

Já no terceiro alerta, a função é atribuída sem ser chamada, mas ela é chamada dentro do alerta, também com o valor 100. Portanto, o resultado será 100*100 = 10.000

Gabarito: Letra A

---

38. (FGV/SEDUC SP/2023) Considere o trecho a seguir quando utilizado numa página Web.

<body onload="alert('xxxxxxx')">

O efeito desse trecho é

a) a exibição automática da mensagem xxxxxxx imediatamente antes que a página seja carregada.
b) a exibição automática da mensagem xxxxxxx imediatamente antes que um formulário seja carregado.
c) a exibição automática da mensagem xxxxxxx imediatamente depois que a página foi carregada.
d) o início imediato de uma operação de upload.
e) um erro, pois o script contém incompatibilidades.

Comentários:

O evento **onload** é um evento que roda um determinado bloco de código assim que o elemento for carregado. No nosso caso, como o evento está na tag <body>, que compreende toda a página da *web*, teremos o alerta, com a mensagem xxxxxxxx, aparecendo imediatamente após carregarmos a página.

Gabarito: Letra C

---

39. (FGV/TJ SE/2023) Considere o script JavaScript a seguir.

```
let x = 16;
let y = [];

y[2] = x;
y[2] %= 3;
y[0] = x<<2;
y[1] = x <<= 5;

alert(y);
```

A execução desse script exibe:

02648216103 - HENRIQUE AMORIM BARRETO

a) 0,128,1
b) 2,512,32
c) 64,true,1
d) 64,512,1
e) 128,64,2

Comentários:

Temos 2 variáveis, o x = 16 e o y sendo uma array vazia, e temos 4 diferentes operações de atribuição de valores à array. Vamos fazer as operações e ver o que acontece.

- y[2] = X
  - Irá atribuir o valor de X (16) à posição 2.
  - Resultado - y = [_, _, 16] *(os _ representam espaços vazios)*
- y[2] %= 3
  - Irá pegar o valor da posição 2(16), e atribuir o valor do resto da divisão por 3, que é 1
  - Resultado - y = [_, _, 1]
- y[0] = x << 2
  - Atribuiremos o valor da operação bitwise 16 << 2 ao campo 0
  - 16 em binários é 10000 - deslocando duas casas para o lado, temos 10000**00**, que equivale a 64
  - Resultado - y = [64, _, 1]
- y[1] = x <<= 5
  - Faremos um deslocamento a esquerda em 5 bits de x (16), e atribuiremos o resultado ao campo 1, o segundo elemento da array
  - 10000 << 5 = 10000**00000**, que equivale a 512.
  - Resultado - y = [64, 512, 1]

Então, como resultado do alerta, teremos 64, 512 ,1.

Gabarito: Letra C

---

40. (FGV/TJ SE/2023) Analise o código JavaScript a seguir.

```
const valores = [4, 3, 2, 1, 0];
let xpto = "";
valores.forEach(fF);
alert (xpto);
function fF(value, index) {
  xpto += index + ":" + value + ">";}
```

A execução desse código exibe:

a) 4:4>
b) 4:0>3:1>2:2>1:3>0:4>
c) 0:4>1:3>2:2>3:1>4:0>
d) 0:0>

e) 0:0>1:1>2:2>3:3>4:4>

Comentários:

Temos uma array chamada de 'valores', igual a [4, 3, 2, 1, 0], e uma variável xpto vazia. Passamos o **forEach(fF)** pela array 'valores', isso é, para cada elemento da array, iremos aplicar a função fF. Essa função irá passar o valor de xpto para cada campo da array - valor esse que é composto do índice do element, concatenado com o carectere ":", o seu valor, e o caractere ">". Então, teremos:

- Índice 0, valor = 4; resultado: "0:4>"
- Índice 1, valor = 3; resultado: "1:3>"
- Índice 2, valor = 2; resultado: "2:2>"
- Índice 3, valor = 1; resultado: "3:1>"
- Índice 4, valor = 0; resultado: "4:0>"

Ao fim, o a *array* será composta pelos valore acima citados.

Gabarito: Letra C

---

41. (FGV/TJ SE/2023) Observe o seguinte trecho de código-fonte em JavaScript:

```
const array = ["Um", "Dois", "Tres"];
array.unshift("Tres");
array.splice(-1);
```

O valor final do array é:

a) ["Um"];
b) ["Dois"];
c) ["Um", "Dois"];
d) ["Um", "Tres", "Tres"];
e) ["Tres", "Um", "Dois"].

Comentários:

Temos duas operações tratando uma array. A primeira delas é um `unshift`, que é responsável por adicionar um elemento no começo da array. Então, o comando `array.unshift("Tres")` transforma nossa array em ["Tres", "Um", "Dois", "Tres"].

Por fim, o splice remove ou adiciona elementos em posições definidas. Seu comando é `splice(x, y, z)`, onde `x` indica o índice que queremos manipular, `y` o número de elementos que queremos remover, e `z` os elementos que queremos inserir. No nosso caso, a sintaxe `splice(-1)` indica que estamos tratando do índice -1, ou seja, o último elemento - e a ausência de demais parâmetro indica que queremos remover esse número. Então, o resultado dessa operação seria ["Tres", "Um", "dois"].

Gabarito: Letra E

---

42. (FGV/MPE GO/2022) Assinale o trecho de código JavaScript que define e ordena corretamente um array.

a) const fruits = ["Casa"; "Apartamento"; "Sítio"]; fruits.sort();
b) const fruits = ["Casa", "Apartamento", "Sítio"]; fruits.sort();
c) const fruits = {"Casa"; "Apartamento"; "Sítio"}; fruits.sort();
d) const fruits = ["Casa", "Apartamento", "Sítio"]; array.sort(fruits);
e) const fruits = ["Casa", "Apartamento", "Sítio"]; array.sort(fruits);

Comentários:

Primeiro, precisamos declarar corretamente uma array usando chaves, e separando os elementos por vírgulas. Dessa forma, eliminamos as letras A, e C. O segundo ponto, o comando para ordenar as arrays é `nomeDaArray.sort()`. Dessa forma, podemos concluir que a afirmativa com o apontamento correto é a letra B.

Gabarito: Letra B

---

43. (FGV/TJDFT/2022) Na página web que Maria está desenvolvendo, há um botão que, quando clicado, exibe data e hora correntes.

O código que implementa essa característica corretamente é:

a) <button id="xpto" onclick= get("xpto").innerHTML=Date()">...</button>
b) <button onclick="button.innerHTML=Date()">...</button>
c) <button onclick="this.innerHTML=Date()">...</button>
d) <button onclick=exhibit Date()>...</button>
e) <button onclick="function() return Date()">...</button>

Comentários:

Quando clicamos em um botão e isso altera seu conteúdo, temos um evento `onclick`. Até então, todas as afirmativas apresentam a função. A segunda parte envolve a alteração do conteúdo do botão para a data e hora correntes – para isso, precisamos usar um `innerHTML` e passar uma função chamada de Date(), que retorna a hora no momento em que é ativada. Sendo assim, a alternativa correta é a letra C.

Gabarito: Letra C

---

44. (FGV/MPE SC/2022) Analise o código JavaScript exibido a seguir.

```
var numeros = [4,9,16];
var xpto = numeros.map(function(num) {
return num << 2;
});
```

Após a execução desse trecho de código, os valores de numeros e xpto são, respectivamente:

a) undefined

[16,36,64]
b) [4,9,16]
[16,36,64]
c) null
[0,1,4]
d) [4,9,16]
Null
e) [16,36,64]
Undefined

Comentários:

Questão que, num primeiro momento, pode assustar – já que envolve uma operação bitwise e um map ao mesmo tempo. Porém, vamos resolver pelo caminho curto. A variável numeros não será alterada, já que o map cria uma array nova. Dessa forma, ja eliminamos as letras A, C e E.

Agora, a variável xpto será o valor de cada elemento de X, com um left-shift de duas casas. Isso retorna um conjunto de números – fazendo com que eliminemos a letra E. Vou fazer cada operação bitwise para que você não esqueça como se faz:

```
4 << 2
```

- 4 em binários = 0100
- Deslocando 2 casas para a esquerda, e preenchendo com os bits significativos, temos:
- 010000 = 16

```
9 << 2
```

- 9 em binários = 1001
- Deslocando 2 casas para a esquerda:
- 100100 = 36

```
16 << 2
```

- 16 em binários = 10000
- Deslocando 2 casas para a esquerda:
- 1000000 = 64

Então a variável XPTO será [16, 36, 54].

Gabarito: Letra B

---

45. (FGV/SEMSA MANAUS/2022) No âmbito de uma página Web, analise o trecho de código JavaScript a seguir.

```
const points = [40, 100, 1, 5, 25, 10];
points.sort(function(a, b){return a - b});
alert(points);
```

A execução do código acima deverá provocar

a) um erro de compilação na segunda linha.
b) um erro de execução na terceira linha.
c) a exibição dos valores de points na ordem em que foram declarados.
d) a exibição dos valores de points na ordem numérica crescente.
e) a exibição dos valores de points na ordem numérica decrescente.

Comentários:

Aluno, nós já vimos essa função na aula. Ela é responsável por fazer a ordenação em ordem crescente da nossa array. Ela recebe como parâmetro dois valores – a e b, que correspondem aos valores dos elementos. É feita uma operação de subtração entre cada um, e, caso o resultado seja negativo, a função sort entende que a deve ficar à frente de b; caso seja positivo, a função sort entende que devemos inverter a ordem dos elementos.

Por exemplo, na primeira iteração, teríamos a = 40, b = 100. O resultado a – b seria negativo, -60, passando para a função sort que o elemento a deve permanecer à frente do elemento b. Isso é feito para todas os pares de números, até que não tenhamos mais resultados positivos.

Gabarito: Letra D

---

46. (FGV/SEMSA MANAUS/2022) (FGV/SEMSA MANAUS/2022) No JavaScript, a expressão typeof {nome:'Maria', cargo:'Professora'} retorna

a) array.
b) function.
c) null.
d) object.
e) undefined.

Comentários:

O comando typeof, de forma simples, lê o tipo de dado que está sendo passado. No caso da questão, o par de colchetes { } caracteriza um objeto - então, o retorno será `object`.

Gabarito: Letra D

---

47. (CESGRANRIO/BASA/2022) O tratamento de erros e exceções é, atualmente, uma exigência para as linguagens de programação.

Qual fragmento de código, em Javascript, é um exemplo válido desse tratamento?

02648216103 - HENRIQUE AMORIM BARRETO

```
a) try {
     adddlert("Bem vindo!");
    } else(err) {
    window.alert(err.message);}
b) try {
        adddlert("Bem vindo!");
     } except(err) {
       window.alert(err.message); }
c) try {
       adddlert("Bem vindo!");
    } catch(err) {
     window.alert(err.message);}
d) try {
       adddlert("Bem vindo!");
   } catch(err){
     window.alert(err.message);
     } else {
     window.alert("sem erros");}
e) try {
       adddlert("Bem vindo!");
    }  catch(err) {
     window.alert(err.message);
    } otherwise {
       window.alert("sem erros");}
```

Comentários:

Para tratamento de erros em JavaScript, usamos o conjunto de blocos de código try/catch e, opcionalmente, o finally. Temos a seguinte estrutura, em geral:

```
try {
   bloco a ser testado
} catch {
  bloco de código se houver erro
} finally {
  bloco executado independentemente de erro
}
```

A única alternativa que traz uma construção correta da sintaxe é a letra C.

Gabarito: Letra C

---

48. (CEBRASPE/PETROBRAS/2022) Julgue o próximo item que trata de CSS, JavaScript e Net Core.

Em JavaScript é possível simular cada uma das quatro categorias de membros de classe da linguagem Java.

Comentários:

Correto, aluno. Existem quatro categorias de membros de classe no Java:

- Membros de instância: São membros específicos de cada objeto. Em JavaScript, isso é alcançado definindo propriedades e métodos dentro do construtor de uma classe ou dentro de uma classe em ES6.
- Membros de classe (ou membros estáticos): São membros compartilhados por todas as instâncias de uma classe. Em JavaScript, esses membros são definidos usando a palavra-chave static na frente da definição de propriedades ou métodos dentro da classe.
- Membros de inicialização (ou blocos de inicialização): Em Java, esses membros são usados para inicializar campos estáticos. Em JavaScript, isso é frequentemente alcançado usando métodos estáticos de inicialização ou atribuindo valores diretamente a propriedades estáticas.
- Membros construtores: São usados para inicializar instâncias de uma classe em Java. Em JavaScript, isso é feito através do método constructor dentro da definição da classe.

A afirmativa é, portanto, correta.

Gabarito: Correto

---

49. (FGV/MPE GO/2022) No contexto do JavaScript, considere as seguintes afirmativas sobre a declaração de variáveis com let ou var.

I. Variáveis declaradas com let não podem ser redeclaradas no mesmo { } bloco.
II. Variáveis declaradas com let podem ser utilizadas em qualquer trecho do código (escopo global).
III. Variáveis declaradas com var no interior de um { } bloco podem ser utilizadas fora do bloco de origem.

Está correto o que se afirma apenas em

a) I.
b) II.
c) I e II.
d) I e III.
e) II e III.

Comentários:

Vamos analisar cada afirmativa.

I. Certo. No JavaScript, ao contrário de var, quando você declara uma variável com let dentro de um mesmo bloco ({ }), não é possível redeclará-la. Tentar redeclarar uma variável com let dentro do mesmo bloco resultará em um erro.

II. Errado. Variáveis declaradas com let têm escopo de bloco, o que significa que elas só podem ser acessadas dentro do bloco em que foram declaradas ou em blocos internos a ele. Elas não têm escopo global.

III. Certo. Variáveis declaradas com var têm escopo de função ou global, não de bloco. Isso significa que, quando você declara uma variável com var dentro de um bloco, ela pode ser acessada fora desse bloco, desde que não seja dentro de uma função diferente.

Corretos, portanto, I e III.

Gabarito: Letra D

---

50. (VUNESP/ALESP/2022) Analise o programa a seguir, que possui um script elaborado em JavaScript. Os pontos assinalados com (1), (2) e (3) e (4) correspondem a informações suprimidas do programa.

```
<!DOCTYPE html>
<html><body>
<script language=JavaScript>
<!--
        horaatual = (1);
        hora = horaatual.(2);
        minutos = horaatual.(3);
        segundos = horaatual.(4);
        document.writeln(" Hora atual: " + hora + ":" +
        minutos + ":" + segundos);
</script>
</body></html>
```

Quando o programa for aberto por um navegador que suporte JavaScript, como o Chrome ou o Edge, é exibida a hora do computador, por exemplo:

Hora atual: 17:43:42

Para que estas exibições ocorram conforme apresentado, (1), (2), (3) e (4) deverão ser, respectivamente:

a) new Hour(), getHours(), getMinutes() e getSeconds().
b) new Date(), Hours(), Minutes() e Seconds().
c) new Hour(), Hours(), Minutes() e Seconds().
d) new Date(), getHours(), getMinutes() e getSeconds().
e) Hour(), getHours(), getMinutes() e getSeconds().

Comentários:

Para acessarmos as informações acerca da hora, data e afins, precisamos usar a função Date( ) - ela criará um objeto com as informações de dia, mês, hora etc. Então, para o campo (1), precisamos primeiramente usá-la - o que já elimina as alternativas A, C e E.

02648216103 - HENRIQUE AMORIM BARRETO

Agora, com o objeto Date definido, podemos navegar pelas suas propriedades através das diretivas `get*` - como `getHours`, `getMinutes` e `getSeconds`. Sendo assim, a alternativa que corretamente traz o preenchimento das lacunas é a letra D.

Gabarito: Letra D

---

51. (CEBRASPE/FUB/2022)

```
<html>
    <p id=“saida”></p>
    <head>
        <script type=“text/javascript” >
              const user = {
              nome: 'Pedro Maria',
              email: 'pedro.maria@gmail.com',
              idade: 25,
              nascimento: '21/02/1996',
              masculino: true
        };
        document.getElementById(“saida”).
              innerHTML = ““;
        for (const key in user) {
             document.getElementById(“saida”).
             InnerHTML += `$ {key}:
             $ {user[key]}`;
        }
        </script>
    </head>
</html>
```

Considerando o código precedente, em HTML, julgue o item a seguir.

O código em questão apresenta um erro de sintaxe, porque atribui valores de tipos diferentes a uma mesma variável.

Comentários:

Aluno, a questão está cobrando conhecimentos acerca da construção de objetos. No caso, temos o objeto `user`, com diversos atributos. No JS, cada atributo tem seu tipo, autonomamente dos demais. Por esse motivo, a alternativa está errada - podemos ter tipos diferentes dentro da mesma variável, se ela for um objeto ou uma array.

Gabarito: Errado

---

52. (FGV/TJDFT/2022) No contexto do JavaScript, analise as afirmativas a seguir a respeito do comportamento do JS de mover declarações para o top (em inglês, Hoisting).

I. Variáveis declaradas por meio de var podem ser utilizadas antes de declaradas.
II. Variáveis declaradas com let e const são movidas para o topo do bloco somente quando inicializadas na declaração.
III. Usar uma constante (const) antes de declarada gera um erro de sintaxe.

Está correto o que se afirma em:

a) somente I;
b) somente I e II;
c) somente I e III;
d) somente II e III;
e) I, II e III.

Comentários:

Vamos analisar cada alternativa.

I. Certo. As variáveis var possuem *hoisting* e, por esse motivo, podem ser utilizadas antes mesmo de declaradas.
II. Errado. As variáveis let e const são movidas ao topo sim - mas elas não são inicializadas. Somente serão inicializadas quando forem declaradas.
III. Certo. A variável *const* não possui *hoisting*, então seu uso antes da declaração irá gerar erros.

Corretos os itens I e III.

Gabarito: Letra C

---

53. (FGV/MPE SC/2022) No JavaScript, as três funções nativas que permitem uma rápida comunicação com o usuário de uma página web são:

a) alert, confirm, input;
b) alert, input, output;
c) exhibit, read, write;
d) input, msgbox, print;
e) read, ring, output.

Comentários:

Das alternativas, a única que traz funções do JavaScript é a letra A. As demais trazem alguma alternativa que não possui correspondência com funções nativas. O **alert** é responsável por gear um pop-up com determinada mensagem; o **confirm** traz uma janela de confirmação ou cancelamento para uma opção; o **input** é um campo que permite inserirmos valores em determinado formulário. Nesse sentido, correta a letra A.

Gabarito: Letra A

54. (FGV/TJ TO/2022) Considere o código JavaScript a seguir.

```
var x1
var x2
var x3
var y = 11
var z = 12
x1= z++ == ++y
x2= 10 + "casa"
x3= y+1 === z && 10 != "20"
```

Após a execução dessas operações, os valores de x1, x2 e x3 são, respectivamente:

a) false
10casa
true
b) false
10casa
false
c) false
undefined
false
d) true
10casa
true
e) true
undefined
false

Comentários:

Vamos lá, analisar cada valor em x1, x2 e x3.

x1 = z++ == ++y

Vejam que temos um pós incremento, que não impacta a variável, e um pré incremento, que impactará. Então z++ = 12; ++y = 11+1 = 12. Nesse sentido, a comparação será 12 == 12, que retornará *true*.

x2 = 10 + “casa”

Estamos fazendo uma concatenação de número e texto - o resultado será o texto “10casa”.

x3 = y+1 === z && 10 != “20”

02648216103 - HENRIQUE AMORIM BARRETO

Temos duas comparações de cada lado, que são, posteriormente, comparadas novamente. Podemos escrever como (y+1 === z) && (10 != “20”). Vamos fazer uma de cada vez.

y+1 === z

Comparação estrita entre y+1 (11+1=12) e z (12). Como ambos tem valores e tipos de dados iguais, o retorno será *true*.

10 != “20”

Comparação de diferença entre string e número. Como a comparação não é estrita, precisamos ver só os valores - que são, de fato, diferentes, retornando *true*.

true && *true*

Como ambos os lados são verdadeiro, a comparação AND (E lógico) entre os lados retornará *true* também.

Sendo assim, retornaremos *true*, 10casa e *true*.

Gabarito: Letra D

---

55. (FGV/TJ TO/2022)

```
function xpto(x) {
  return x % 2 == 0;
}
const numeros = [22, 0, 18, 1];
alert (numeros.every(xpto));
```

O valor exibido na execução desse código é:

a) 1
b) 3
c) false
d) true
e) undefined

Comentários:

A função **every**, usada no alerta, é responsável por verificar se todos os elementos de uma array atendem a uma determinada condição. No nosso caso em análise, a condição é definida pela função xpto.

A função xpto é responsável por pegar cada número da array, e fazer o módulo desse número por 2, isso é, o resto da divisão. Se essa divisão tiver resto 0, teremos true; caso contrário, teremos false. Estamos verificando, basicamente, se cada um dos elementos da array é par.

02648216103 - HENRIQUE AMORIM BARRETO

Mas veja, essa comparação é feita a **todos** os elementos, e só retornará *true* se **todos** forem pares. Como não é o caso, temos o número 1, que é ímpar (0 é tratado como par no JS), o retorno será *false*.

Gabarito: Letra C

---

56. (FGV/TJ TO/2022) No contexto da linguagem JavaScript, analise o trecho a seguir.

```
hello = (val) => "Hello " + val;
```

Essa definição é equivalente ao trecho:

a)
```
function hello(val) {
        return 'Hello ' + val;
   }
```

b)
```
((val == ' ') ? 'Hello ' + val : val)
```

c)
```
function hello(val) {
         if (val != '') {
            return 'Hello ' + val }
        else { return '' }
  }
```

d)
```
function hello(val) {
        return val;
   }
```

e)
```
((val != ' ') ? val : '')
```

Comentários:

Questão que cobra conhecimentos acerca das *arrow functions*. Lembrem da estrutura que trouxe a vocês: função = (parâmetros) => Resultado.
Nesse sentido, podemos traduzir a sintaxe apresentada na questão hello = (val) => "Hello " + val para:

```
function hello(val) {
        return 'Hello ' + val;
   }
```

Portanto, a alternativa correta é a letra A.

Gabarito: Letra C

---

57. (FGV/SEMSA MANAUS/2022) Considere o comando JavaScript a seguir.

```
const teste = new Xpto("a", 50);
```


02648216103 - HENRIQUE AMORIM BARRETO

Assinale a opoção que indica a definição correta para Xpto.

a) const array Xpto[ ];
b) const object Xpto;
c) const Xpto = [ ];
d) function Xpto(p1, p2) {this.x1 = p1;this.x2 = p2};
e) Xpto = {x1:undefined, x2:undefined};

Comentários:

A criação de uma função com a diretiva **new** indica que estamos trabalhando com uma **função construtora**. No comando new `Xpto("a", 50);`, Xpto é a função construtora e `"a"` e `50` são os argumentos passados para a função. As propriedades x1 e x2 do novo objeto são então inicializadas com esses valores. As outras opções (a, b, c, e) não definem Xpto como uma função construtora.

Gabarito: Letra D

---

58. (VUNESP/CAMPINAS/2022) Um programador da linguagem JavaScript utilizou em seu programa, os seguintes comandos:

```
...
if (true == "true")
...
if (true === "true")
...
```

Em relação a esses comandos, é correto afirmar que

a) ambos são equivalentes, resultando no mesmo resultado.
b) o primeiro comando, ao ser executado, retornará o valor false.
c) o segundo comando, ao ser executado, retornará o valor false.
d) o segundo comando não será executado, originando um erro.
e) o segundo comando significa “não idêntico a”.

Comentários:

Temos duas comparações, uma delas é a comparação padrão, que verifica somente a compatibilidade entre valores, representada pelo ==, e outra compara valores e tipo de dado, representada pelo ===. Ness sentido, os valores de true e “true” são os mesmos, o que retornaria *true* na primeira comparação. Porém, na segunda comparação, temos tipos de dados diferentes - true é um valor *boolean*, e “true” é uma string”. Nesse caso, teremos como retorno o valor *false.*

Gabarito: Letra C

---

59. (FCC/TRT 23/2022) Considere a página web com o código JavaScript abaixo.

```
<!DOCTYPE html>
<html>
  <body>
      <script>
        let organization = "TRIBUNAL REGIONAL DO TRABALHO";
        for (...I...) {
         document.write(x + "<br>");
        }
      </script>
   </body>
 </html>
```

Para exibir `TRIBUNAL REGIONAL DO TRABALHO` na vertical, com uma letra por linha, a lacuna I deve ser preenchida corretamente com

a) let x = organization.get(char)
b) let x of organization
c) let i=0; i<organization.length; x = organization.text[i]; i++
d) let x in organization
e) get x in organization

Comentários:

Quando queremos iterar sobre cada elemento de uma variável, podemos usar a forma específica do `for` - a `for...of`. Na questão em análise, estamos aplicando o for...of em uma string e, aqui, cada caractere é analisado isoladamente pelo comando. Então, pegaremos o caractere e aplicaremos uma função específica a ele. Para elencar as letras na vertical, devemos retornar o próprio caractere, seguido de uma quebra de parágrafo (`<br>`). O código ficará assim:

```
for (let x of organization) {
  return x + “<br>”
}
```

Como estamos inserindo essas letras no body do site, podemos substituir o `return` por `document. write (x + “<br>”)`. Teremos como gabarito, portanto, a letra B.

Gabarito: Letra B

---

60. (VUNESP/PREF. SOROCABA/2022) Em um programa elaborado em JavaScript, incluiu-se a seguinte estrutura condicional, na qual X e Y representam comandos removidos:

```
switch (semaforo) {
        case "verde":
                alert("siga")
                X
        case "amarelo":
```

```
            alert("atencao")
            X
        case "vermelho":
            alert("pare")
            X
        Y:
            alert("cor invalida")
}
```

Os comandos X e Y que devem ser colocados para que a estrutura condicional seja executada adequadamente são, respectivamente,

a) break e default.
b) break e end.
c) end e close.
d) stop e close.
e) stop e default.

Comentários:

O `switch` é uma estrutura de condicionais que elenca diversas situações, uma em cada `case`, e atribui um resultado a essa situação. Quando definimos o resultado, temos dois casos:

- O uso de `return`, que não exige nenhum tratamento
- Um valor sem return, caso em que precisaremos empregar a diretiva `break` para "quebrar" a iteração e não seguirmos às outras opções. Se não tivermos essa diretiva, seguiremos para a próxima situação, mesmo ela não sendo equivalente a uma situação implementada.

Por fim, para termos um caso residual, que impedirá nossa sintaxe de entrar em um loop infinito, temos o `default`. Se não tivermos nenhuma situação ocorrendo, implementamos o bloco de código do `default`.

Sendo assim, precisamos substituir I e II por `break` e `default`, respectivamente.

Gabarito: Letra A

---

61. (FGV/TCE TO/2022) No contexto de HTML e JavaScript, na construção

```
function myFunction() {
  document.getElementById("xxxx").submit();
}
```

o elemento xxxx deve identificar:

a) o path completo de um arquivo;
b) um elemento da classe EntryData;
c) um Form object;
d) um link externo;

e) uma página Web.

Comentários:

A função submit() é usada para confirmar e enviar os dados de um formulário. Portanto, no campo “xxxx”, devemos ter um ID associado a objetos de formulário ou, em inglês, *form object*.

Gabarito: Letra C

---

62. (FGV/TCE TO/2022) Analise o script JavaScript a seguir.

```
<script>
.....
let s = "";
x.forEach(f);
alert(s);
function f(value, index, array) {
  s += index + " " + value;
}
</script>
```

Dado que, na execução desse script, o comando alert exibiu os valores

0 451 42 93 164 25

é correto afirmar que o trecho pontilhado foi substituído por:

a) const x = [1, 451, 42, 93, 164, 25];
b) const x = [45, 4, 9, 16, 25];
c) let x = [45, 1, 42, 93, 16, 42, 5];
d) var x = [451, 42, 93, 164, 25];
e) var x = [451, 4293, 93, 425];

Comentários:

A função f, que está sendo aplicada no forEach, traz uma operação de soma entre diferentes elementos. Porém, como estamos envolvendo strings e número, essa soma será uma concatenação dos elementos. Então, cada retorno da função será ÍNDICE VALOR.

02648216103 - HENRIQUE AMORIM BARRETO

Mas essa questão tem uma sacada - como não temos espaço após o valor, o resultado irá encadear os valores. Por exemplo, digamos que no índice 0, temos 3, e no índice 1 temos 4 - ficaríamos com 0 31 4. Com isso, conseguimos identificar os valores da array - irei destacar os índices em **azul**, e os valores em **vermelho**.

0 451 42 93 164 25

Nossa array é, portanto, [45, 4, 9, 16, 25].

Gabarito: Letra B

---

63. (FGV/CM Taubaté/2022) No contexto de paginas Web, analise a linha de código JavaScript a seguir,.

```
<button onclick="this.innerHTML = Date()">The time is?</button>
```

É correto afirmar que, quando o botão é clicado,

a) um campo de input é apresentado, no qual deve ser digitada uma data/hora.
b) o botão é preenchido com a data/hora corrente.
c) é exibida uma mensagem com a data/hora corrente.
d) cada elemento da página com a tag <innerHTML> é preenchido com a data/hora corrente.
e) qualquer elemento HTML da página que esteja selecionado é preenchido com a data/hora corrente.

Comentários:

A sintaxe apresentada aponta um botão com a captura de um evento de clique. Ao clicarmos o conteúdo do botão é alterado - mudando-se o texto “The time is?” para o momento atual, da hora em que clicarmos no botão. Detalhe, não é uma mensagem, é a alteração do conteúdo do próprio botão que é feita. Portanto, correta a letra B.

Gabarito: Letra B

---

64. (FGV/TRT 13/2022) Considere o código Javascript exibido a seguir.

```
<script>
        var x = 10;
        {
        var x = 2;
        }
        alert( x);
</script>
```

A execução desse script provoca

a) a exibição do valor 10.
b) a exibição do valor 12.

02648216103 - HENRIQUE AMORIM BARRETO

c) a exibição do valor 2.
d) um erro de execução.
e) um erro de sintaxe.

Comentários:
Lembre-se, o **var** é uma variável de escopo global - então seu valor pode ser alterado por qualquer bloco dentro do código. Nesse caso, o var x = 2 declarado dentro do bloco altera o valor da variável, tornando-a igual a 2. Se tivéssemos let no lugar de var, isso não ocorreria - o número ficaria o mesmo.

Portanto, ao chamarmos o alerta, será feita a exibição do valor 2.

Gabarito: Letra C

---

65. (FGV/TRT 13/2022) Considere o código Javascript exibido a seguir.

```
<script>
const registro =
{nome:"Carlos", cidade:"São Paulo", idade:42};
let texto = "";
for (let r in registro) {
texto += registro[r] + " ";
}
alert(texto);
</script>
```

A execução desse script provoca

a) a exibição da string “Carlos São Paulo 42”.
b) a exibição da string “Carlos”.
c) a exibição de uma string vazia.
d) um erro de execução.
e) um erro de sintaxe.

Comentários:

Estamos realizando um `for...in` em um objeto. Vamos iterar em cada registro e inserir o resultado da iteração na variável texto. A iteração envolverá, basicamente, inserir o nome de cada atributo, separados por um espaço.

O r, em let r in registro, servirá como um índice. No primeiro laço, por exemplo, teremos `texto += registro[0] + “ “`, que equivale a “Carlos “. Posteriormente, o laço será `registro[1] + “ “`, resultando em “São Paulo “ e, por fim, `registro[2] + “ “`, que é igual a “42 “. Esses três laços serão concatenados na variável texto, resultando em Carlos São Paulo 42.

Gabarito: Letra A

02648216103 - HENRIQUE AMORIM BARRETO

66. (FGV/SEAD AP/2022) Considere uma página Web que contém o código JavaScript a seguir.

```
<script>
let text = "";
for (let i = 0; i < 5; i++) {
  text += " " + i ;
}
alert(text);
</script>
```

O valor exibido pela execução desse script é

a) 0 0 0 0
b) 4
c) 0 4
d) 1 1 1 1 1
e) 0 1 2 3 4

Comentários:

Estamos fazendo um loop for, com i começando em 0, subindo a incrementos de 1 unidade e indo até i = 4 (já que queremos i < 5). A cada laço, iremos inserir um espaço vazio e o valor atual de i - ficando, portanto, com 0 1 2 3 4 (os respectivos números de i a cada laço).

Gabarito: Letra E

---

67. (FCC/TRT 14/2022) Dada a instrução JavaScript const d = new Date();, para se obter e exibir o dia do mês (entre 1 e 31), utiliza-se a instrução

a) document.write(d.getDate().getDay());
b) document.write(d.getDate());
c) document.write(d.getDay());
d) document.println(d.getDate());
e) document.print(d.getDay());

Comentários:

Tendo nosso construtor new Date() constituído dentro da variável d, podemos acessar e obter o dia do mês por duas diretivas diferentes:

- getDay() - que indica um número de 0 a 6, correspondente ao dia da semana
- getDate() - que indica um número de 1 a 31, correspondente ao dia do mês

Como a questão exige que tenhamos o dia do mês, podemos navegar na variável d usando a diretiva getDate() e escrever no documento - ficando da seguinte forma: `document.write(d.getDate())`.

Gabarito: Letra B

---

68. (CEBRASPE/TRT 8/2022)
```
<html>
 <script>
console.log(('a'+ +'b'+'a'+'s').toLowerCase());
 </script>
</html>
```

Assinale a opção que corresponde ao resultado do código JavaScript anterior.

a) abas
b) ananbas
c) ananas
d) anas
e) bananas

Comentários:

Temos um log de terminal envolvendo a concatenação de diversos caracteres. Mas um detalhe: quando temos 'a'+ +'b', estamos fazendo um operador de concatenação (o primeiro +) e tentando somá-lo a um determinado valor b (com o segundo +) - porém como 'b' é uma string e não um valor, o valor retornado será NaN (Not a Number). O resultado seria aNaNa - mas também passamos essa frase para o toLowerCase(), que torna todos os caracteres minúsculos. Portanto, o resultado é 'ananas'.

Gabarito: Letra C

---

69. (FCC/TRT 17/2022) Utilizando JavaScript, em condições ideais, um analista converteu os dados de Paulo Silva, de 50 anos, com renda de 1450.45, para string JSON, no formato abaixo.

```
{"nome":"Paulo Silva","idade":50,"renda":1450.45}
```

Tal conversão pode ser realizada por meio da instrução

a) JSON.toStrings({nome: 'Paulo Silva', idade: 50, renda: 1450.45 })
b) document.toJSON("{nome: 'Paulo Silva', idade: 50, renda: 1450.45 }")
c) JSON.stringify({nome:new String('Paulo Silva'), idade: 50, renda: 1450.45})
d) document.parseToJSon("{nome: 'Paulo Silva', idade: 50, renda: 1450.45 }")
e) JSON.parse({nome:new String('Paulo Silva'), idade: 50, renda: 1450.45 })

Comentários:

Para convertemos um objeto JSON para string, usamos a função JSON.stringify({objeto}). Portanto, a alternativa correta é a letra C.

Gabarito: Letra C

---

70. (VUNESP/CM FERNANDÓPOLIS/2022) Na linguagem Javascript, existem diversos métodos que podem ser utilizados para a obtenção de informação sobre um objeto de data, como o que obtém o tempo, em milissegundos, transcorrido desde 01.01.1970.

Esse método é denominado:

a) get FullTime
b) getDate()
c) getFullYear()
d) getMilliseconds()
e) getTime()

Comentários:

Por mais "estranho" que pareça, temos um método específico que traz a contagem de tempo, em milissegundos, contados desde 01.01.1970 - ela se chama getTime().

Gabarito: Letra E

---

71. (FGV/PC AP/2022) Analise o código JavaScript a seguir.

```
const cidades =
      ["Natal", "Manaus", "Recife", "Brasília"];
cidades.reverse();
alert (cidades);
```

Assinale o que é exibido na execução do trecho acima.

a) Brasília,Manaus,Natal,Recife
b) Brasília,Natal,Manaus,Recife
c) Brasília,Recife,Manaus,Natal
d) Manaus,Recife,Brasília,Natal
e) Recife,Natal,Manaus,Brasília

Comentários:

Temos um array e, sobre ela, é usada a função reverese(). Essa função é responsável por inverter a ordem dos elementos da array - portanto, o resultado será ['Brasília', 'Recife', 'Manaus', 'Natal'].

Gabarito: Letra C

---

72. (VUNESP/PREF. PIRACICABA/2022) A linguagem JavaScript apresenta algumas características, como:

a) a palavra chave var pode ser usada para declarar tanto variáveis locais como variáveis globais.
b) JavaScript não é case-sensitive, de forma que a variável Casa também pode ser escrita como casa ou cASA, ou ainda admite outras variações.
c) JavaScript não é uma linguagem dinamicamente tipada, de forma que se precisa especificar o tipo de dado de uma variável quando declará-la.
d) null é uma palavra-chave que indica valor nulo e pode ser escrita de diversas formas, como Null, NULL, ou ainda outra variação.

02648216103 - HENRIQUE AMORIM BARRETO

e) uma string literal consiste em zero ou mais caracteres dispostos em aspas duplas ("), e aspas simples (') não podem ser utilizadas em uma string, pois acarretará erro.

Comentários:

Vamos analisar cada alternativa.

a) Correto. A palavra-chave var pode ser usada para declarar tanto variáveis locais como variáveis globais em JavaScript. Se var é usada dentro de uma função, a variável é local, ou seja, só pode ser acessada dentro dessa função. Se var é usada fora de uma função, a variável é global.
b) Incorreto. JavaScript é case-sensitive, o que significa que diferencia maiúsculas de minúsculas. Portanto, Casa, casa e cASA seriam consideradas variáveis diferentes.
c) Incorreto. JavaScript é uma linguagem dinamicamente tipada. Isso significa que não é necessário especificar o tipo de dado de uma variável ao declará-la. O tipo de dado de uma variável pode mudar durante a execução do programa.
d) Incorreto. null é uma palavra-chave que indica valor nulo em JavaScript, mas ela é case-sensitive. Portanto, só pode ser escrita como null. Variações como Null, NULL não são válidas e resultarão em um erro.
e) Incorreto. Uma string literal em JavaScript pode consistir em zero ou mais caracteres dispostos em aspas duplas (") ou aspas simples ('). Ambas são aceitas e não causarão erro. Por exemplo, "Hello, World!" e 'Hello, World!' são ambas strings válidas em JavaScript.

Gabarito: Letra A

---

73. (VUNESP/UNICAMP/2022) Na linguagem JavaScript, quando um evento do mouse é acionado, a propriedade do objeto MouseEvent que retorna a coordenada horizontal do ponteiro do mouse relativa ao documento é

a) offsetX
b) pageX
c) screenX
d) shiftKey
e) movementX

Comentários:

Para encontrarmos a coordenada do mouse em um evento que o envolva, usamos a função page. Para coordenadas horizontais, pageX, já para as verticais, pageY.

Gabarito: Letra B

---

74. (VUNESP/UNICAMP/2022) A Linguagem JavaScript permite diversos tipos de eventos (events), como o

a) onBlur: o evento ocorre quando um objeto perde o foco.
b) onChange: o evento ocorre quando um formulário foi enviado.

02648216103 - HENRIQUE AMORIM BARRETO

c) onKeyPress: o evento ocorre quando o usuário pressiona a tecla Enter do teclado.
d) onMouseOut: o evento ocorre quando o ponteiro do mouse é movido sobre um elemento.
e) onSubmit: o evento ocorre quando valor é digitado, seguido da tecla Enter.

Comentários:

Vamos analisar cada alternativa:

a) Correto. onBlur é um evento que ocorre quando um objeto perde o foco. Por exemplo, quando um campo de entrada de texto perde o foco porque o usuário clicou em outro lugar na página.
b) Incorreto. onChange é um evento que ocorre quando o valor de um elemento de formulário é alterado. Por exemplo, quando o usuário digita em um campo de entrada de texto ou seleciona uma opção em um menu suspenso. O evento onSubmit é o que ocorre quando um formulário é enviado.
c) Incorreto. onKeyPress é um evento que ocorre quando o usuário pressiona uma tecla do teclado, não apenas a tecla Enter.
d) Incorreto. onMouseOut é um evento que ocorre quando o ponteiro do mouse é movido para fora de um elemento, não sobre um elemento. O evento onMouseOver é o que ocorre quando o ponteiro do mouse é movido sobre um elemento.
e) Incorreto. onSubmit é um evento que ocorre quando um formulário é enviado, não quando um valor é digitado seguido da tecla Enter. O evento onKeyPress ou onKeyDown poderia ser usado para detectar quando a tecla Enter é pressionada.

Gabarito: Letra B

---

75. (VUNESP/TJM SP/2021) Em um console JavaScript, um usuário digitou a instrução:

typeof 10

Como resultado da execução dessa instrução, foi impresso:

a) "number"
b) "string"
c) "int"
d) "byte"
e) "char"

Comentários:

A função typeof é usada para verificar o tipo de dado de um valor. No caso, 10 é um número e, no JS, retratamos números (inteiros e de ponto flutuante) como *number*.

Gabarito: Letra B

---

76. (VUNESP/TJM SP/2021) Considere o trecho JavaScript a seguir.

var x = 0;

02648216103 - HENRIQUE AMORIM BARRETO

```
for (var i = 10; i < 15; i++) {
x += i;
}
```

Ao término da execução desse código, o conteúdo da variável “x” será:

a) 0
b) 10
c) 15
d) 60
e) 75

Comentários:

Estamos fazendo um loop for, começando com i = 10, indo até i < 15 com incrementos unitários. A cada laço, iremos somar o número atual da variável x com o valor de i - ou seja, teremos:

- 0 + 10 = 10
- 10 + 11 = 21
- 21 + 12 = 33
- 33 + 13 = 46
- 46 + 14 = 60

Portanto, o valor final da variável x será 60.

Gabarito: Letra D

---

77. (FGV/IMBEL/2021) Analise o código JavaScript exibido a seguir.

```
function xpto (n) {
        var text = "";
        var i = 1;
        while (i <= n) {
                text += i + ';';
                i++;
        }
    return text;
}
```

Num comando JS, a expressão

```
xpto(5)
```

retorna

a) 54321
b) 12345

c) 1;2;3;4;5;
d) 5;4;3;2;1;
e) 1;2;3;4;5;4;3;2;1;

Comentários:

Temos uma variável texto, compreendida de uma string vazia, e uma variável i, igual a 1. Enquanto i for igual ou menor ao parâmetro da função (n), iremos executar os laços do while. Cada laço irá adicionar o valor atual de i na variável texto, e adicionar um ponto e vírgula `;` de separação. Em seguida, fazemos um incremento em i.

Chamamos a função com n = 5, então, o primeiro laço será com i = 1:

- Adicionaremos 1 (o valor de i) e o ; na variável text, através de uma concatenação.
- O resultado é text = 1;
- Somamos 1 unidade a i - ficando i = 2.

Para o segundo laço teremos i = 2, para o terceiro i = 3, para o quarto i = 4 e, para o último laço que será executado, teremos i = 5. Nós terminaremos com i = 6 (pois houve um incremento após a realização do último laço), mas não iremos executar o próximo laço já que a condição i <= 5 não é mais cumprida. Portanto, o resultado do loop while é 1;2;3;4;5;.

Gabarito: Letra C

---

78. (FGV/IMBEL/2021) Analise o código JavaScript exibido a seguir.

```
function f1 (n) {
        if (n % 2 == 0) {
        return --n; }
    else {
       return ++n; }
}
```

Em um comando JS, a expressão f1(8) retorna

a) -8
b) -7
c) 7
d) 8
e) 9

Comentários:

A função f1, de parâmetro n, irá retorna um decréscimo do valor de n, se o resto da divisão for 0 (ou seja, se o número for par). Caso contrário, irá retornar um acréscimo ao valor de n. Como ambos são feitos antes de chamarmos a variável, irão impactar o número.

02648216103 - HENRIQUE AMORIM BARRETO

A questão chama a função com o parâmetro 8 que, por ser par, cairá no bloco *if*. O retorno será --n, ou seja, n - 1 = 8 - 1 = 7.

Gabarito: Letra C

---

79. (FGV/TCE AM/2021) Considere o código JavaScript exibido logo abaixo.

```
var x1 = 16;
var x2 = "16";
var x3 = [1,2,3];
```

Usando as definições acima, analise as expressões a seguir.

```
x3[0] == 1
x1 == x2
x1 === x2
true == x1
true == x3[0]
```

Respectivamente, os valores retornados pelas expressões acima são:

a) false, false, false, true, true;
b) false, true, false, false, false;
c) false, true, true, true, false;
d) true, true, false, false, true;
e) true, true, true, false, true.

Comentários:

Vamos analisar cada uma das expressões.

```
x3[0] == 1
```

O valor de x3[0] é 1. Então, como 1 == 1, temos *true*.

```
x1 == x2
```

Estamos fazendo uma comparação de valores entre 16 e “16” - o que é verdade. Teremos *true*.

```
x1 === x2
```

Agora é uma comparação estrita, em que vemos os valores e o tipo de dados. Apesar dos valores 16 e “16” serem iguais, um é de tipo numérico, e outro de tipo textual - portanto, como são diferentes, teremos um retorno *false*.

```
true == x1
```


02648216103 - HENRIQUE AMORIM BARRETO

Item interessante. Na lógica, true equivale a 1, enquanto false equivale a 0. Então, estamos comparando 1 == 16 - o que é falso. Portanto, retornaremos *false*.

true == x3[0]

Agora a comparação é entre 1, o equivalente de true, com 1. No caso, 1 == 1 é verdadeiro, por isso retornaremos *true*.

As operações são, portanto, *true, true, false, false, true*.d

Gabarito: Letra D

---

80. (CEBRASPE/SEED PR/2021) JavaScript trabalha com números usando os operadores aritméticos fornecidos pela própria linguagem. No entanto, a linguagem aceita operações matemáticas mais complexas por meio de um conjunto de funções e constantes definidas como propriedades do objeto Math.

Assinale a opção que apresenta a função que permite realizar, em JavaScript, a operação matemática de arredondar para cima o número 1,17, obtendo-se o valor 2.

a) Math.ceil()
b) Math.exp()
c) Math.pow()
d) Math.round()
e) Math.sqrt()

Comentários:

Vamos relembrar as operações de arredondamento no JS?

- `Math.round(x)` → arredonda para o inteiro mais próximo
- `Math.ceil(x)` → arredonda para cima
- `Math.floor(x)` → arredonda para baixo
- `Math.trunc(x)` → retorna apenas a parte inteira

Como queremos arrendodar 1,17 para 2, ou seja, para cima, usamos Math.ceil().

Gabarito: Letra A

02648216103 - HENRIQUE AMORIM BARRETO

QUESTÕES COMENTADAS

01. (CEBRASPE/CAU BR/2024) A respeito de Javascript, julgue o próximo item.

Suponha-se que um código precise tratar uma string que apresente o valor 10.5 como um número inteiro. Nessa hipótese, para suportar essa operação, deve-se usar a função parseInt.

02. (CEBRASPE/TST/2024)

```
<html>
  <body>
     <p id="demo"></p>
        <script>

        let x = 10;{
         let x = 1;
         x = x - 1;
        }
        {
         let x = 3;
         x = x + 5;
        }
        {
         let x = 20;
         x = x + 2;
        }

document.getElementById("demo").innerHTML = x;
</script>
</body>
</html>
```

Considerando o código precedente, desenvolvido em ECMAScript 6, assinale a opção que apresenta o resultado mostrado ao usuário ao final da execução desse código.

a) 0
b) 8
c) 10
d) 12
e) 22

03. (FGV/CGE PB/2024) Considere o seguinte código em ECMAScript 2021:

02648216103 - HENRIQUE AMORIM BARRETO

```
const num = 1_2_3_4_5+1_0;
console.log(num)
```

Ao ser executado, o código acima exibe o seguinte texto no console:

a) NaN;
b) 12355;
c) undefined;
d) 1_2_3_4_6_0;
e) 1_2_3_4_5+1_0.

04. (CEBRASPE/FINEP/2024)

```
var frutas = ["maçã", "laranja", "pera", "banana", "kiwi", "abacaxi", "manga", "uva"];
var posicao = frutas.indexOf("banana");
```

Considerando o código precedente, escrito em JavaScript, assinale a opção em que é corretamente indicado o valor da variável posicao após a execução do código.

a) -1
b) -2
c) 2
d) 3
e) 4

05. (FGV/SEF MG/2023) Assinale a opção que indica o tipo de vulnerabilidade que é geralmente associada à função javascript eval()

a) Cross-origin resource sharing (CORS)
b) Cross-site request forgery (CSRF)
c) Cross-site scripting (XSS)
d) Server-side request forgery (SSRF)
e) Cross Site History Manipulation (XSHM)

06. (VUNESP/TCM SP/2023) Analise o programa a seguir, elaborado na Linguagem HTML com JavaScript:

```
<!DOCTYPE html>
<html>
<body>
<p id=”conta”></p>
<script>

var a = 1;
var b = 2;
var c = 3;
```

```
x = a + b * c;

y = x % 5;

z = Math.pow (y,2);

z--;

document.getElementById(“conta”).innerHTML = z;
</script>
</body>
</html>
```

Quando esse programa for aberto por um navegador como o Chrome ou o Edge, será exibido na tela:

a) ERROR, pois uma das operações aritméticas resultará em um valor inválido.
b) 1.
c) 2.
d) 3.
e) 4.

07. (VUNESP/TCM SP/2023) Considere o seguinte código HTML 5 com JavaScript:

```
<!DOCTYPE html>
<html>
<body>
<p id=”texto”></p>
<script>

document.getElementById(“texto”).innerHTML = (~(5^2))<<2;

</script>
</body>
</html>
```

O texto na saída apresentado no navegador, ao carregar essa página, é:

a) -12.5
b) -20
c) -32
d) -50
e) -100

08. (FCC/TRT 18/2023) Um técnico deseja manipular precisamente números inteiros em uma aplicação JavaScript com mais de 15 dígitos. Para armazenar um número nessa proporção em uma variável x, ele deve utilizar a instrução

a) let x = Long("4556567456345345322466456789098877666555");
b) let x = 4556567456345345322466456789098877666555n;
c) let x = Long.parseLong("4556567456345345322466456789098877666555");
d) let x = 4556567456345345322466456789098877666555L;
e) let x = toNumber("4556567456345345322466456789098877666555");

Comentários:

Coruja, o JavaScript possui como limite padrão para números 15 dígitos. Para “contornar” esse problema, foi desenvolvido o tipo numérico chamado de BigInt. A sua caracterização é dada por um n ao final do conjunto de números.

Gabarito: Letra B

---

09. (VUNESP/TCM SP/2023) Considere o seguinte código HTML 5 com JavaScript:

```
<!DOCTYPE html>
<html>
<body>

<canvas id="c" width="200" height="100">
</canvas>

<script>
var c = document.getElementById("c");
var ctx = c.getContext("2d");
ctx.beginPath();
ctx.arc(100,50,40,Math.PI,2*Math.PI);
ctx.stroke();
</script>
</body>
</html>
```

O desenho resultante no canvas, exibido no navegador, é

a) uma circunferência completa.
b) uma semicircunferência na metade superior do canvas.
c) uma semicircunferência na metade inferior do canvas.
d) um arco correspondente a ¼ de circunferência, na metade superior do canvas.
e) um arco correspondente a ¼ de circunferência, na metade inferior do canvas.

10. (VUNESP/TCM SP/2023) Elaborou-se um programa na Linguagem HTML com JavaScript, apresentado a seguir (os traços nas linhas 4 e 10 representam partes do programa que foram omitidas):

```
1 <!DOCTYPE html>
2 <html>
```

```
3 <body>
4 <button _____="mostra()">Mostrar
6 dia e hora</button>
7 <script>
8 function mostra() {
9   document.getElementById("teste").
10 innerHTML = _____;
11 }
12 </script>
13 <p id="teste"></p>
14 </body>
```

Supõe-se que esse programa será aberto por um navegador como o Chrome ou o Edge. Deseja-se que ele apresente na tela um botão que, ao ser pressionado, exiba a data e hora no formato:

Sun Mar 01 2020 23:20:11 GMT-0300 (Horário Padrão de Brasília)

Para tanto, os traços deverão ser substituídos, respectivamente, por:

a) onchange e Date()
b) onchange e DayHour()
c) onclick e Date()
d) onclick e Day()Hour()
e) onclick e DayHour()

11. (VUNESP/PREF. MARÍLIA/2023) Um dos tipos de operadores da linguagem JavaScript é o operador de deslocamento. Considere uma variável x inicialmente com o valor 10 (em decimal). Após se realizar a operação

x = x << 2

seu novo valor será

a) 2
b) 5
c) 10
d) 20
e) 40

12. (VUNESP/PREF. MARÍLIA/2023) O código JavaScript

window.alert(2 + 3 + "5");

quando executado exibe uma caixa de diálogo com o seguinte conteúdo:

a) 55

b) 10
c) 0
d) 5
e) 235

13. (FCC/MPE PB/2023) Para mudar, com JavaScript, o conteúdo de um elemento <h1 id= "title"> para "MPEPB123", utiliza-se o comando

a) document.getElementById("title").text = "MPEPB123";
b) document.innerHTML("h1#title") = "MPEPB123";
c) document.getElementById("#title").innerHTML = "MPEPB123";
d) document.text("#title") = "MPEPB123";
e) document.getElementById("title").innerHTML = "MPEPB123";

14. (VUNESP/UNICAMP/2023) Considere o seguinte trecho de código escrito em Javascript:

let str = “Tenha uma boa prova!”;
let x = str.search(“a”);

O valor armazenado na variável x será:

a) –1
b) 0
c) 3
d) 4
e) 5

15. (VUNESP/UNICAMP/2023) Dois dos operadores utilizados na linguagem Javascript são os operadores de incremento/decremento. Considere o seguinte trecho de código:

```
let x = 10;
x = x++;
x = --x;
```

O valor da variável x após a execução do trecho de código será:

a) 8
b) 9
c) 10
d) 11
e) 12

16. (VUNESP/UNICAMP/2023) A linguagem Javascript, ao contrário de outras linguagens, possui apenas uma definição para qualquer tipo de número, utilizando para tal um conjunto de bits para sua representação. O número de bits utilizado no Javascript para representar um número qualquer é:

a) 16

02648216103 - HENRIQUE AMORIM BARRETO

b) 32
c) 64
d) 128
e) 256

17. (VUNESP/UNICAMP/2023) Considere o seguinte trecho de código escrito na linguagem Javascript.

```
var x = 1;
for(var i = 0; i < 3; i++){
  x = x << 1;}
```

É correto afirmar que o valor da variável x após a execução do trecho de código apresentado será:

a) 1
b) 2
c) 3
d) 8
e) 16

18. (VUNESP/EPC/2023) Considere a seguinte linha de código escrita na linguagem Javascript.

```
let x = 5 * 2 >> 1 + 2;
```

É correto afirmar que o valor da variável x após essa execução será:

a) 0
b) 1
c) 5
d) 7
e) 8

19. (FGV/DPE RS/2023) Atualmente sob responsabilidade da ECMA International, o Java Script (ECMA 6) utiliza o padrão de arrow function para a definição de funções, aproximando um pouco mais a linguagem do paradigma funcional.

Considerando uma função para a multiplicação de dois valores, na forma recursiva, sua implementação no padrão arrow function é:

a) let fnx = (a,b) => (b>1) ? a * fnx(a, b-1) : a;
b) let fnx = (a,b) => (b>1) ? b + fnx(a, b-1) : a;
c) let fnx = (a,b) => (a>1) ? b * fnx(a-1, b) : a;
d) let fnx = (a,b) => (b>1) ? a + fnx(a, b-1) : a;
e) let fnx = (a,b) => (a>1) ? b * fnx(a, b) : a-1;

02648216103 - HENRIQUE AMORIM BARRETO

20. (CEBRASPE/PREF. FORTALEZA/2023) Julgue o item subsequente, a respeito de HTML5, CSS3 e JavaScript.

O código JavaScript a seguir, ao ser executado, apresenta o valor 3 no console de desenvolvimento do navegador web.

```
<script>
      var cores = [“vermelho”, “verde”, “azul”];
      console.log(cores.length);
</script>
```

21. (VUNESP/DPE SP/2023) Um programador deseja fazer uma busca em uma String utilizando Expressões Regulares na linguagem Javascript. Dada uma String texto, ele deseja verificar a posição da String em que são encontrados três dígitos, seguidos de um espaço em branco (whitespace), seguidos de pelo menos um caractere a. Para tal, ele pode escrever a linha de código

a) let indice = texto.search(/3\d\sa*/);
b) let indice = texto.search(/000\sa+/);
c) let indice = texto.search(/000_a*/);
d) let indice = texto.search(/\d\d\d_a*/);
e) let indice = texto.search(/\d\d\d\sa+/);

22. (FGV/ALEMA/2023) Analise o código JavaScript a seguir.

```
function enigma(n) {
  if (n == 0 || n == 1) {
    return n
} else {
    return enigma (n-1) + enigma (n-2)
}}
```

Assinale o valor retornado para a expressão enigma(4).

a) 21
b) 13
c) 8
d) 5
e) 3

23. (FGV/ALEMA/2023) Os trechos JavaScript a seguir apresentados funcionam corretamente, à exceção de um. Assinale-o.

a)
```
bomDia = function() {
return "Bom dia!";
}
alert(bomDia());
```
b)
```
bomDia = () => {
return "Bom dia!";
```


02648216103 - HENRIQUE AMORIM BARRETO

```
    }
    alert(bomDia());
```
c)
```
    x = "!";
    bomDia = (val) => "Bom dia" + val;
    alert(bomDia(x));
```
d)
```
    bomDia => "Bom dia!";
    alert(bomDia);
```
e)
```
    bomDia = (a) => a + "!";
    alert(bomDia("Bom dia"));
```

24. (FGV/ALEMA/2023) Analise o código JavaScript a seguir.

```
var x = document.getElementById("xpto");
while (x.hasChildNodes()) {
x.removeChild(x.childNodes[0]);}
```
O efeito da execução desse trecho é

a) a remoção de todos os elementos subordinados ao elemento cujo id é “xpto”.
b) um erro de execução, pois “xtpo” é usado equivocadamente onde deveria haver uma referência numérica.
c) um erro de execução, pois o índice 0 não se aplica no HTML.
d) m erro de sintaxe, pois a declaração da variável “x” deveria explicitar o tipo element.
e) um erro de sintaxe, pois o método removeChild é inválido.

25. (CEBRASPE/DATAPREV/2023) Julgue o item abaixo, relacionados com JavaScript, Web Services e análise estatística de código-fonte.

Para garantir uma correta compilação de um código escrito em JavaScript, é necessário que as variáveis sejam definidas e inicializadas antes de seu uso no código.

26. CEBRASPE/DATAPREV/2023) Considerando a linguagem de programação JavaScript, julgue o item a seguir.

Caso o código a seguir seja executado, o console.log() mostrará que a variável inicio tem valor igual a 1.

```
const inicio = 0;
const fim = 10;
if (inicio >= fim) {
inicio = fim;
} else {
        inicio++;
}
console.log(inicio);
```

27. (FGV/MPE SC/2023) No contexto do JavaScript, analise um trecho do HTML de uma página web.
```
<div id="xpto">
        <p>Texto de teste 1</p>
```


02648216103 - HENRIQUE AMORIM BARRETO

```
        <p id="234">Texto de teste 2</p>
</div>
```

A linha de código que altera o tamanho da fonte do segundo parágrafo acima é:

a) document.getElementById("234").style.fontSize="48px";
b) document.getElementById("xpto.234").style.fontSize="48px";
c) document.getElementById(p<"234">).style.fontSize="48px";
d) document.getElementsById("div.p").style.fontSize="48px";
e) xpto.getElementById("234").style.fontSize="48px";

28. (FGV/TCE SP/2023) Leandro precisa escrever uma mensagem dinâmica em uma div, através do Java Script, e sabe que deve ser utilizado DOM, ou Document Object Model, para acessar os elementos de sua página.

Considerando que o id utilizado na div é "msgOut", e que a variável msg contém a mensagem, Leandro irá utilizar o comando:

a) document["msgOut"].innerHTML=msg;
b) document.getElementById("msgOut").innerHTML=msg;
c) document.setHtmlContent("msgOut",msg);
d) document["msgOut"].html(msg);
e) document.setElementById("msgOut",msg);

29. (CEBRASPE/DPE RO/2023)

```
int a = 3;
int b = 7;
int c;
c = ++a + (--a * b++);
```

A execução do código apresentado, escrito em JavaScript, resulta na variável c com o valor

a) 14.
b) 32.
c) 25.
d) 24.
e) 16.

30. (FGV/SEFAZ AM/2022) Arrays são estruturas de dados que armazenam uma coleção de elementos.

A maneira apropriada para criar um array de elementos em JavaScript é

a) const meuarray = new Array ["maçã", "laranja", "2"]
b) const meuarray = {"maçã", "laranja", "2"}
c) const meuarray = ["maçã", "laranja", 2]
d) const meuarray = ("maçã", "laranja", 2)
e) const meuarray := [2]string{"maçã", "laranja"}

31. (VUNESP/UFABC/2023) Considere a seguinte linha de código escrita na linguagem Javascript:

```
let x = 10 >> 1
```

É correto afirmar que o valor da variável x, após a execução dessa linha, será:

a) true
b) false
c) 5
d) 10
e) 20

32. (CESGRANRIO/TRANSPETRO/2023) Um desenvolvedor web está trabalhando em uma aplicação que envolve a coleta de dados numéricos dos usuários, e, para isso, está usando HTML e JavaScript (ECMAScript 2020) para criar um campo de entrada de texto e um botão. Quando um usuário clicar nesse botão, o valor inserido será processado. Nesse cenário, considere que esse desenvolvedor quer obter o valor atual do campo de texto HTML com o ID idade. Para obter esse valor, ele deve utilizar o seguinte fragmento de código JavaScript:

a) document.getElementById(“idade”).getText()
b) document.getElementById(“idade”).innerHTML
c) document.getElementById(“idade”).value
d) document.getElementByName(“idade”).value
e) document.querySelector(“#idade”).text

33. (FGV/TCE SP/2023) Leandro precisa escrever uma mensagem dinâmica em uma div, através do Java Script, e sabe que deve ser utilizado DOM, ou Document Object Model, para acessar os elementos de sua página.

Considerando que o id utilizado na div é "msgOut", e que a variável msg contém a mensagem, Leandro irá utilizar o comando:

a) document["msgOut"].innerHTML=msg;
b) document.getElementById("msgOut").innerHTML=msg;
c) document.setHtmlContent("msgOut",msg);
d) document["msgOut"].html(msg);
e) document.setElementById("msgOut",msg);

34. (FGV/TCE SP/2023) Jorge é consultor em uma empresa de desenvolvimento e está analisando o código a seguir, na linguagem JavaScript, encontrado em uma das páginas do site.

```
const va = [1,2,5,7,12,14,19,20];
const vb = [], vc = [];

for(let a of va)
        if(a%2==0)
```


02648216103 - HENRIQUE AMORIM BARRETO

```
        vb.push(a);
    else
        vc.push(a);

console.log(vb.reduce((a,b)=>a+b));
console.log(vc.reduce((a,b)=>a+b));
```

Jorge abriu a página e viu impressos no console de depuração, respectivamente, os valores:

a) 1 e 20;
b) 1 e 79;
c) 15 e 65;
d) 48 e 32;
e) 80 e 0.

35. (FGV/TCE SP/2023) Beatriz tem um site com diversos elementos do tipo div, onde foi utilizada uma classe CSS com o nome "opcional", e deseja que eles fiquem ocultos quando forem clicados. Para adicionar a funcionalidade em todos os elementos via JavaScript, ela desenvolveu a função apresentada a seguir.

```
const associar = (elementos) => {
for(let obj of elementos)
obj.addEventListener('click',(e)=>{
e.target.style.display='none'});
}
```

Para executar a função criada, associando corretamente aos elementos do tipo div, Beatriz deve utilizar para o parâmetro elementos a expressão:

a) document.getElementsByTagName("div.opcional");
b) document.getElementsByClassName("opcional");
c) document.getElementsByName("div:opcional");
d) document.getElementsByClass("opcional");
e) document.getElementsById("div.opcional").

36. (FGV/CÂMARA DOS DEPUTADOS/2023) Assinale o método JavaScript que retorna o número de caracteres em uma string.

a) charAt()
b) count
c) len()
d) length
e) size

37. (FGV/CÂMARA DOS DEPUTADOS/2023) Considere o script JavaScript a seguir.

```
function f(f) { return (f*f); }

let value1 = f;
alert(value1);

let value2 = f(100);
alert(value2);

let value3 = f;
alert(value3(100));
```

Assinale os valores exibidos na execução do script acima, na ordem.

a) function f(f) { return (f*f); }
   10000
   10000
b) function f(f) { return (f*f); }
   10000
   undefined
c) undefined
   10000
   undefined
d) 10000
e) undefined
   undefined
   undefined

38. (FGV/SEDUC SP/2023) Considere o trecho a seguir quando utilizado numa página Web.

```
<body onload="alert('xxxxxxx')">
```

O efeito desse trecho é

a) a exibição automática da mensagem xxxxxxx imediatamente antes que a página seja carregada.
b) a exibição automática da mensagem xxxxxxx imediatamente antes que um formulário seja carregado.
c) a exibição automática da mensagem xxxxxxx imediatamente depois que a página foi carregada.
d) o início imediato de uma operação de upload.
e) um erro, pois o script contém incompatibilidades.

39. (FGV/TJ SE/2023) Considere o script JavaScript a seguir.

```
let x = 16;
let y = [];

y[2] = x;
y[2] %= 3;
```

```
y[0] = x<<2;
y[1] = x <<= 5;

alert(y);
```

A execução desse script exibe:

a) 0,128,1
b) 2,512,32
c) 64,true,1
d) 64,512,1
e) 128,64,2

40. (FGV/TJ SE/2023) Analise o código JavaScript a seguir.

```
const valores = [4, 3, 2, 1, 0];
let xpto = "";
valores.forEach(fF);
alert (xpto);
function fF(value, index) {
  xpto += index + ":" + value + ">";}
```

A execução desse código exibe:

a) 4:4>
b) 4:0>3:1>2:2>1:3>0:4>
c) 0:4>1:3>2:2>3:1>4:0>
d) 0:0>
e) 0:0>1:1>2:2>3:3>4:4>

41. (FGV/TJ SE/2023) Observe o seguinte trecho de código-fonte em JavaScript:

```
const array = ["Um", "Dois", "Tres"];
array.unshift("Tres");
array.splice(-1);
```

O valor final do array é:

a) [“Um”];
b) [“Dois”];
c) [“Um”, “Dois”];
d) [“Um”, “Tres”, “Tres”];
e) [“Tres”, “Um”, “Dois”].

42. (FGV/MPE GO/2022) Assinale o trecho de código JavaScript que define e ordena corretamente um array.

a) const fruits = ["Casa"; "Apartamento"; "Sítio"]; fruits.sort();

b) const fruits = ["Casa", "Apartamento", "Sítio"]; fruits.sort();
c) const fruits = {"Casa"; "Apartamento"; "Sítio"}; fruits.sort();
d) const fruits = ["Casa", "Apartamento", "Sítio"]; array.sort(fruits);
e) const fruits = ["Casa", "Apartamento", "Sítio"]; array.sort(fruits);

43. (FGV/TJDFT/2022) Na página web que Maria está desenvolvendo, há um botão que, quando clicado, exibe data e hora correntes.

O código que implementa essa característica corretamente é:

a) <button id="xpto" onclick= get("xpto").innerHTML=Date()">...</button>
b) <button onclick="button.innerHTML=Date()">...</button>
c) <button onclick="this.innerHTML=Date()">...</button>
d) <button onclick=exhibit Date()>...</button>
e) <button onclick="function() return Date()">...</button>

44. (FGV/MPE SC/2022) Analise o código JavaScript exibido a seguir.

```
var numeros = [4,9,16];
var xpto = numeros.map(function(num) {
return num << 2;
});
```

Após a execução desse trecho de código, os valores de numeros e xpto são, respectivamente:

a) undefined
   [16,36,64]
b) [4,9,16]
   [16,36,64]
c) null
   [0,1,4]
d) [4,9,16]
   Null
e) [16,36,64]
   Undefined

45. (FGV/SEMSA MANAUS/2022) No âmbito de uma página Web, analise o trecho de código JavaScript a seguir.

```
const points = [40, 100, 1, 5, 25, 10];
points.sort(function(a, b){return a - b});
alert(points);
```

A execução do código acima deverá provocar

a) um erro de compilação na segunda linha.
b) um erro de execução na terceira linha.

c) a exibição dos valores de points na ordem em que foram declarados.
d) a exibição dos valores de points na ordem numérica crescente.
e) a exibição dos valores de points na ordem numérica decrescente.

Comentários:

Aluno, nós já vimos essa função na aula. Ela é responsável por fazer a ordenação em ordem crescente da nossa array. Ela recebe como parâmetro dois valores – a e b, que correspondem aos valores dos elementos. É feita uma operação de subtração entre cada um, e, caso o resultado seja negativo, a função sort entende que a deve ficar à frente de b; caso seja positivo, a função sort entende que devemos inverter a ordem dos elementos.

Por exemplo, na primeira iteração, teríamos a = 40, b = 100. O resultado a – b seria negativo, -60, passando para a função sort que o elemento a deve permanecer à frente do elemento b. Isso é feito para todas os pares de números, até que não tenhamos mais resultados positivos.

Gabarito: Letra D

---

46. (FGV/SEMSA MANAUS/2022) (FGV/SEMSA MANAUS/2022) No JavaScript, a expressão typeof {nome:'Maria', cargo:'Professora'} retorna

a) array.
b) function.
c) null.
d) object.
e) undefined.

47. (CESGRANRIO/BASA/2022) O tratamento de erros e exceções é, atualmente, uma exigência para as linguagens de programação.

Qual fragmento de código, em Javascript, é um exemplo válido desse tratamento?

a)
```
try {
  adddlert("Bem vindo!");
 } else(err) {
 window.alert(err.message);}
```
b)
```
try {
   adddlert("Bem vindo!");
  } except(err) {
    window.alert(err.message); }
```
c)
```
try {
   adddlert("Bem vindo!");
 } catch(err) {
  window.alert(err.message);}
```
d)
```
try {
   adddlert("Bem vindo!");
```


02648216103 - HENRIQUE AMORIM BARRETO

```
  } catch(err){
    window.alert(err.message);
    } else {
    window.alert("sem erros");}
e) try {
      adddlert("Bem vindo!");
   }  catch(err) {
    window.alert(err.message);
   } otherwise {
     window.alert("sem erros");}
```

48. (CEBRASPE/PETROBRAS/2022) Julgue o próximo item que trata de CSS, JavaScript e Net Core.

Em JavaScript é possível simular cada uma das quatro categorias de membros de classe da linguagem Java.

49. (FGV/MPE GO/2022) No contexto do JavaScript, considere as seguintes afirmativas sobre a declaração de variáveis com let ou var.

I. Variáveis declaradas com let não podem ser redeclaradas no mesmo { } bloco.
II. Variáveis declaradas com let podem ser utilizadas em qualquer trecho do código (escopo global).
III. Variáveis declaradas com var no interior de um { } bloco podem ser utilizadas fora do bloco de origem.
Está correto o que se afirma apenas em

a) I.
b) II.
c) I e II.
d) I e III.
e) II e III.

50. (VUNESP/ALESP/2022) Analise o programa a seguir, que possui um script elaborado em JavaScript. Os pontos assinalados com (1), (2) e (3) e (4) correspondem a informações suprimidas do programa.

```
<!DOCTYPE html>
<html><body>
<script language=JavaScript>
<!--
        horaatual = (1);
        hora = horaatual.(2);
        minutos = horaatual.(3);
        segundos = horaatual.(4);
        document.writeln(" Hora atual: " + hora + ":" +
        minutos + ":" + segundos);
</script>
</body></html>
```

Quando o programa for aberto por um navegador que suporte JavaScript, como o Chrome ou o Edge, é exibida a hora do computador, por exemplo:

02648216103 - HENRIQUE AMORIM BARRETO

Hora atual: 17:43:42

Para que estas exibições ocorram conforme apresentado, (1), (2), (3) e (4) deverão ser, respectivamente:

a) new Hour(), getHours(), getMinutes() e getSeconds().
b) new Date(), Hours(), Minutes() e Seconds().
c) new Hour(), Hours(), Minutes() e Seconds().
d) new Date(), getHours(), getMinutes() e getSeconds().
e) Hour(), getHours(), getMinutes() e getSeconds().

51. (CEBRASPE/FUB/2022)

```
<html>
    <p id=“saida”></p>
    <head>
        <script type=“text/javascript” >
              const user = {
              nome: 'Pedro Maria',
              email: 'pedro.maria@gmail.com',
              idade: 25,
              nascimento: '21/02/1996',
              masculino: true
        };
        document.getElementById(“saida”).
              innerHTML = ““;
        for (const key in user) {
             document.getElementById(“saida”).
             InnerHTML += `$ {key}:
             $ {user[key]}`;
        }
        </script>
    </head>
</html>
```

Considerando o código precedente, em HTML, julgue o item a seguir.

O código em questão apresenta um erro de sintaxe, porque atribui valores de tipos diferentes a uma mesma variável.

52. (FGV/TJDFT/2022) No contexto do JavaScript, analise as afirmativas a seguir a respeito do comportamento do JS de mover declarações para o top (em inglês, Hoisting).

I. Variáveis declaradas por meio de var podem ser utilizadas antes de declaradas.
II. Variáveis declaradas com let e const são movidas para o topo do bloco somente quando inicializadas na declaração.
III. Usar uma constante (const) antes de declarada gera um erro de sintaxe.

02648216103 - HENRIQUE AMORIM BARRETO

Está correto o que se afirma em:

a) somente I;
b) somente I e II;
c) somente I e III;
d) somente II e III;
e) I, II e III.

53. (FGV/MPE SC/2022) No JavaScript, as três funções nativas que permitem uma rápida comunicação com o usuário de uma página web são:

a) alert, confirm, input;
b) alert, input, output;
c) exhibit, read, write;
d) input, msgbox, print;
e) read, ring, output.

54. (FGV/TJ TO/2022) Considere o código JavaScript a seguir.

```
var x1
var x2
var x3
var y = 11
var z = 12
x1= z++ == ++y
x2= 10 + "casa"
x3= y+1 === z && 10 != "20"
```

Após a execução dessas operações, os valores de x1, x2 e x3 são, respectivamente:

a) false
   10casa
   true
b) false
   10casa
   false
c) false
   undefined
   false
d) true
   10casa
   true
e) true
   undefined
   false

55. (FGV/TJ TO/2022)

```
function xpto(x) {
  return x % 2 == 0;
}
const numeros = [22, 0, 18, 1];
alert (numeros.every(xpto));
```

O valor exibido na execução desse código é:

a) 1
b) 3
c) false
d) true
e) undefined

56. (FGV/TJ TO/2022) No contexto da linguagem JavaScript, analise o trecho a seguir.

```
hello = (val) => "Hello " + val;
```

Essa definição é equivalente ao trecho:

a)
```
function hello(val) {
        return 'Hello ' + val;
   }
```

b)
```
((val == ' ') ? 'Hello ' + val : val)
```

c)
```
function hello(val) {
         if (val != '') {
            return 'Hello ' + val }
        else { return '' }
  }
```

d)
```
function hello(val) {
        return val;
   }
```

e)
```
((val != ' ') ? val : '')
```

57. (FGV/SEMSA MANAUS/2022) Considere o comando JavaScript a seguir.

```
const teste = new Xpto("a", 50);
```

Assinale a opoção que indica a definição correta para Xpto.

02648216103 - HENRIQUE AMORIM BARRETO

a) const array Xpto[ ];
b) const object Xpto;
c) const Xpto = [ ];
d) function Xpto(p1, p2) {this.x1 = p1;this.x2 = p2};
e) Xpto = {x1:undefined, x2:undefined};

58. (VUNESP/CAMPINAS/2022) Um programador da linguagem JavaScript utilizou em seu programa, os seguintes comandos:

```
...
if (true == "true")
...
if (true === "true")
...
```

Em relação a esses comandos, é correto afirmar que

a) ambos são equivalentes, resultando no mesmo resultado.
b) o primeiro comando, ao ser executado, retornará o valor false.
c) o segundo comando, ao ser executado, retornará o valor false.
d) o segundo comando não será executado, originando um erro.
e) o segundo comando significa “não idêntico a”.

59. (FCC/TRT 23/2022) Considere a página web com o código JavaScript abaixo.

```
<!DOCTYPE html>
<html>
  <body>
    <script>
      let organization = "TRIBUNAL REGIONAL DO TRABALHO";
      for (...I...) {
       document.write(x + "<br>");
      }
    </script>
  </body>
</html>
```

Para exibir `TRIBUNAL REGIONAL DO TRABALHO` na vertical, com uma letra por linha, a lacuna I deve ser preenchida corretamente com

a) let x = organization.get(char)
b) let x of organization
c) let i=0; i<organization.length; x = organization.text[i]; i++
d) let x in organization
e) get x in organization

60. (VUNESP/PREF. SOROCABA/2022) Em um programa elaborado em JavaScript, incluiu-se a seguinte estrutura condicional, na qual X e Y representam comandos removidos:

```
switch (semaforo) {
        case "verde":
                alert("siga")
                X
        case "amarelo":
                alert("atencao")
               X
        case "vermelho":
                alert("pare")
                X
         Y:
                 alert("cor invalida")
}
```

Os comandos X e Y que devem ser colocados para que a estrutura condicional seja executada adequadamente são, respectivamente,

a) break e default.
b) break e end.
c) end e close.
d) stop e close.
e) stop e default.

61. (FGV/TCE TO/2022) No contexto de HTML e JavaScript, na construção

```
function myFunction() {
  document.getElementById("xxxx").submit();
}
```

o elemento xxxx deve identificar:

a) o path completo de um arquivo;
b) um elemento da classe EntryData;
c) um Form object;
d) um link externo;
e) uma página Web.

62. (FGV/TCE TO/2022) Analise o script JavaScript a seguir.

```
<script>
.....
let s = "";
```


02648216103 - HENRIQUE AMORIM BARRETO

```
x.forEach(f);
alert(s);
function f(value, index, array) {
  s += index + " " + value;
}
</script>
```

Dado que, na execução desse script, o comando alert exibiu os valores

0 451 42 93 164 25

é correto afirmar que o trecho pontilhado foi substituído por:

a) const x = [1, 451, 42, 93, 164, 25];
b) const x = [45, 4, 9, 16, 25];
c) let x = [45, 1, 42, 93, 16, 42, 5];
d) var x = [451, 42, 93, 164, 25];
e) var x = [451, 4293, 93, 425];

63. (FGV/CM Taubaté/2022) No contexto de paginas Web, analise a linha de código JavaScript a seguir,.

```
<button onclick="this.innerHTML = Date()">The time is?</button>
```

É correto afirmar que, quando o botão é clicado,

a) um campo de input é apresentado, no qual deve ser digitada uma data/hora.
b) o botão é preenchido com a data/hora corrente.
c) é exibida uma mensagem com a data/hora corrente.
d) cada elemento da página com a tag <innerHTML> é preenchido com a data/hora corrente.
e) qualquer elemento HTML da página que esteja selecionado é preenchido com a data/hora corrente.

64. (FGV/TRT 13/2022) Considere o código Javascript exibido a seguir.

```
<script>
        var x = 10;
        {
        var x = 2;
        }
        alert( x);
</script>
```

A execução desse script provoca

a) a exibição do valor 10.
b) a exibição do valor 12.
c) a exibição do valor 2.
d) um erro de execução.

e) um erro de sintaxe.

65. (FGV/TRT 13/2022) Considere o código Javascript exibido a seguir.

```
<script>
const registro =
{nome:"Carlos", cidade:"São Paulo", idade:42};
let texto = "";
for (let r in registro) {
texto += registro[r] + " ";
}
alert(texto);
</script>
```

A execução desse script provoca

a) a exibição da string “Carlos São Paulo 42”.
b) a exibição da string “Carlos”.
c) a exibição de uma string vazia.
d) um erro de execução.
e) um erro de sintaxe.

66. (FGV/SEAD AP/2022) Considere uma página Web que contém o código JavaScript a seguir.

```
<script>
let text = "";
for (let i = 0; i < 5; i++) {
  text += " " + i ;
}
alert(text);
</script>
```

O valor exibido pela execução desse script é

a) 0 0 0 0
b) 4
c) 0 4
d) 1 1 1 1 1
e) 0 1 2 3 4

67. (FCC/TRT 14/2022) Dada a instrução JavaScript const d = new Date();, para se obter e exibir o dia do mês (entre 1 e 31), utiliza-se a instrução

a) document.write(d.getDate().getDay());
b) document.write(d.getDate());
c) document.write(d.getDay());
d) document.println(d.getDate());
e) document.print(d.getDay());

02648216103 - HENRIQUE AMORIM BARRETO

68. (CEBRASPE/TRT 8/2022)

```
<html>
 <script>
console.log(('a'+ +'b'+'a'+'s').toLowerCase());
 </script>
</html>
```

Assinale a opção que corresponde ao resultado do código JavaScript anterior.

a) abas
b) ananbas
c) ananas
d) anas
e) bananas

69. (FCC/TRT 17/2022) Utilizando JavaScript, em condições ideais, um analista converteu os dados de Paulo Silva, de 50 anos, com renda de 1450.45, para string JSON, no formato abaixo.

```
{"nome":"Paulo Silva","idade":50,"renda":1450.45}
```

Tal conversão pode ser realizada por meio da instrução

a) JSON.toStrings({nome: 'Paulo Silva', idade: 50, renda: 1450.45 })
b) document.toJSON("{nome: 'Paulo Silva', idade: 50, renda: 1450.45 }")
c) JSON.stringify({nome:new String('Paulo Silva'), idade: 50, renda: 1450.45})
d) document.parseToJSon("{nome: 'Paulo Silva', idade: 50, renda: 1450.45 }")
e) JSON.parse({nome:new String('Paulo Silva'), idade: 50, renda: 1450.45 })

70. (VUNESP/CM FERNANDÓPOLIS/2022) Na linguagem Javascript, existem diversos métodos que podem ser utilizados para a obtenção de informação sobre um objeto de data, como o que obtém o tempo, em milissegundos, transcorrido desde 01.01.1970.

Esse método é denominado:

a) get FullTime
b) getDate()
c) getFullYear()
d) getMilliseconds()
e) getTime()

71. (FGV/PC AP/2022) Analise o código JavaScript a seguir.

```
const cidades =
      ["Natal", "Manaus", "Recife", "Brasília"];
cidades.reverse();
alert (cidades);
```

Assinale o que é exibido na execução do trecho acima.

a) Brasília,Manaus,Natal,Recife
b) Brasília,Natal,Manaus,Recife
c) Brasília,Recife,Manaus,Natal
d) Manaus,Recife,Brasília,Natal
e) Recife,Natal,Manaus,Brasília

72. (VUNESP/PREF. PIRACICABA/2022) A linguagem JavaScript apresenta algumas características, como:

a) a palavra chave var pode ser usada para declarar tanto variáveis locais como variáveis globais.
b) JavaScript não é case-sensitive, de forma que a variável Casa também pode ser escrita como casa ou cASA, ou ainda admite outras variações.
c) JavaScript não é uma linguagem dinamicamente tipada, de forma que se precisa especificar o tipo de dado de uma variável quando declará-la.
d) null é uma palavra-chave que indica valor nulo e pode ser escrita de diversas formas, como Null, NULL, ou ainda outra variação.
e) uma string literal consiste em zero ou mais caracteres dispostos em aspas duplas ("), e aspas simples (') não podem ser utilizadas em uma string, pois acarretará erro.

73. (VUNESP/UNICAMP/2022) Na linguagem JavaScript, quando um evento do mouse é acionado, a propriedade do objeto MouseEvent que retorna a coordenada horizontal do ponteiro do mouse relativa ao documento é

a) offsetX
b) pageX
c) screenX
d) shiftKey
e) movementX

74. (VUNESP/UNICAMP/2022) A Linguagem JavaScript permite diversos tipos de eventos (events), como o

a) onBlur: o evento ocorre quando um objeto perde o foco.
b) onChange: o evento ocorre quando um formulário foi enviado.
c) onKeyPress: o evento ocorre quando o usuário pressiona a tecla Enter do teclado.
d) onMouseOut: o evento ocorre quando o ponteiro do mouse é movido sobre um elemento.
e) onSubmit: o evento ocorre quando valor é digitado, seguido da tecla Enter.

75. (VUNESP/TJM SP/2021) Em um console JavaScript, um usuário digitou a instrução:

typeof 10

Como resultado da execução dessa instrução, foi impresso:

a) "number"
b) "string"
c) "int"

d) "byte"
e) "char"

Comentários:

A função typeof é usada para verificar o tipo de dado de um valor. No caso, 10 é um número e, no JS, retratamos números (inteiros e de ponto flutuante) como *number*.

Gabarito: Letra B

---

76. (VUNESP/TJM SP/2021) Considere o trecho JavaScript a seguir.

```
var x = 0;
for (var i = 10; i < 15; i++) {
x += i;
}
```

Ao término da execução desse código, o conteúdo da variável “x” será:

a) 0
b) 10
c) 15
d) 60
e) 75

77. (FGV/IMBEL/2021) Analise o código JavaScript exibido a seguir.

```
function xpto (n) {
        var text = "";
        var i = 1;
        while (i <= n) {
                text += i + ';';
                i++;
        }
    return text;
}
```

Num comando JS, a expressão

```
  xpto(5)
```

retorna

a) 54321
b) 12345
c) 1;2;3;4;5;
d) 5;4;3;2;1;

02648216103 - HENRIQUE AMORIM BARRETO

e) 1;2;3;4;5;4;3;2;1;

78. (FGV/IMBEL/2021) Analise o código JavaScript exibido a seguir.

```
function f1 (n) {
        if (n % 2 == 0) {
        return --n; }
    else {
       return ++n; }
}
```

Em um comando JS, a expressão f1(8) retorna

a) -8
b) -7
c) 7
d) 8
e) 9

79. (FGV/TCE AM/2021) Considere o código JavaScript exibido logo abaixo.

```
var x1 = 16;
var x2 = "16";
var x3 = [1,2,3];
```

Usando as definições acima, analise as expressões a seguir.

```
x3[0] == 1
x1 == x2
x1 === x2
true == x1
true == x3[0]
```

Respectivamente, os valores retornados pelas expressões acima são:

a) false, false, false, true, true;
b) false, true, false, false, false;
c) false, true, true, true, false;
d) true, true, false, false, true;
e) true, true, true, false, true.

80. (CEBRASPE/SEED PR/2021) JavaScript trabalha com números usando os operadores aritméticos fornecidos pela própria linguagem. No entanto, a linguagem aceita operações matemáticas mais complexas por meio de um conjunto de funções e constantes definidas como propriedades do objeto Math.

02648216103 - HENRIQUE AMORIM BARRETO

Assinale a opção que apresenta a função que permite realizar, em JavaScript, a operação matemática de arredondar para cima o número 1,17, obtendo-se o valor 2.

a) Math.ceil()
b) Math.exp()
c) Math.pow()
d) Math.round()
e) Math.sqrt()

02648216103 - HENRIQUE AMORIM BARRETO

GABARITO

GABARITO



1. Correto
2. Letra C
3. Letra B
4. Letra D
5. Letra C
6. Letra D
7. Letra C
8. Letra B
9. Letra B
10. Letra C
11. Letra E
12. Letra A
13. Letra E
14. Letra E
15. Letra B
16. Letra C
17. Letra D
18. Letra B
19. Letra D
20. Correto
21. Letra E
22. Letra E
23. Letra D
24. Letra A
25. Errado
26. Errado
27. Letra A
28. Letra B
29. Letra C
30. Letra C
31. Letra C
32. Letra B
33. Letra B
34. Letra D
35. Letra B
36. Letra D
37. Letra A
38. Letra C
39. Letra C
40. Letra C
41. Letra E
42. Letra B
43. Letra B
44. Letra B
45. Letra D
46. Letra D
47. Letra C
48. Correto
49. Letra D
50. Letra D
51. Errado
52. Letra C
53. Letra A
54. Letra D
55. Letra C
56. Letra C
57. Letra D
58. Letra C
59. Letra B
60. Letra A
61. Letra C
62. Letra B
63. Letra B
64. Letra C
65. Letra A
66. Letra E
67. Letra B
68. Letra C
69. Letra C
70. Letra E
71. Letra C
72. Letra A
73. Letra B
74. Letra B
75. Letra B
76. Letra D
77. Letra C
78. Letra C
79. Letra D
80. Letra A

ESSA LEI TODO MUNDO CONHECE:
PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.

1. 

Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. 

Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. 

Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. 

Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. 

Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. 

Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. 

Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. 

O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.